LAPORAN TUGAS AKHIR

IMPLEMENTASI *COMPOSABLE ARCHITECTURE* BERBASIS *STRATEGY* DAN *DECORATOR PATTERNS* PADA ALGORITMA *FUZZY* C-MEANS UNTUK KLASTERISASI KINERJA KARYAWAN

SKRIPSI

Diajukan Sebagai Pedoman Pelaksanaan Penilaian Tugas Akhir pada Jurusan Informatika Fakultas Teknik Universitas Jenderal Soedirman

Oleh:

GHAZA INDRA PRATAMA

H1D022073

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS TEKNIK

JURUSAN INFORMATIKA

PURWOKERTO

2026

untuk menilai kinerja masing-masing jabatan, di mana Driver dinilai berdasarkan aspek teknis operasional seperti jumlah ritase, efisiensi bahan bakar, dan keselamatan berkendara, sementara Staff Accounting dinilai berdasarkan aspek keuangan seperti akurasi data dan kepatuhan pajak. Perbedaan variabel penilaian antar jabatan inilah yang menjadi dasar penerapan *Strategy Pattern* dan *Decorator Pattern* dalam sistem, di mana variabel KPI yang berlaku untuk setiap karyawan ditentukan secara dinamis berdasarkan jabatan dan departemennya masing-masing.

Untuk mengakomodasi heterogenitas tersebut, variabel KPI dalam sistem diorganisasikan ke dalam tiga cakupan *assignment*, yaitu cakupan global yang berlaku untuk seluruh karyawan tanpa terkecuali, cakupan departemen yang berlaku untuk seluruh karyawan dalam satu departemen tertentu, dan cakupan jabatan yang berlaku eksklusif untuk satu jabatan tertentu. Kombinasi ketiga cakupan ini memungkinkan sistem menangani variasi variabel penilaian antar jabatan secara fleksibel tanpa perlu merombak struktur kode program ketika terjadi perubahan konfigurasi penilaian.

4.2 Perancangan Sistem

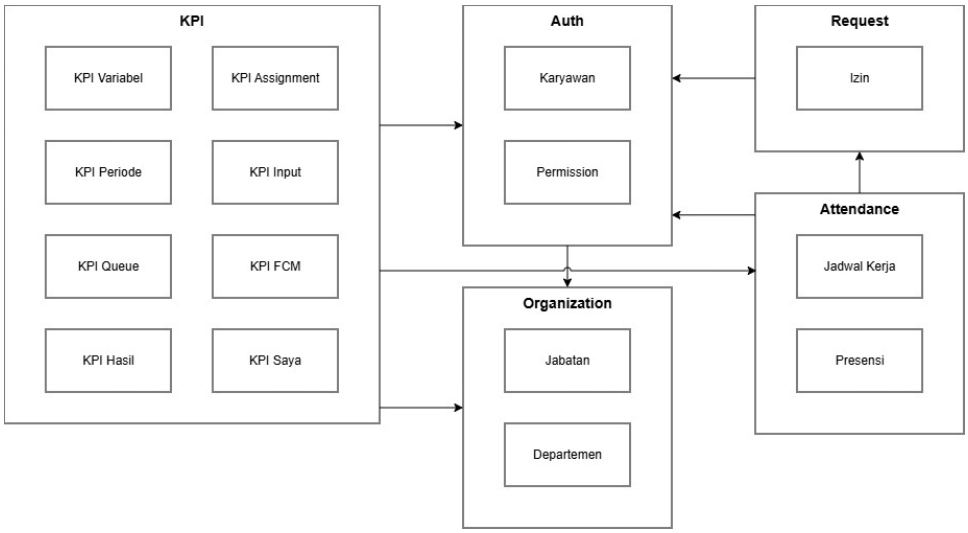
Tahap perancangan sistem dilakukan berdasarkan hasil analisis kebutuhan untuk menyusun cetak biru sistem sebagai panduan teknis sebelum implementasi, dengan fokus pada desain *Composable Architecture* menggunakan *framework* NestJS dengan pendekatan *Modular Monolith*, di mana setiap domain bisnis dipisahkan ke dalam modul-modul yang terisolasi namun tetap berjalan dalam satu kesatuan aplikasi. Pemodelan sistem dilakukan menggunakan *Unified Modeling Language* (UML) yang meliputi *Use Case Diagram*, *Activity Diagram*, *Sequence Diagram*, dan *Class Diagram* untuk menggambarkan kebutuhan fungsional, alur proses, dan struktur kelas, sementara struktur penyimpanan data dirancang menggunakan *Entity Relationship Diagram* (ERD) untuk memastikan integritas dan konsistensi data antar modul. Perancangan juga mencakup penyusunan *mockup* antarmuka pengguna serta desain arsitektur inti sistem yaitu *Composable Architecture*,

Strategy Pattern, *Decorator Pattern*, dan algoritma *Fuzzy C-Means* yang menjadi fokus utama penelitian ini.

4.2.1 Perancangan *Composable Architecture*

Sistem evaluasi kinerja karyawan ini dirancang menggunakan pendekatan *Composable Architecture* dengan pola *Modular Monolith*. Pendekatan ini dipilih karena mampu menggabungkan kesederhanaan arsitektur monolitik dengan keunggulan modularitas, di mana setiap domain bisnis dipisahkan ke dalam modul-modul yang terisolasi namun tetap berjalan dalam satu kesatuan aplikasi. Berbeda dengan *Microservices* yang memisahkan modul ke dalam proses dan infrastruktur yang berbeda, *Modular Monolith* memisahkan tanggung jawab di level kode melalui mekanisme *Dependency Injection* NestJS tanpa *overhead* komunikasi jaringan antar modul.

Sistem ini terdiri dari lima kelompok modul sebagaimana digambarkan pada arsitektur modular sistem pada Gambar 5 berikut.



Gambar 5. Arsitektur Modular Sistem

Sistem ini terdiri dari beberapa kelompok modul pendukung. Modul *auth* bertanggung jawab atas autentikasi, manajemen data karyawan, dan sistem *permission*, di mana proses *login* dan *logout* menghasilkan *JWT token* yang digunakan sebagai *guard* oleh seluruh modul, sementara sistem *permission*-nya menentukan hak akses setiap pengguna berdasarkan jabatan sehingga HR, Atasan, dan Karyawan hanya dapat mengakses fitur sesuai

perannya. Modul *organization* mengelola struktur organisasi perusahaan melalui sub-modul jabatan dan departemen, yang menjadi fondasi mekanisme *Strategy Pattern* dan *Decorator Pattern* pada modul KPI. Modul *attendance* mengelola jadwal kerja dan presensi karyawan, di mana data presensi menjadi sumber data otomatis untuk variabel KPI global yang diambil oleh modul *kpi-datasource* saat periode penilaian dibuka. Modul *request* mengelola pengajuan izin karyawan yang berpengaruh langsung terhadap data presensi, yang selanjutnya mempengaruhi nilai variabel presensi dalam perhitungan skor akhir FCM.

Modul *kpi* merupakan inti dari penelitian ini dan terdiri dari sembilan sub-modul yang mengikuti alur *lifecycle* penilaian kinerja. Modul *kpi-variabel* menyimpan definisi variabel KPI yang dapat dikonfigurasi secara dinamis, sedangkan modul *kpi-assignment* menghubungkan variabel ke tiga cakupan (global, departemen, dan jabatan) sekaligus memvalidasi total bobot setiap jabatan sebelum periode dapat dibuka. Modul *kpi-datasource* menyediakan *registry* sumber data otomatis, modul *kpi-periode* mengelola *lifecycle* periode penilaian mulai dari pembuatan hingga penutupan yang memicu proses klasterisasi, dan modul *kpi-input* mengimplementasikan *Strategy Pattern* serta *Decorator Pattern* untuk menentukan variabel KPI yang berlaku bagi setiap karyawan secara dinamis. Modul *kpi-queue* menerima *trigger* dari *kpi-periode* saat periode ditutup dan menjalankan proses klasterisasi secara *asynchronous* melalui *Bull Queue*, modul *kpi-fcm* menjalankan algoritma *Fuzzy C-Means* dan menghitung *Silhouette Coefficient*, modul *kpi-hasil* menyimpan dan menampilkan hasil klasterisasi kepada HR dan Atasan, sementara modul *kpi-saya* memungkinkan karyawan melihat hasil klasterisasi kinerjanya sendiri berdasarkan identitas yang sedang *login*.

Penerapan *Composable Architecture* dalam sistem ini tercermin dari dua aspek utama, yaitu variabel KPI, bobot, dan konfigurasi parameter FCM dapat diubah melalui konfigurasi *database* tanpa perlu mengubah kode program, serta logika pemilihan variabel penilaian dirancang menggunakan *Strategy Pattern* sehingga penambahan logika baru untuk jabatan baru

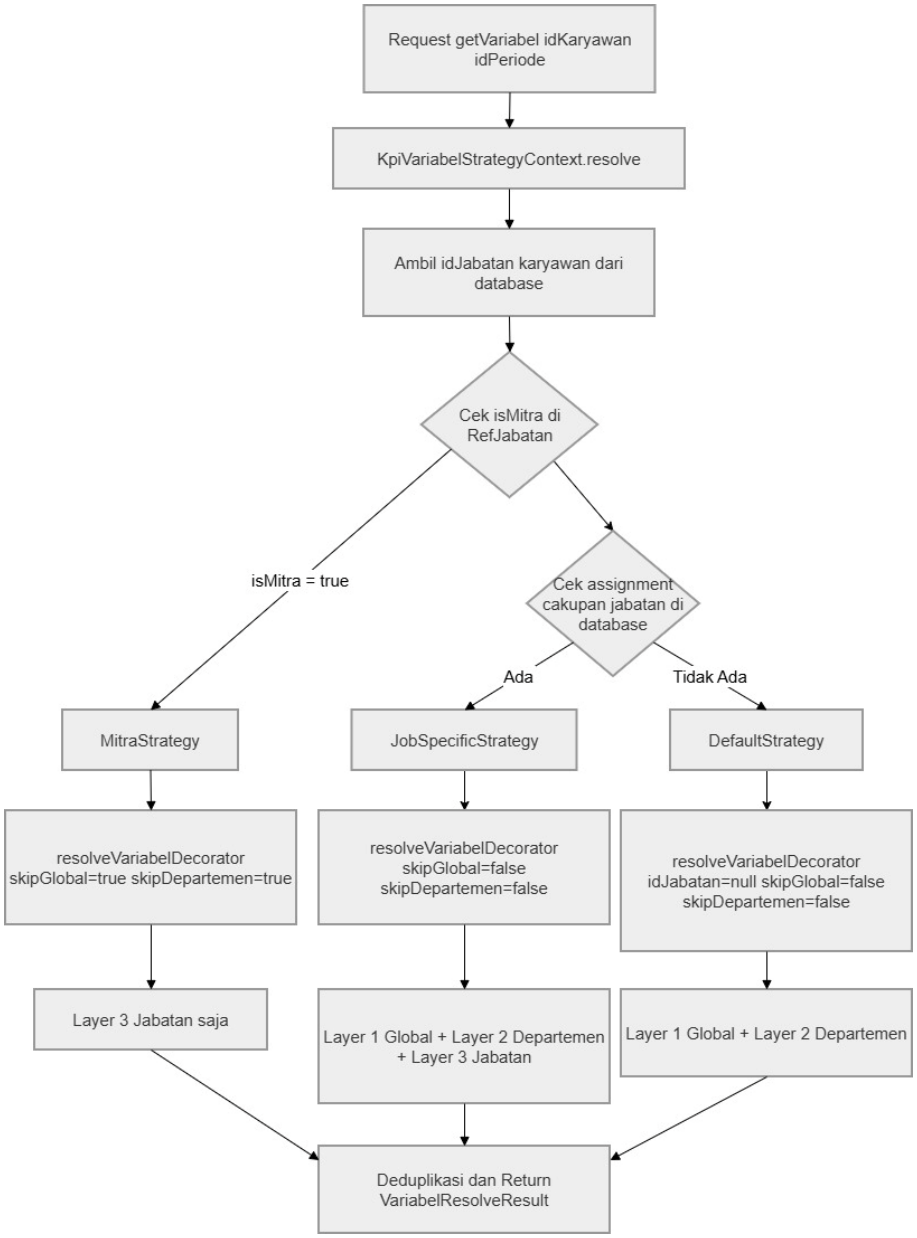
cukup dilakukan dengan membuat *concrete strategy* baru tanpa menyentuh kode yang sudah ada. Kedua aspek ini menjadikan sistem adaptif terhadap perubahan struktur organisasi dan kebutuhan bisnis tanpa memerlukan perombakan arsitektur secara keseluruhan.

4.2.2 Perancangan *Strategy Pattern*

Strategy Pattern diterapkan dalam sistem ini untuk menangani heterogenitas logika pemilihan variabel KPI antar jabatan secara dinamis. Tanpa *Strategy Pattern*, sistem harus menggunakan percabangan kondisional yang kaku untuk menentukan variabel yang berlaku bagi setiap jabatan sehingga penambahan jabatan baru akan selalu membutuhkan perubahan kode program, sedangkan dengan *Strategy Pattern* logika pemilihan variabel dienkapsulasi ke dalam kelas-kelas *strategy* yang dapat dipertukarkan sehingga sistem dapat beradaptasi terhadap perubahan konfigurasi jabatan tanpa menyentuh kode inti.

Sistem ini mengimplementasikan tiga *strategy* yang masing-masing mengimplementasikan *interface IKpiVariabelStrategy* dengan kontrak *method resolve()*. *DefaultStrategy* digunakan untuk jabatan yang tidak memiliki *assignment* cakupan jabatan di *database* dan menjalankan *resolve* variabel dengan dua *layer* yaitu global dan departemen, *JobSpecificStrategy* digunakan untuk jabatan karyawan tetap yang memiliki *assignment* cakupan jabatan di *database* dan menjalankan *resolve* variabel dengan tiga *layer* yaitu global, departemen, dan jabatan, sedangkan *MitraStrategy* digunakan khusus untuk mitra pengemudi yang memiliki *flag isMitra = true* pada data jabatan di *database* dan hanya menggunakan *layer* jabatan tanpa mengikutsertakan *layer* global maupun departemen karena mitra pengemudi dievaluasi berdasarkan parameter operasional yang sepenuhnya berbeda dari karyawan tetap. Pemilihan *strategy* dilakukan secara dinamis oleh *KpiVariabelStrategyContext* berdasarkan data jabatan karyawan di *database*.

Alur pemilihan *strategy* digambarkan pada Gambar 6 berikut.



Gambar 6. *Flowchart* Pemilihan *Strategy*

Ketika atasan membuka halaman penilaian untuk seorang karyawan, sistem memanggil *KpiVariabelStrategyContext.resolve()* dengan parameter *idKaryawan* dan *idPeriode*, kemudian *Context* mengambil data jabatan karyawan dari *database* dan melakukan pemeriksaan secara bertahap. Pemeriksaan pertama mengecek apakah jabatan karyawan memiliki *flag isMitra = true*; jika ya, *Context* memilih *MitraStrategy* yang memanggil *resolveVariabelDecorator* dengan parameter *skipGlobal=true* dan

skipDepartemen=true sehingga hanya *layer* jabatan yang diproses. Jika *isMitra = false*, *Context* melanjutkan pemeriksaan kedua yaitu mengecek keberadaan *assignment* cakupan jabatan di *database*, di mana jika ditemukan akan memilih *JobSpecificStrategy* yang memanggil *resolveVariabelDecorator* dengan ketiga *layer* aktif, dan jika tidak ditemukan akan memilih *DefaultStrategy* yang memanggil *resolveVariabelDecorator* hanya dengan *layer* global dan departemen.

Sebagai contoh, karyawan dengan jabatan *Driver* memiliki *flag isMitra = true* sehingga sistem memilih *MitraStrategy* dan hanya memproses variabel eksklusif jabatan *Driver* seperti *On-Time Delivery*, Total Ritase, Efisiensi BBM, *POD Return Speed*, Kepuasan Pelanggan, *Safety Score*, dan Perawatan Truk dengan total bobot 100%. Sebaliknya, karyawan dengan jabatan *Staff Accounting* yang memiliki *assignment* cakupan jabatan akan menggunakan *JobSpecificStrategy* dengan tiga *layer* aktif, sedangkan jabatan yang hanya memiliki variabel global dan departemen akan menggunakan *DefaultStrategy*.

4.2.3 Perancangan *Decorator Pattern*

Decorator Pattern diterapkan dalam sistem ini untuk menyusun variabel KPI yang berlaku bagi setiap karyawan secara dinamis dan bertingkat. Pendekatan ini memungkinkan sistem menggabungkan variabel dari beberapa *layer assignment* secara komposabel, di mana setiap *layer* dapat menambahkan atau menggantikan variabel dari *layer* sebelumnya tanpa mengubah struktur kode program. Implementasi *Decorator Pattern* dalam sistem ini diwujudkan melalui fungsi *resolveVariabelDecorator* yang dipanggil oleh setiap *strategy* dengan parameter yang berbeda sesuai dengan jenis *strategy*-nya.

Sistem ini mengorganisasikan variabel KPI ke dalam tiga cakupan *assignment* yang membentuk hierarki *layer* sebagai berikut:

Layer 1 Global berlaku untuk seluruh karyawan tetap tanpa terkecuali. Variabel pada *layer* ini merupakan indikator dasar yang relevan bagi semua jabatan. Variabel KPI pada *layer global* disajikan pada Tabel 9 berikut.

Tabel 8. Variabel KPI *Layer Global*

Variabel	Tipe	Arah	Bobot
Presensi	%	↑	10%

***Layer 2* Departemen** berlaku untuk seluruh karyawan dalam satu departemen tertentu dan dapat menggantikan variabel *global* jika terdapat variabel yang sama. Variabel KPI pada *layer* departemen disajikan pada Tabel 10 berikut.

Tabel 9. Variabel KPI *Layer* Departemen

Departemen	Variabel	Tipe	Arah	Bobot
Operasional	*Incident Response*	Jam	↓	10%
Marketing	*Complaint Rate*	Jumlah	↓	15%
Marketing	*SLA Compliance*	%	↑	15%
Finance	-	-	-	-

***Layer 3* Jabatan** berlaku eksklusif untuk satu jabatan tertentu dan memiliki prioritas tertinggi dapat menggantikan variabel dari *layer* departemen maupun *global* jika terdapat variabel yang sama. Variabel KPI pada *layer* jabatan disajikan pada Tabel 11 berikut.

Tabel 10. Variabel KPI *Layer* Jabatan

Jabatan	Variabel	Tipe	Arah	Bobot
Manager/Supervisor Operasional	*Fleet Utility*	%	↑	25%
	Order Fulfillment	%	↑	25%
	SLA Compliance	%	↑	15%
	Complaint Rate	Jumlah	↓	10%
	Budget Absorption	%	↑	5%
Manager/Supervisor Marketing & CS	*Repeat Order Rate*	%	↑	20%
	Order Fulfillment	%	↑	15%
	Response Time	Jam	↓	10%
	Budget Absorption	%	↑	10%
	New Volume Order	Jumlah	↑	5%
Staff Accounting	*Data Accuracy*	%	↑	25%
	AR Collection	Hari	↓	20%
	Invoice Processed	Dokumen	↑	20%
	Tax Compliance	%	↑	15%
	Days to Close	Hari	↓	5%
	Payroll Accuracy	%	↑	5%
Staff Operasional	*Order Fulfillment*	%	↑	25%
	Fleet Utility	%	↑	25%
	Document Accuracy	%	↑	15%
	Complaint Rate	Jumlah	↓	10%
	On-Time Dispatch	%	↑	5%

Staff Garasi	*Fleet Downtime Rate*	%	↓	20%
	Schedule Compliance	%	↑	20%
	Servis Turnaround Time	Hari	↓	15%
	Inventory Accuracy	%	↑	15%
	Vendor Management	Skala 1-5	↑	5%
	Budget Absorption	%	↑	5%
Customer Service	*Response Time*	Jam	↓	20%
	Order Fulfillment	%	↑	15%
	Repeat Order Rate	%	↑	10%
	Issue Resolution Rate	%	↑	10%
	First Contact Resolution	%	↑	5%
Staff Marketing	*Campaign Execution Rate*	%	↑	20%
	Lead Generation	Jumlah	↑	15%
	Content Output	Jumlah	↑	15%
	Market Research Report	Jumlah	↑	10%
Driver	*On-Time Delivery*	%	↑	25%
	Total Ritase	Trip	↑	20%
	Efisiensi BBM	Km/Liter	↑	15%
	POD Return Speed	Hari	↓	10%
	Kepuasan Pelanggan	Skala 1-5	↑	20%
	Safety Score	Poin 0-100	↑	5%
	Perawatan Truk	Skala 1-5	↑	5%

Mekanisme penggabungan ketiga *layer* dilakukan melalui struktur *variabelMap* yang menggunakan *idVariabel* sebagai kunci. *Layer* global dimasukkan terlebih dahulu sebagai prioritas terendah, kemudian *layer* departemen menggantikan variabel global jika *idVariabel* sama, dan terakhir *layer* jabatan menggantikan variabel departemen maupun global jika *idVariabel* sama. Hasil akhirnya adalah daftar variabel unik tanpa duplikasi yang mencerminkan konfigurasi penilaian yang tepat untuk setiap karyawan.

4.2.4 Perancangan Algoritma FCM

Algoritma *Fuzzy C-Means* (FCM) diterapkan dalam sistem ini untuk melakukan klasterisasi kinerja karyawan berdasarkan skor akhir yang dihasilkan dari proses penilaian KPI. FCM dipilih karena kemampuannya dalam melakukan *soft clustering*, di mana setiap karyawan memiliki derajat keanggotaan terhadap setiap *cluster* antara 0 hingga 1, sehingga mampu merepresentasikan kinerja karyawan yang seringkali berada pada area samar dan tidak dapat dikategorikan secara mutlak. Berbeda dengan *K-Means* yang bersifat *hard clustering*, FCM menghasilkan informasi yang lebih kaya karena karyawan dapat memiliki keanggotaan parsial di lebih dari satu *cluster* sekaligus.

Perancangan algoritma FCM dalam sistem ini mencakup dua aspek utama yaitu alur integrasi FCM ke dalam sistem dan alur algoritma FCM itu sendiri. Kedua aspek ini dirancang sebelum tahap implementasi untuk memastikan algoritma dapat berjalan dengan benar dan terintegrasi dengan baik ke dalam arsitektur *Modular Monolith* yang telah dirancang.

1. Parameter FCM

Sistem ini menggunakan parameter FCM yang dapat dikonfigurasi oleh HR melalui modul kpi-periode sebelum periode dibuka. Parameter yang digunakan disajikan pada Tabel 12 berikut.

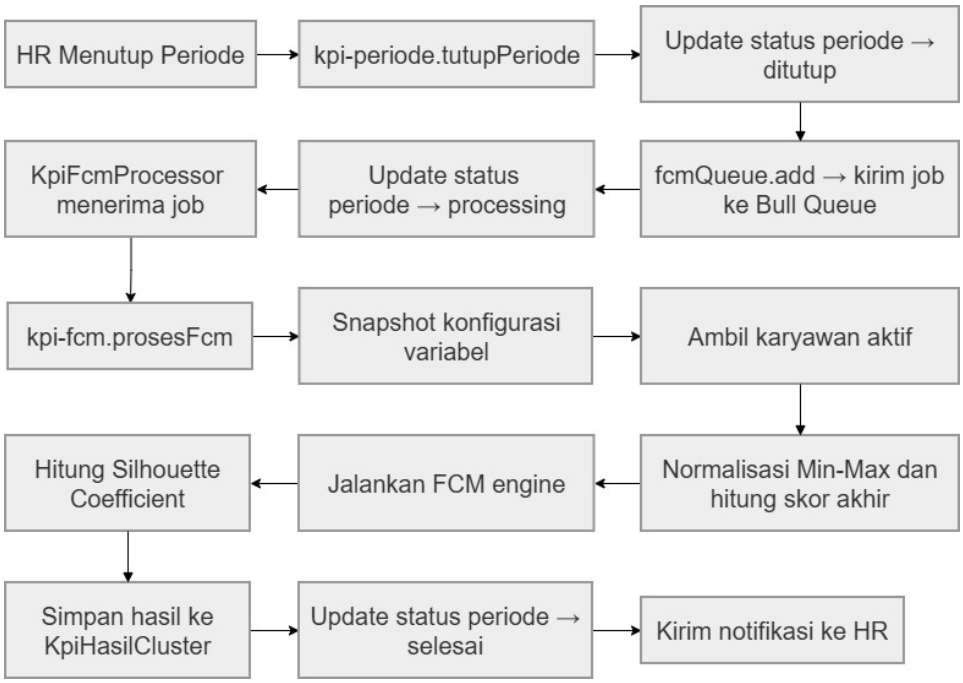
Tabel 11. Parameter FCM

Parameter	Nilai Default	Keterangan
Jumlah *cluster* (c)	3	*Fixed: High Performer, Standard, Low Performer*
Pangkat pembobot (m)	2.0	Eksponen fuzzy yang mengatur tingkat kekaburan
Maksimum iterasi	100	Batas maksimum iterasi jika belum konvergen
Error minimum (ε)	0.000001	Batas toleransi konvergensi
Seed	idPeriode	Seed deterministik via Linear Congruential Generator

Penggunaan *seed* deterministik berbasis idPeriode memastikan hasil klasterisasi bersifat *reproducible* untuk data dan periode yang sama, sistem selalu menghasilkan output yang identik meskipun dijalankan berulang kali.

2. Alur Integrasi FCM dalam Sistem

Proses FCM dijalankan secara otomatis setelah HR menutup periode penilaian. Alur integrasi FCM ke dalam sistem digambarkan pada Gambar 7 berikut.



Gambar 7. Diagram Alur Sistem FCM

Mengacu pada Gambar 7, proses dimulai ketika HR menutup periode melalui modul *kpi-periode*. Sistem kemudian mengirimkan *job* ke *Bull Queue* secara *asynchronous* sehingga proses klasterisasi berjalan di *background* tanpa mengganggu operasional sistem. *KpiFcmProcessor* menerima *job* dan mengorkestrasi enam langkah proses FCM. Langkah pertama adalah pembuatan *snapshot* seluruh konfigurasi *assignment* variabel yang aktif saat periode ditutup disimpan ke tabel *KpiVariabelSnapshot* untuk memastikan konsistensi hasil meskipun konfigurasi *assignment* diubah di kemudian hari. Langkah kedua adalah pengambilan seluruh karyawan aktif yang tidak masuk daftar *skip*. Langkah ketiga adalah normalisasi nilai dan perhitungan skor akhir per karyawan. Langkah keempat adalah menjalankan FCM *engine*. Langkah kelima adalah menghitung *Silhouette Coefficient* untuk mengukur kualitas hasil klasterisasi. Langkah keenam adalah menyimpan hasil ke tabel

KpiHasilCluster, memperbarui status periode menjadi selesai, dan mengirimkan notifikasi ke seluruh HR.

3. Normalisasi Min-Max

Sebelum data diproses oleh FCM *engine*, nilai KPI setiap karyawan dinormalisasi menggunakan metode *Min-Max* agar seluruh variabel berada dalam rentang 0 hingga 1. Normalisasi dilakukan dengan mempertimbangkan arah nilai variabel sebagai berikut:

a. Untuk variabel dengan arah lebih tinggi lebih baik:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Keterangan :

- x_{norm} = nilai variabel setelah dinormalisasi
- x = nilai aktual variabel karyawan
- x_{min} = nilai minimum variabel yang dikonfigurasi
- x_{max} = nilai maksimum variabel yang dikonfigurasi

b. Untuk variabel dengan arah lebih rendah lebih baik:

$$x_{norm} = \frac{x_{max} - x}{x_{max} - x_{min}}$$

Keterangan :

- x_{norm} = nilai variabel setelah dinormalisasi
- x = nilai aktual variabel karyawan
- x_{min} = nilai minimum variabel yang dikonfigurasi
- x_{max} = nilai maksimum variabel yang dikonfigurasi

Hasil normalisasi kemudian dikalikan dengan bobot masing-masing variabel dan diakumulasikan menjadi satu skor akhir per karyawan dalam rentang 0 hingga 1 dengan rumus:

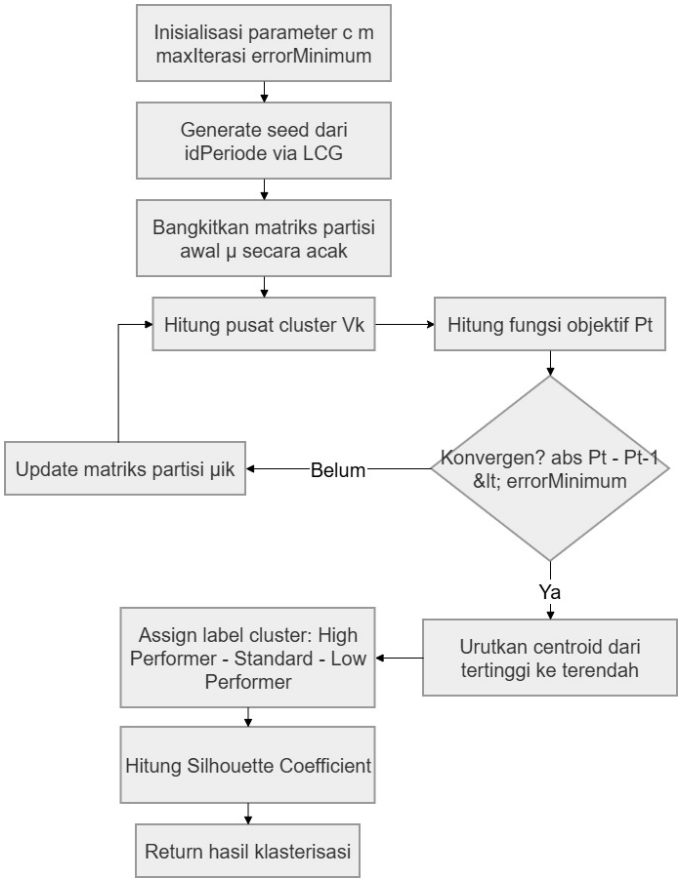
$$skor_{akhir} = \sum_{i=1}^{n} \frac{x_{norm_i} \times bobot_i}{100}$$

Keterangan :

- $skor_{akhir}$ = skor akhir karyawan dalam rentang 0 hingga 1
- x_{norm_i} = nilai normalisasi variabel ke-i
- $bobot_i$ = bobot variabel ke-i dalam persentase
- n = jumlah variabel KPI yang berlaku untuk karyawan

4. Alur Algoritma FCM

Alur algoritma FCM yang dijalankan oleh *fcm.engine.ts* digambarkan pada Gambar 8 berikut.



Gambar 8. Diagram Alur Algoritma FCM

Mengacu pada Gambar 8, algoritma FCM dimulai dengan inisialisasi parameter dan pembangkitan *seed* deterministik dari *idPeriode* menggunakan *Linear Congruential Generator* (LCG). Matriks partisi awal μ dibangkitkan secara acak menggunakan *seed* tersebut, kemudian dinormalisasi per karyawan sehingga jumlah derajat keanggotaan setiap karyawan terhadap seluruh *cluster* sama dengan 1. Selanjutnya algoritma

masuk ke tahap iterasi yang mengacu pada tahapan FCM sebagaimana telah dijelaskan pada 2.3.2, meliputi perhitungan pusat *cluster*, perhitungan fungsi objektif, pembaruan matriks partisi, dan pengecekan konvergensi. Iterasi berhenti ketika selisih fungsi objektif antar iterasi lebih kecil dari nilai *error* minimum atau maksimum iterasi telah tercapai.

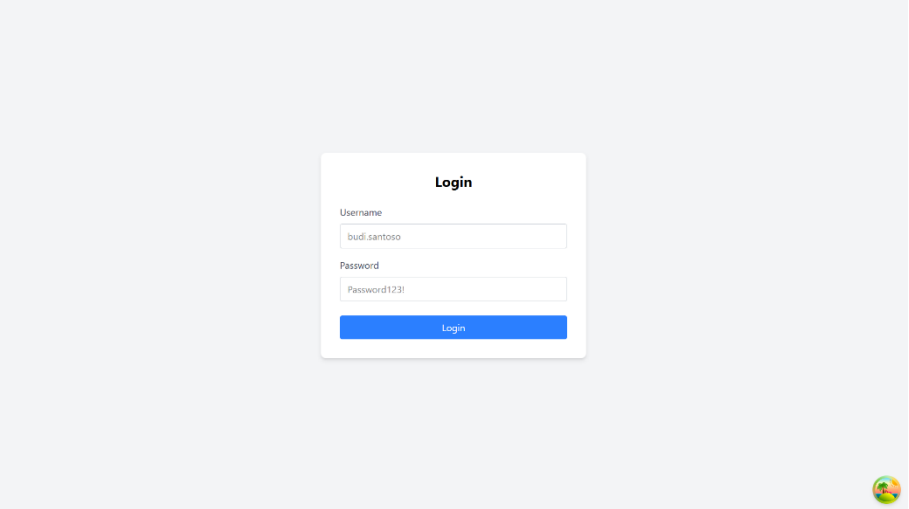
Setelah konvergen, sistem menentukan label *cluster* secara otomatis berdasarkan nilai *centroid cluster* dengan *centroid* tertinggi mendapat label *High Performer*, *cluster* tengah mendapat label *Standard*, dan *cluster* dengan *centroid* terendah mendapat label *Low Performer*. Kualitas hasil klasterisasi kemudian diukur menggunakan *Silhouette Coefficient* yang mengacu pada rumus sebagaimana telah dijelaskan pada 2.3.3.

4.2.5 Perancangan Antarmuka Pengguna (*Mockup*)

Perancangan antarmuka pengguna dilakukan untuk menggambarkan tampilan visual sistem yang akan dibangun sebelum tahap implementasi. Antarmuka dirancang berdasarkan kebutuhan fungsional yang telah diidentifikasi pada sub-bab 4.1.1, di mana *system* memiliki tiga peran pengguna yaitu HR, Atasan, dan Karyawan dengan hak akses yang berbeda. Setiap pengguna hanya dapat mengakses halaman yang sesuai dengan *permission* jabatannya, sebagaimana telah dirancang dalam modul *auth* pada sub-bab 4.2.1.

Perancangan antarmuka dikelompokkan berdasarkan peran pengguna untuk memudahkan pemahaman alur navigasi dan fungsionalitas yang tersedia bagi masing-masing pengguna. Bagian pertama menampilkan halaman *login* yang dapat diakses oleh seluruh peran pengguna, dilanjutkan dengan halaman-halaman yang dapat diakses oleh HR, Atasan, dan Karyawan secara berurutan.

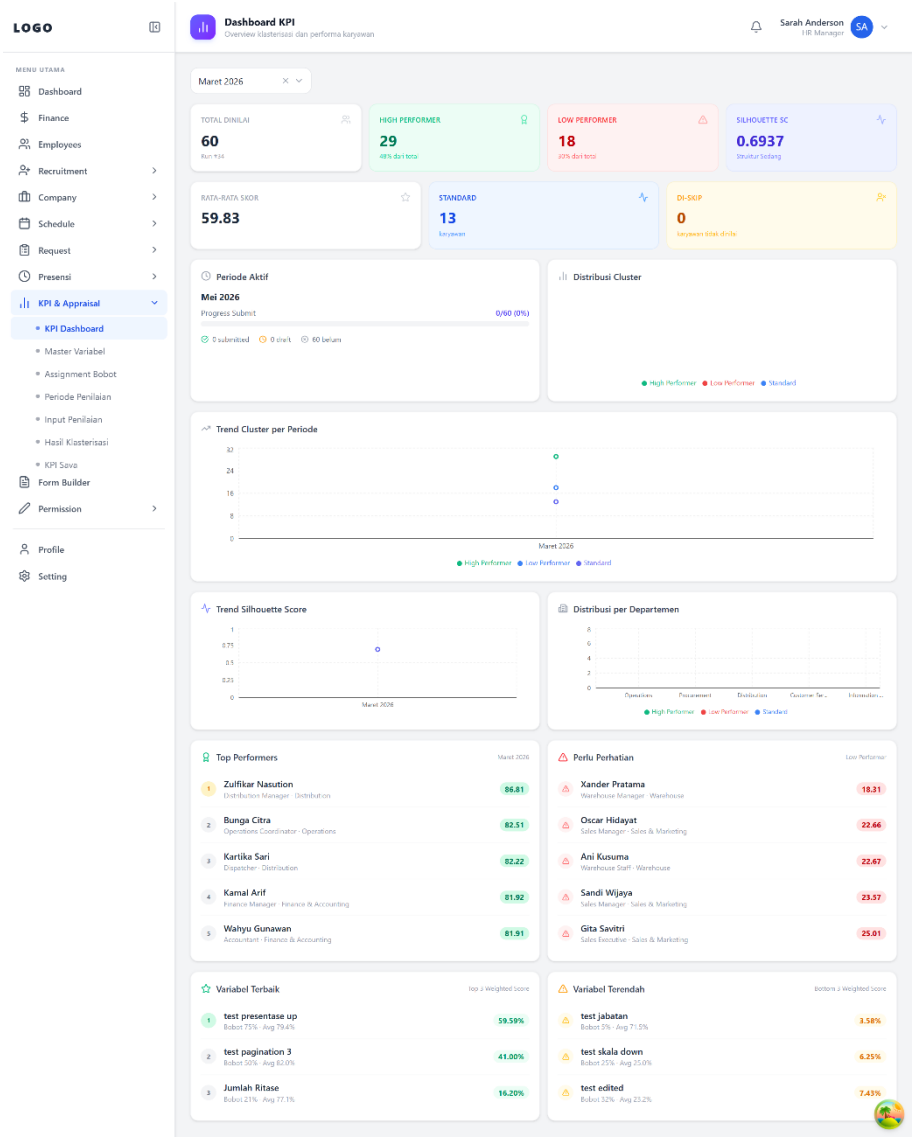
1. Halaman *Login* (URS-HR-01, URS-A-01, URS-K-01)



Gambar 9. Halaman *Login*

Halaman *login* merupakan halaman pertama yang ditampilkan kepada seluruh pengguna sistem sebelum dapat mengakses fitur-fitur yang tersedia. Halaman ini menampilkan *form login* yang terdiri dari dua *field input* yaitu *field Username* untuk memasukkan nama pengguna dan *field Password* untuk memasukkan kata sandi, serta tombol *Login* berwarna biru untuk memproses autentikasi. Tampilan halaman dirancang minimalis dengan *card form* yang terpusat di tengah layar pada latar belakang abu-abu terang, sehingga pengguna dapat langsung fokus pada proses *login* tanpa distraksi visual. Setelah pengguna berhasil melakukan autentikasi, sistem akan mengarahkan pengguna ke halaman *dashboard* sesuai dengan *permission* jabatan yang dimiliki.

2. Halaman KPI *Dashboard* (URS-HR-14)



Gambar 10. Halaman KPI *Dashboard*

Halaman *KPI Dashboard* merupakan halaman utama modul *KPI & Appraisal* yang dapat diakses oleh HR, di mana halaman ini menyajikan ringkasan komprehensif hasil klasterisasi kinerja karyawan secara visual. Pada bagian atas halaman terdapat *selector* periode yang memungkinkan HR memilih periode selesai yang ingin ditampilkan datanya.

Bagian atas konten menampilkan dua baris *summary card* yang merangkum indikator kunci hasil klasterisasi. Baris pertama menampilkan Total Dinilai, *High Performer*, *Low Performer*, dan

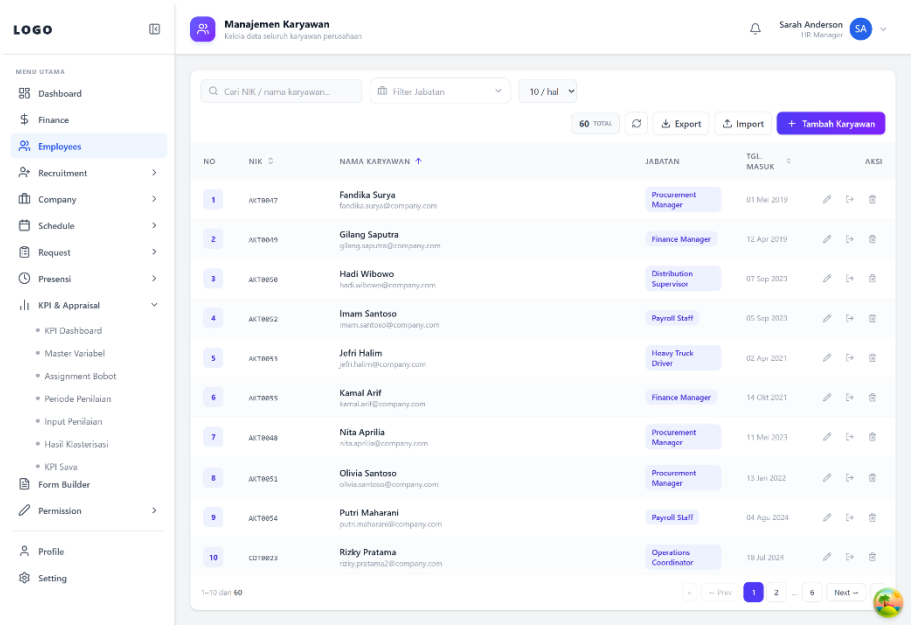
Silhouette SC beserta keterangan kategori kualitas klasterisasi. Baris kedua menampilkan Rata-rata Skor beserta tren perbandingan periode sebelumnya, jumlah karyawan *Standard*, dan jumlah karyawan yang Di-*Skip*.

Di bawah *summary card* terdapat dua panel berdampingan yaitu Periode Aktif dan Distribusi *Cluster*. Panel Periode Aktif menampilkan informasi periode penilaian yang sedang berjalan beserta *progress submit* karyawan dalam bentuk *progress bar* yang menunjukkan jumlah *submitted*, *draft*, dan belum diisi. Panel Distribusi *Cluster* menampilkan proporsi setiap *cluster* dalam bentuk *donut chart* dengan keterangan warna per label *cluster*.

Bagian tengah halaman menampilkan visualisasi tren dalam dua panel. Panel *Trend Cluster* per Periode menampilkan tren jumlah karyawan per *cluster* antar periode dalam bentuk *line chart*. Panel *Trend Silhouette Score* menampilkan tren kualitas klasterisasi antar periode, sedangkan panel Distribusi per Departemen menampilkan sebaran *cluster* per departemen dalam bentuk *stacked bar chart*.

Bagian bawah halaman menampilkan empat panel analisis. Panel *Top Performers* menampilkan lima karyawan dengan skor tertinggi beserta jabatan, departemen, dan skornya. Panel Perlu Perhatian menampilkan karyawan dengan label *Low Performer* beserta skornya sebagai bahan evaluasi HR. Panel Variabel Terbaik dan Variabel Terendah menampilkan analisis *weighted score* per variabel KPI untuk mengidentifikasi variabel dengan kontribusi tertinggi dan terendah terhadap skor akhir karyawan.

3. Halaman Manajemen Karyawan (URS-HR-03)

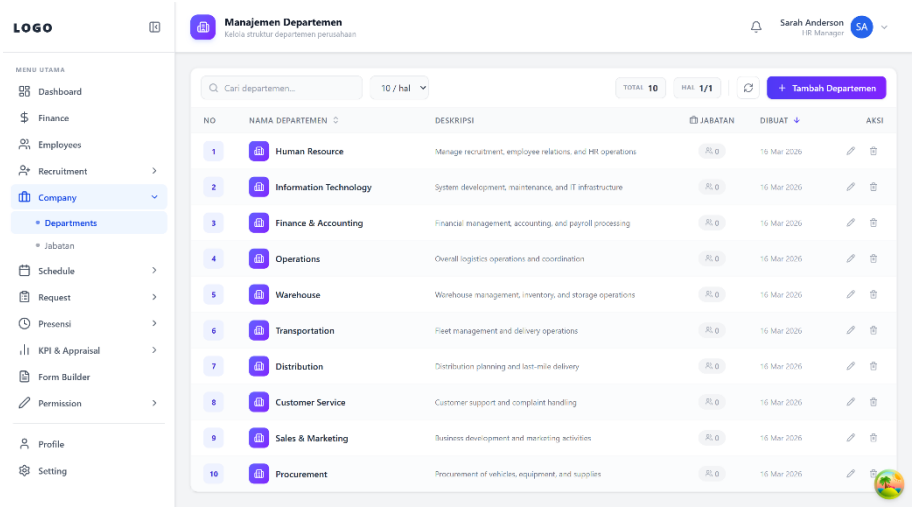


Gambar 11. Halaman Manajemen Karyawan

Halaman Manajemen Karyawan dapat diakses oleh HR untuk mengelola data seluruh karyawan perusahaan. Halaman ini menampilkan daftar karyawan dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan NIK atau nama karyawan, *filter* berdasarkan jabatan, dan pengaturan jumlah data per halaman.

Tabel karyawan menampilkan kolom NIK, nama karyawan beserta *email*, jabatan yang ditampilkan dalam bentuk *badge* berwarna, tanggal masuk, dan kolom aksi yang berisi tombol *edit*, *reset password*, dan hapus. Bagian atas tabel menampilkan total keseluruhan karyawan beserta tombol *Export* untuk mengunduh data dan tombol *Import* untuk mengunggah data karyawan secara massal, serta tombol Tambah Karyawan untuk menambahkan karyawan baru. Navigasi halaman tersedia di bagian bawah tabel dalam bentuk *pagination*.

4. Halaman Manajemen Departemen (URS-HR-04)

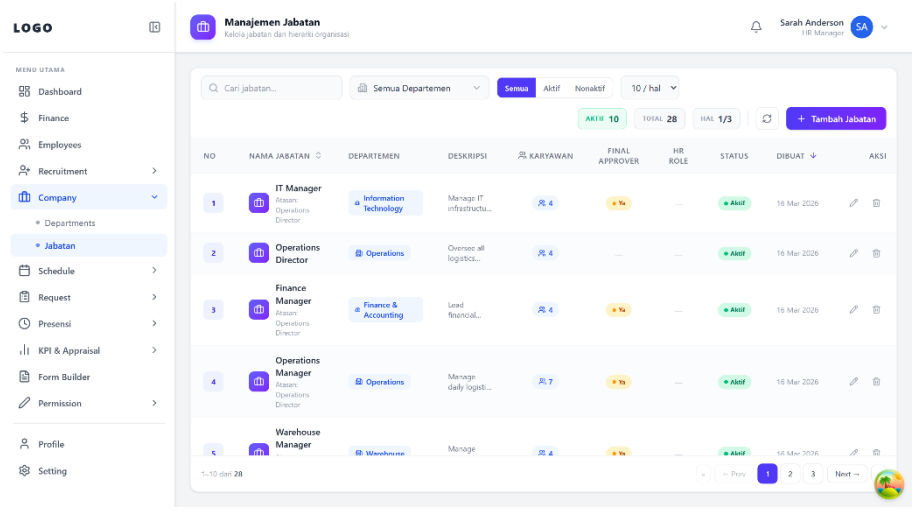


Gambar 12. Halaman Manajemen Departemen

Halaman Manajemen Departemen dapat diakses oleh HR untuk mengelola struktur departemen perusahaan. Halaman ini menampilkan daftar departemen dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama departemen dan pengaturan jumlah data per halaman.

Tabel departemen menampilkan kolom nama departemen beserta ikon identitas, deskripsi singkat fungsi departemen, jumlah jabatan yang terdaftar dalam departemen tersebut, tanggal dibuat, dan kolom aksi yang berisi tombol *edit* dan hapus. Bagian atas tabel menampilkan total keseluruhan departemen beserta informasi halaman saat ini dan tombol Tambah Departemen untuk menambahkan departemen baru.

5. Halaman Manajemen Jabatan (URS-HR-05)

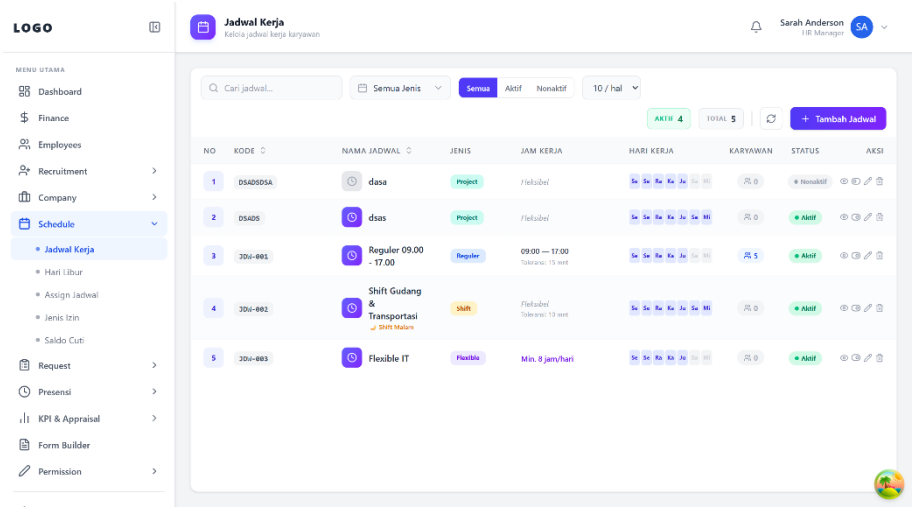


Gambar 13. Halaman Manajemen Jabatan

Halaman Manajemen Jabatan dapat diakses oleh HR untuk mengelola jabatan dan hierarki organisasi perusahaan. Halaman ini menampilkan daftar jabatan dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama jabatan, *filter* berdasarkan departemen, *filter* status aktif atau nonaktif, dan pengaturan jumlah data per halaman.

Tabel jabatan menampilkan kolom nama jabatan beserta informasi atasan langsung, departemen yang ditampilkan dalam bentuk *badge* berwarna, deskripsi singkat, jumlah karyawan yang menduduki jabatan tersebut, status *Final Approver*, status *HR Role*, status aktif atau nonaktif, tanggal dibuat, dan kolom aksi yang berisi tombol *edit* dan hapus. Bagian atas tabel menampilkan jumlah jabatan aktif, total keseluruhan jabatan, informasi halaman saat ini, dan tombol Tambah Jabatan untuk menambahkan jabatan baru.

6. Halaman Jadwal Kerja (URS-HR-06)

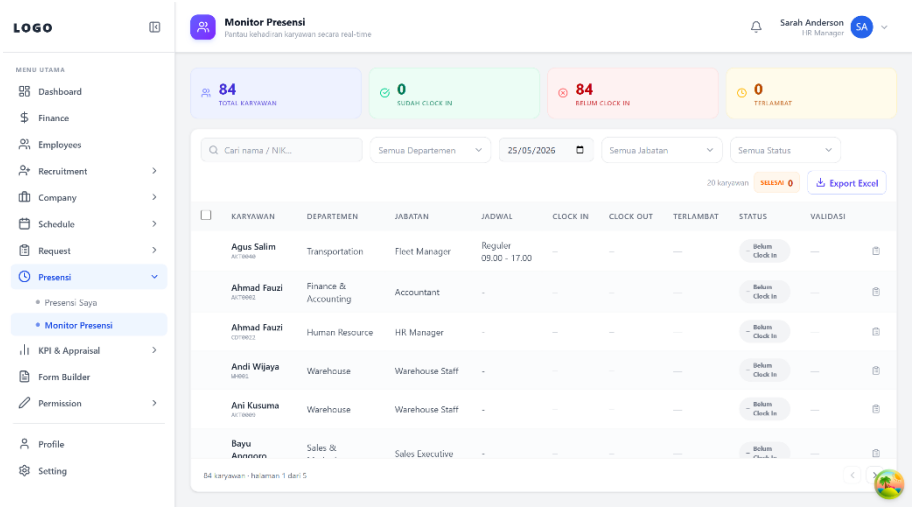


Gambar 14. Halaman Jadwal Kerja

Halaman Jadwal Kerja dapat diakses oleh HR untuk mengelola jadwal kerja karyawan yang menjadi dasar perhitungan presensi. Halaman ini menampilkan daftar jadwal kerja dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama jadwal, *filter* berdasarkan jenis jadwal, *filter* status aktif atau nonaktif, dan pengaturan jumlah data per halaman.

Tabel jadwal kerja menampilkan kolom kode jadwal, nama jadwal beserta ikon jenis, jenis jadwal yang ditampilkan dalam bentuk *badge* berwarna seperti *Reguler*, *Shift*, *Project*, dan *Flexible*, jam kerja beserta toleransi keterlambatan, hari kerja yang ditampilkan dalam bentuk indikator hari aktif dan nonaktif, jumlah karyawan yang menggunakan jadwal tersebut, status aktif atau nonaktif, dan kolom aksi yang berisi tombol lihat detail, *edit*, dan hapus. Bagian atas tabel menampilkan jumlah jadwal aktif, total keseluruhan jadwal, dan tombol Tambah Jadwal untuk menambahkan jadwal kerja baru.

7. Halaman *Monitor* Presensi (URS-HR-07)

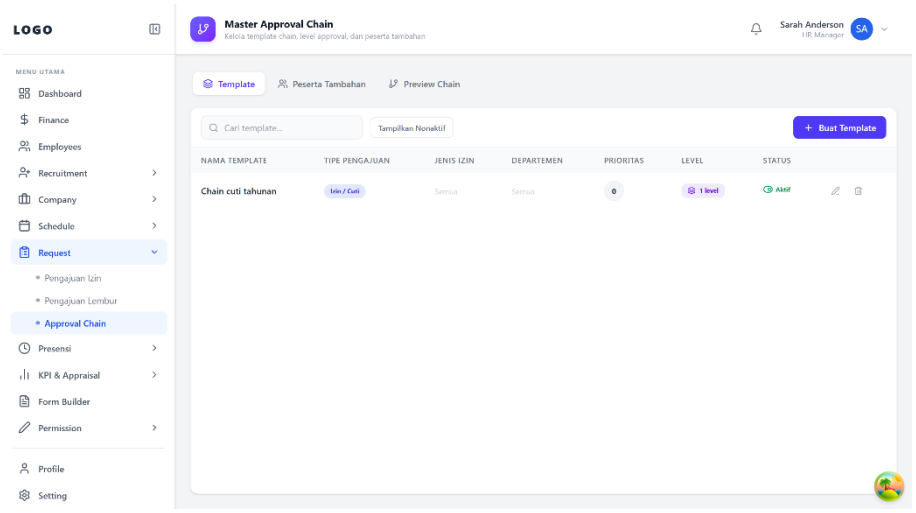


Gambar 15. Halaman *Monitor* Presensi

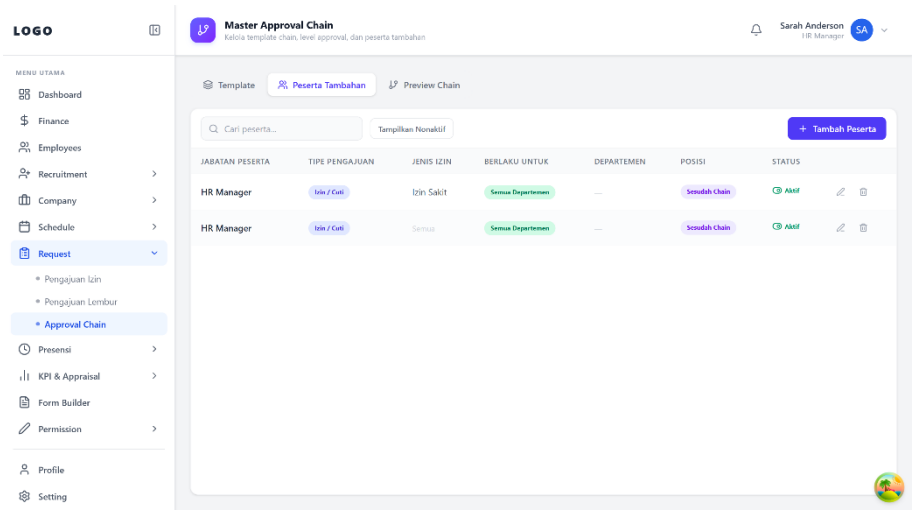
Halaman *Monitor* Presensi dapat diakses oleh HR untuk memantau kehadiran seluruh karyawan secara *real-time*. Bagian atas halaman menampilkan empat *summary card* yang merangkum kondisi kehadiran pada hari yang dipilih yaitu Total Karyawan, Sudah *Clock In*, Belum *Clock In*, dan Terlambat.

Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama atau NIK karyawan, *filter* berdasarkan departemen, tanggal, jabatan, dan status kehadiran. Tabel presensi menampilkan kolom karyawan beserta NIK, departemen, jabatan, jadwal kerja, waktu *clock in*, waktu *clock out*, durasi keterlambatan, status kehadiran, dan kolom validasi. Bagian atas tabel menampilkan jumlah karyawan yang ditampilkan, indikator jumlah data yang selesai divalidasi, dan tombol *Export Excel* untuk mengunduh data presensi.

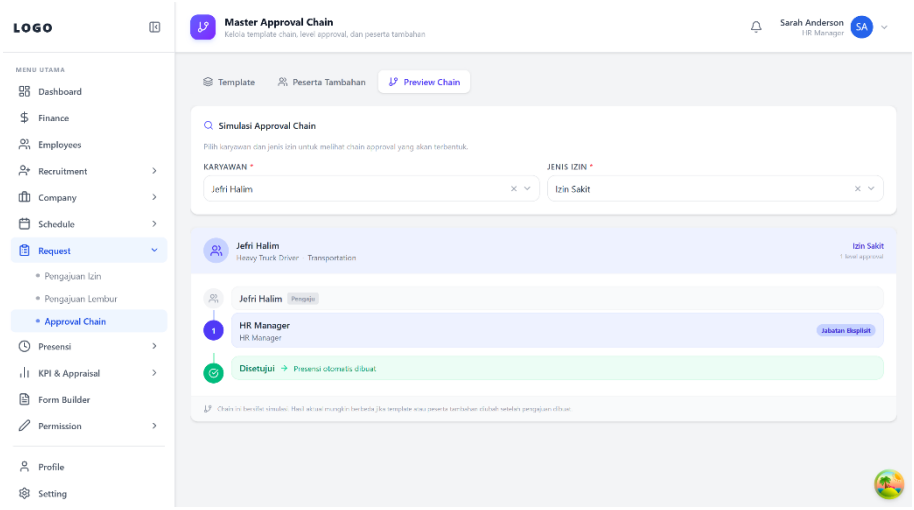
8. Halaman *Master Approval Chain* (URS-HR-08)



Gambar 16. Halaman *Master Approval Chain — Tab Template*



Gambar 17. Halaman *Master Approval Chain — Tab* Peserta Tambahan



Gambar 18. Halaman *Master Approval Chain — Tab Preview Chain*

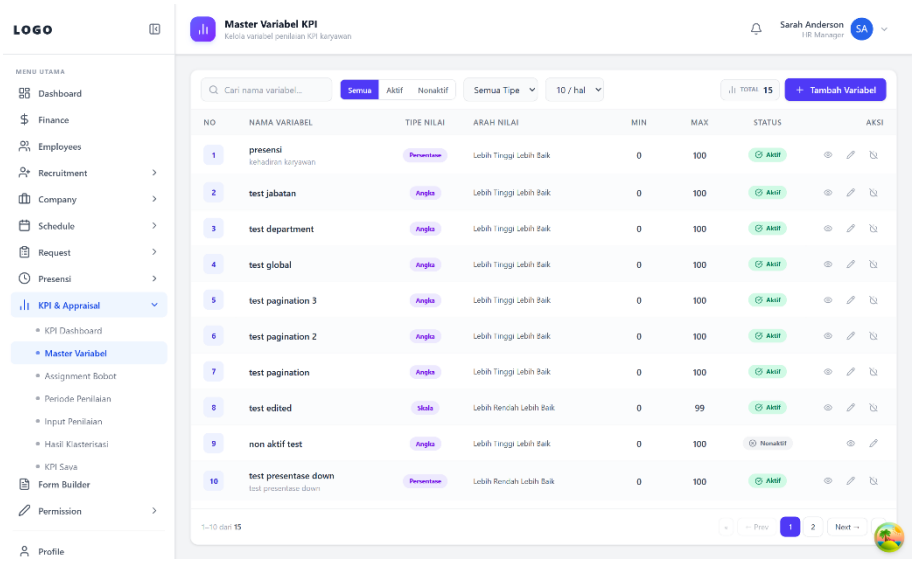
Halaman *Master Approval Chain* dapat diakses oleh HR untuk mengkonfigurasi alur persetujuan pengajuan izin karyawan. Halaman ini terdiri dari tiga *tab* yaitu *Template*, Peserta Tambahan, dan *Preview Chain*.

Tab Template menampilkan daftar *template chain approval* yang telah dikonfigurasi, di mana setiap *template* memiliki informasi nama *template*, tipe pengajuan, jenis izin, departemen yang berlaku, prioritas, jumlah *level approval*, dan status aktif atau nonaktif. HR dapat membuat *template* baru melalui tombol Buat *Template*.

Tab Peserta Tambahan menampilkan daftar jabatan yang ditambahkan sebagai peserta di luar *chain* utama, di mana setiap peserta memiliki informasi jabatan peserta, tipe pengajuan, jenis izin, cakupan departemen yang berlaku, posisi dalam *chain*, dan status aktif atau nonaktif. HR dapat menambahkan peserta baru melalui tombol Tambah Peserta.

Tab Preview Chain menyediakan fitur simulasi *chain approval*, di mana HR dapat memilih karyawan dan jenis izin untuk melihat susunan *chain approval* yang akan terbentuk secara dinamis beserta urutan *level approver* dan hasil akhir setelah disetujui.

9. Halaman Master Variabel KPI (URS-HR-09)

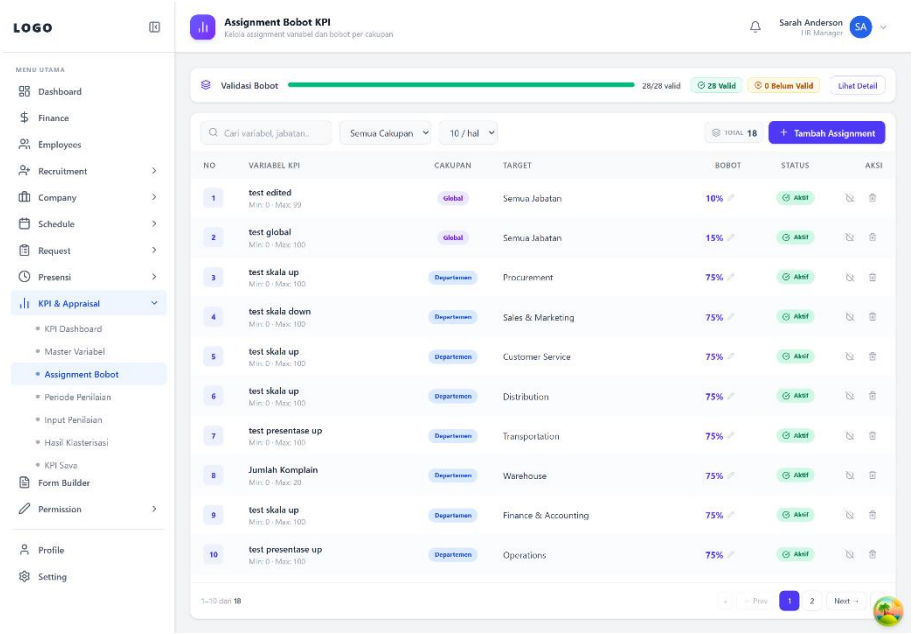


Gambar 19. Halaman Master Variabel KPI

Halaman Master Variabel KPI dapat diakses oleh HR untuk mengelola definisi variabel KPI yang digunakan dalam penilaian kinerja karyawan. Halaman ini menampilkan daftar variabel KPI dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama variabel, *filter* status aktif atau nonaktif, *filter* berdasarkan tipe nilai, dan pengaturan jumlah data per halaman.

Tabel variabel KPI menampilkan kolom nama variabel beserta deskripsi singkat, tipe nilai yang ditampilkan dalam bentuk *badge* berwarna seperti Persentase, Angka, dan Skala, arah nilai yang menunjukkan apakah variabel bersifat lebih tinggi lebih baik atau lebih rendah lebih baik, nilai minimum, nilai maksimum, status aktif atau nonaktif, dan kolom aksi yang berisi tombol lihat detail, *edit*, dan nonaktifkan. Bagian atas tabel menampilkan total keseluruhan variabel dan tombol Tambah Variabel untuk menambahkan variabel KPI baru.

10. Halaman *Assignment* Bobot KPI (URS-HR-10)

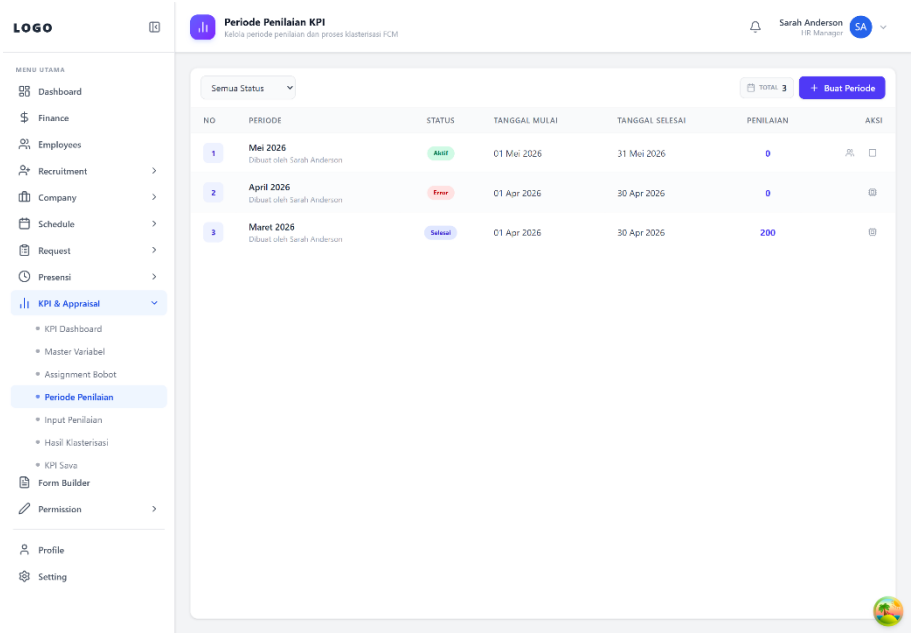


Gambar 20. Halaman *Assignment* Bobot KPI

Halaman *Assignment* Bobot KPI dapat diakses oleh HR untuk mengelola *assignment* variabel KPI ke tiga cakupan yaitu global, departemen, dan jabatan beserta bobotnya masing-masing. Bagian atas halaman menampilkan indikator Validasi Bobot dalam bentuk *progress bar* hijau yang menunjukkan jumlah jabatan yang telah memiliki total bobot valid sebesar 100%, di mana pada contoh ini seluruh 28 jabatan telah valid dan 0 jabatan belum valid. HR dapat melihat detail validasi per jabatan melalui tombol Lihat Detail.

Tabel *assignment* menampilkan daftar *assignment* variabel KPI yang dilengkapi dengan fitur pencarian berdasarkan nama variabel atau jabatan dan *filter* berdasarkan cakupan. Kolom yang ditampilkan meliputi nama variabel KPI beserta rentang nilai minimum dan maksimum, cakupan *assignment* yang ditampilkan dalam bentuk *badge* berwarna seperti *Global* dan Departemen, target cakupan yang menunjukkan jabatan atau departemen yang berlaku, bobot dalam persentase yang dapat di-*edit* langsung, status aktif atau nonaktif, dan kolom aksi yang berisi tombol nonaktifkan dan hapus. Bagian atas tabel menampilkan total keseluruhan *assignment* dan tombol Tambah *Assignment* untuk menambahkan *assignment* baru.

11. Halaman Periode Penilaian KPI (URS-HR-11)

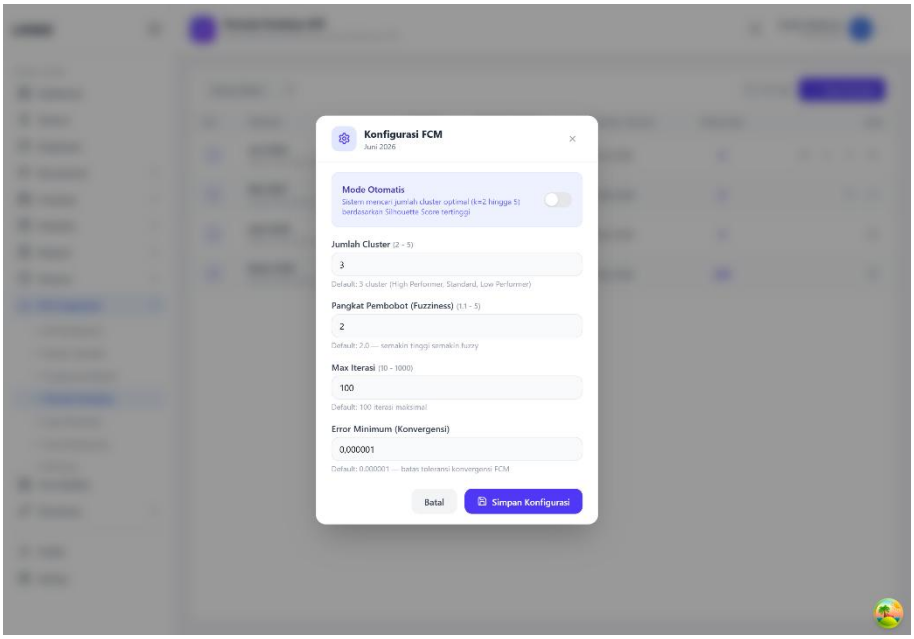


Gambar 21. Halaman Periode Penilaian KPI

Halaman Periode Penilaian KPI dapat diakses oleh HR untuk mengelola periode penilaian dan proses klasterisasi FCM. Halaman ini menampilkan daftar periode penilaian dalam bentuk tabel yang dilengkapi dengan *filter* berdasarkan status periode.

Tabel periode menampilkan kolom nama periode beserta informasi pembuat, status periode yang ditampilkan dalam bentuk *badge* berwarna di mana status Aktif berwarna hijau, *Error* berwarna merah, dan Selesai berwarna ungu, tanggal mulai, tanggal selesai, jumlah penilaian yang telah diinput, dan kolom aksi. Kolom aksi disesuaikan dengan status periode, yaitu periode berstatus Aktif menampilkan tombol *monitor progress input* dan tutup periode, sedangkan periode berstatus Selesai atau *Error* menampilkan tombol konfigurasi FCM untuk menjalankan ulang proses klasterisasi. Bagian atas tabel menampilkan total keseluruhan periode dan tombol Buat Periode untuk membuat periode penilaian baru.

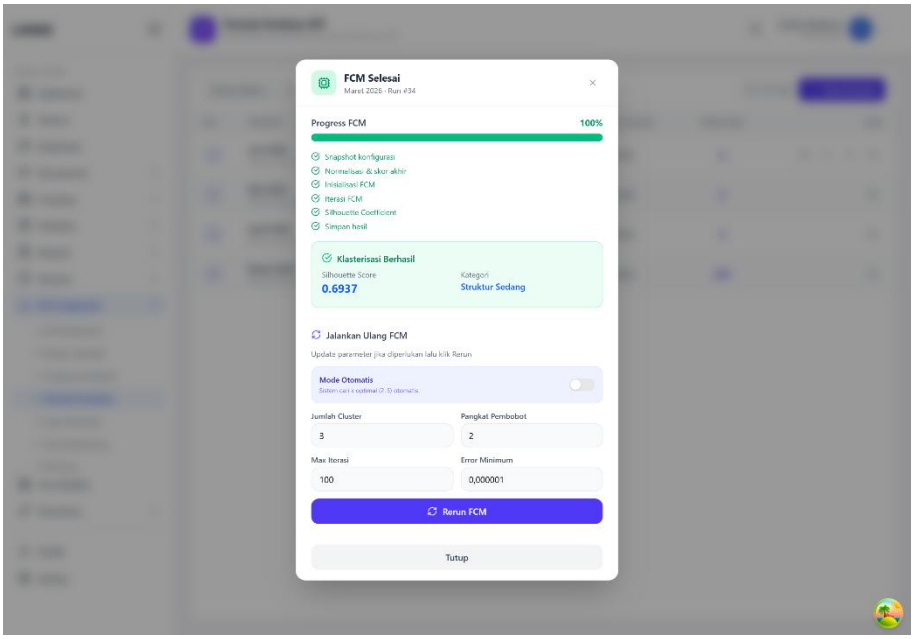
12. Modal Konfigurasi FCM (URS-HR-12)



Gambar 22. *Modal* Konfigurasi FCM

Modal Konfigurasi FCM muncul ketika HR mengklik tombol konfigurasi pada periode yang berstatus *draft*. *Modal* ini menampilkan empat parameter FCM yang dapat dikonfigurasi yaitu Jumlah *Cluster* dengan rentang 2 hingga 5 dan nilai *default* 3 *cluster* yang menghasilkan label *High Performer*, *Standard*, dan *Low Performer*, Pangkat Pembobot (*Fuzziness*) dengan rentang 1.1 hingga 5 dan nilai *default* 2.0, *Max* Iterasi dengan rentang 10 hingga 1000 dan nilai *default* 100 iterasi, serta *Error* Minimum (Konvergensi) dengan nilai *default* 0.000001 sebagai batas toleransi konvergensi FCM. Setiap parameter dilengkapi dengan keterangan nilai *default* untuk memandu HR dalam mengkonfigurasi. HR dapat menyimpan konfigurasi melalui tombol Simpan Konfigurasi atau membatalkan melalui tombol Batal.

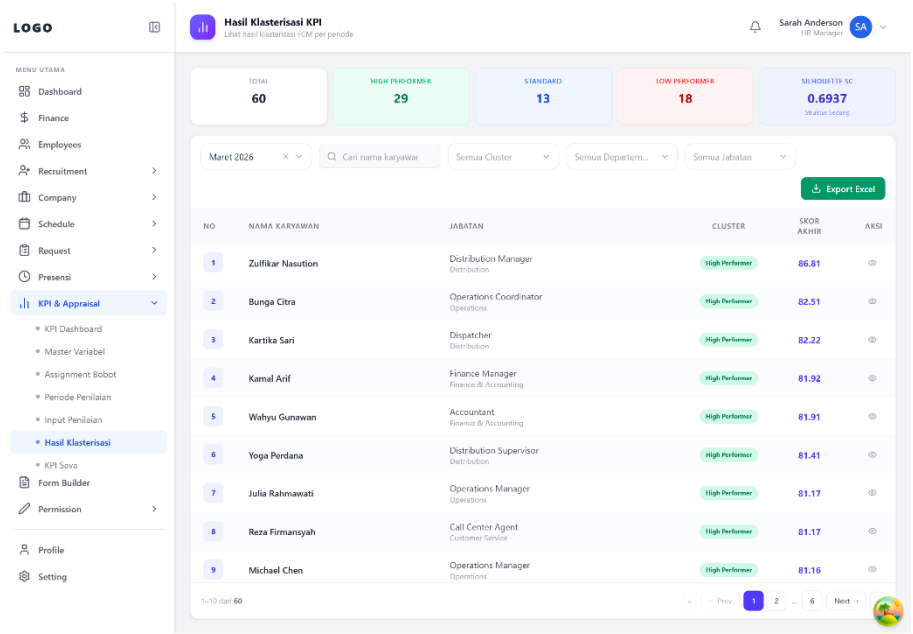
13. Modal FCM Selesai (URS-HR-13)



Gambar 23. *Modal* FCM Selesai

Modal FCM Selesai muncul setelah HR menutup periode dan proses klasterisasi FCM berhasil dijalankan. *Modal* ini menampilkan *progress bar* FCM yang telah mencapai 100% beserta daftar tahapan yang telah selesai dieksekusi yaitu *Snapshot* konfigurasi, Normalisasi dan skor akhir, Inisialisasi FCM, Iterasi FCM, *Silhouette Coefficient*, dan Simpan hasil. Bagian bawah *progress* menampilkan ringkasan hasil klasterisasi berupa nilai *Silhouette Score* dan kategori kualitas klasterisasi. Selain itu *modal* ini juga menyediakan fitur Jalankan Ulang FCM yang memungkinkan HR memperbarui parameter dan menjalankan ulang proses klasterisasi melalui tombol *Rerun* FCM.

14. Halaman Hasil Klasterisasi KPI (URS-HR-14, URS-HR-15)

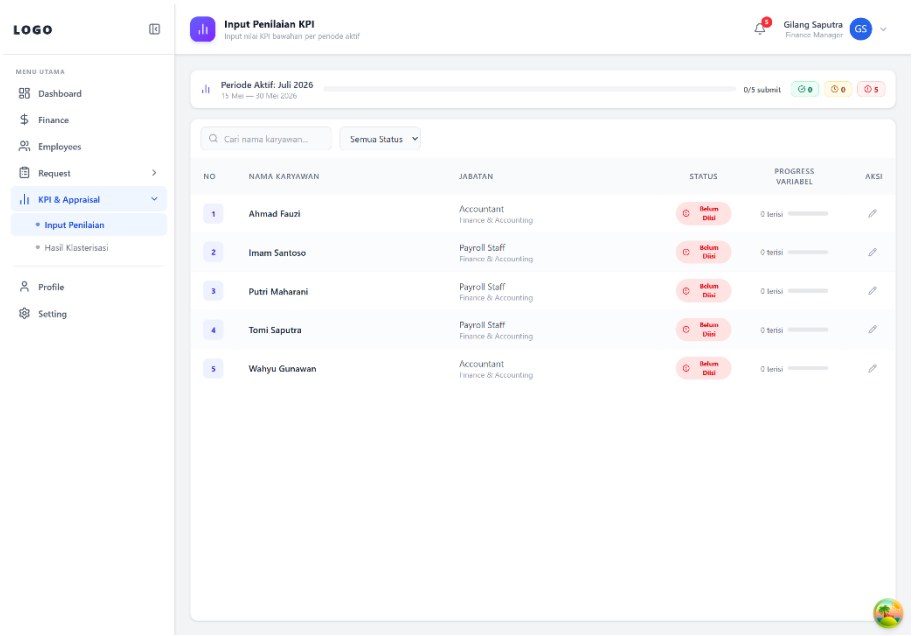


Gambar 24. Halaman Hasil Klasterisasi KPI

Halaman Hasil Klasterisasi KPI dapat diakses oleh HR untuk melihat hasil klasterisasi FCM per periode. Bagian atas halaman menampilkan lima *summary card* yang merangkum hasil klasterisasi yaitu Total karyawan yang dinilai, jumlah *High Performer*, jumlah *Standard*, jumlah *Low Performer*, dan nilai *Silhouette SC* beserta kategori kualitas klasterisasi.

Halaman ini dilengkapi dengan fitur *selector* periode, pencarian berdasarkan nama karyawan, *filter* berdasarkan *cluster*, departemen, dan jabatan, serta tombol *Export Excel* untuk mengunduh data hasil klasterisasi secara keseluruhan. Tabel hasil klasterisasi menampilkan kolom nama karyawan, jabatan beserta departemen, label *cluster* yang ditampilkan dalam bentuk *badge* berwarna, skor akhir dalam skala 0 hingga 100, dan kolom aksi berupa tombol lihat detail untuk melihat rincian nilai per variabel karyawan. Data ditampilkan diurutkan berdasarkan skor akhir tertinggi ke terendah.

15. Halaman *Input* Penilaian KPI (URS-A-04, URS-A-05)

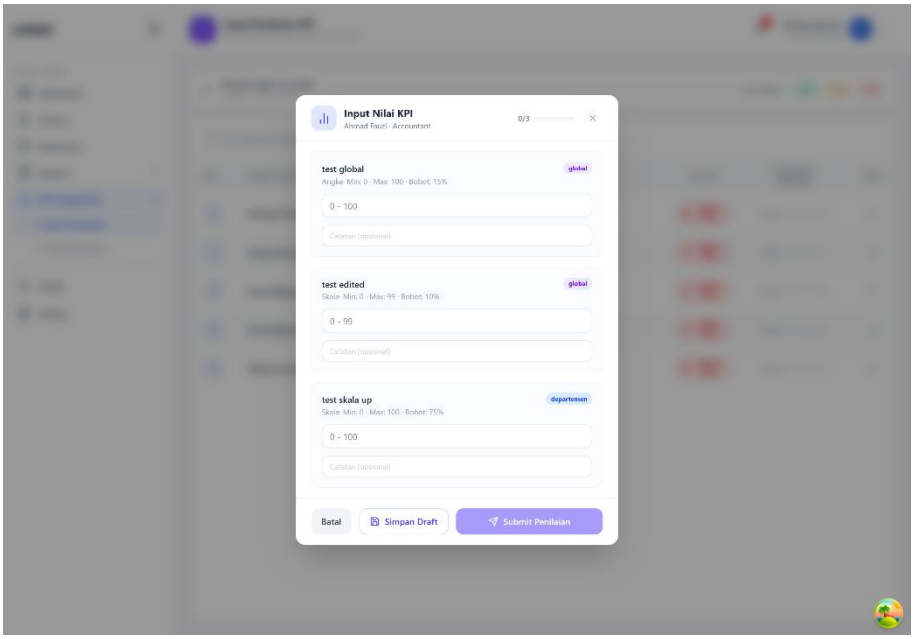


Gambar 25. Halaman *Input* Penilaian KPI

Halaman *Input* Penilaian KPI dapat diakses oleh Atasan untuk menginput nilai KPI bawahan pada periode yang sedang aktif. Bagian atas halaman menampilkan informasi periode aktif beserta rentang tanggal penilaian dan ringkasan *progress submit* yang menunjukkan jumlah bawahan yang sudah *submit*, sedang dalam status *draft*, dan belum diisi.

Tabel bawahan menampilkan daftar karyawan yang menjadi tanggung jawab atasan yang sedang *login*, di mana setiap baris menampilkan nama karyawan, jabatan beserta departemen, status penilaian yang ditampilkan dalam bentuk *badge* berwarna seperti Belum Diisi, *Draft*, dan *Submitted*, *progress* variabel yang menunjukkan jumlah variabel yang telah terisi beserta *progress bar*, dan tombol *edit* untuk membuka *form input* nilai. Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama karyawan dan *filter* berdasarkan status penilaian.

16. *Modal Input* Nilai KPI (URS-A-06, URS-A-07)



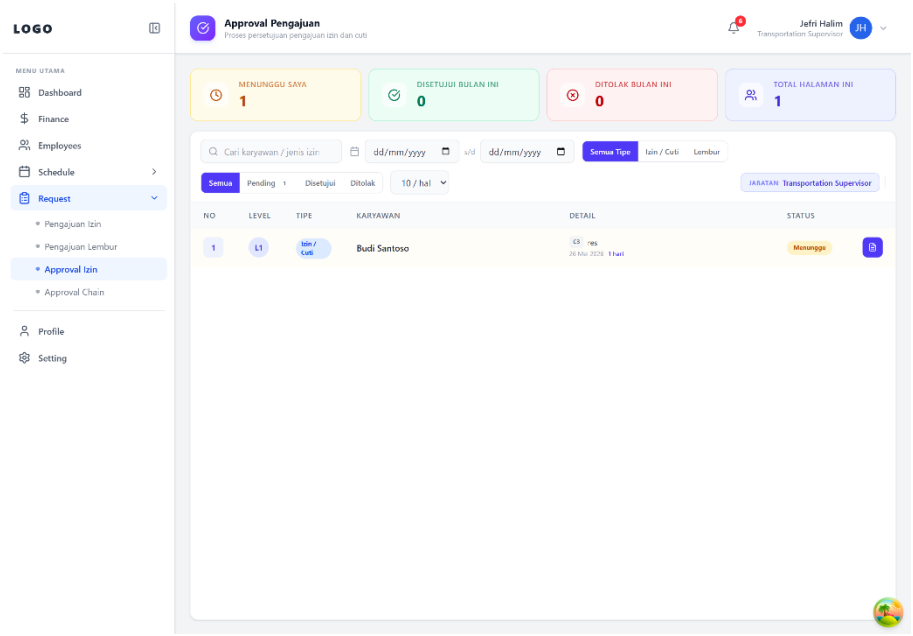
Gambar 26. *Modal Input* Nilai KPI

Modal Input Nilai KPI muncul ketika Atasan mengklik tombol *edit* pada salah satu bawahan di halaman *Input* Penilaian. *Modal* ini menampilkan nama karyawan beserta jabatan yang sedang dinilai dan indikator *progress* pengisian variabel di bagian atas.

Modal ini menampilkan seluruh variabel KPI yang berlaku untuk karyawan tersebut secara dinamis berdasarkan hasil *resolve Strategy Pattern* dan *Decorator Pattern*. Setiap variabel ditampilkan beserta informasi cakupan *assignment* dalam bentuk *badge* berwarna, yaitu *badge* ungu untuk variabel bercakupan global dan *badge* biru muda untuk variabel bercakupan departemen, tipe nilai, rentang nilai minimum hingga maksimum, dan bobot variabel. Atasan dapat mengisi nilai aktual pada *field input* yang telah dibatasi sesuai rentang minimum dan maksimum variabel, serta menambahkan catatan opsional untuk setiap variabel.

Bagian bawah *modal* menyediakan tiga tombol aksi yaitu tombol Batal untuk menutup *modal* tanpa menyimpan, tombol Simpan *Draft* untuk menyimpan nilai sementara tanpa mengunci data sesuai URS-A-06, dan tombol *Submit* Penilaian untuk mengunci seluruh nilai yang telah diisi sesuai URS-A-07.

17. Halaman *Approval* Pengajuan (URS-A-03)

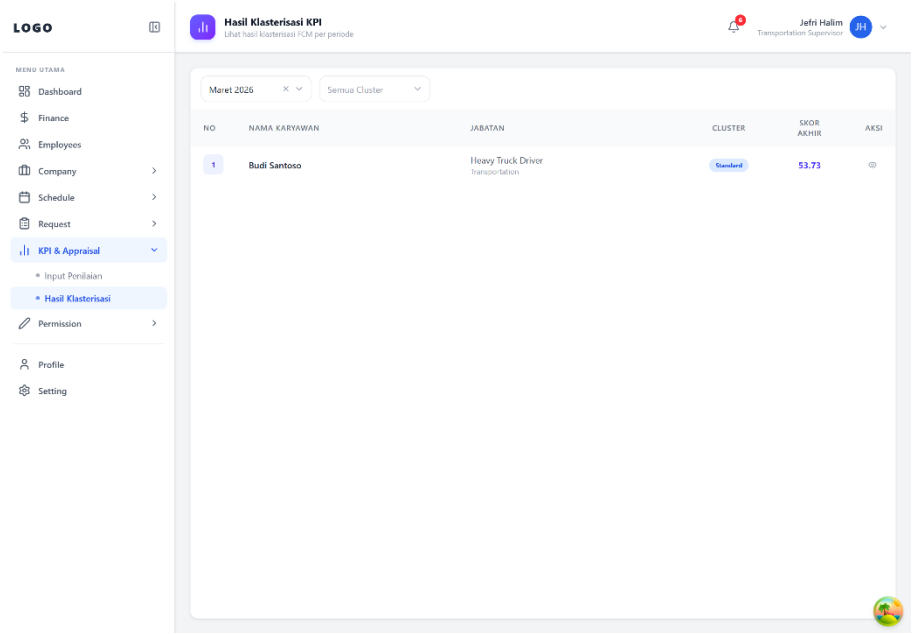


Gambar 27. Halaman *Approval* Pengajuan

Halaman *Approval* Pengajuan dapat diakses oleh Atasan untuk memproses pengajuan izin dan cuti bawahan berdasarkan *chain approval* yang telah dikonfigurasi. Bagian atas halaman menampilkan empat *summary card* yaitu Menunggu Saya yang menunjukkan jumlah pengajuan yang menunggu keputusan atasan yang sedang *login*, Disetujui Bulan Ini, Ditolak Bulan Ini, dan Total Halaman Ini.

Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama karyawan atau jenis izin, *filter* berdasarkan rentang tanggal, *filter* tipe pengajuan yaitu Semua Tipe, Izin/Cuti, dan Lembur, serta *filter* status yaitu Semua, *Pending*, Disetujui, dan Ditolak. Bagian kanan atas tabel menampilkan informasi jabatan Atasan yang sedang *login* sebagai konteks *level approval*. Tabel pengajuan menampilkan kolom *level approval*, tipe pengajuan dalam bentuk *badge* berwarna, nama karyawan pengaju, detail pengajuan beserta tanggal dan durasi, dan status pengajuan. Atasan dapat memproses setiap pengajuan melalui tombol aksi pada kolom kanan tabel.

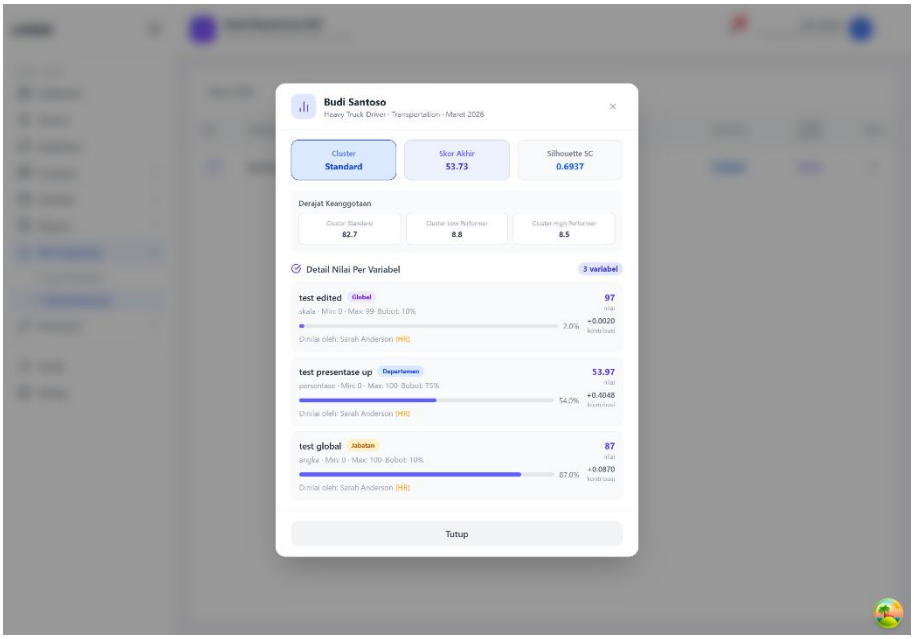
18. Halaman Hasil Klasterisasi KPI — Atasan (URS-A-08)



Gambar 28. Halaman Hasil Klasterisasi KPI Atasan

Halaman Hasil Klasterisasi KPI untuk peran Atasan menampilkan hasil klasterisasi FCM khusus bawahan langsung dari Atasan yang sedang *login*. Berbeda dengan tampilan HR yang menampilkan seluruh karyawan, halaman ini secara otomatis mem-*filter* data berdasarkan relasi atasan-bawahan sehingga Atasan hanya dapat melihat hasil klasterisasi kinerja bawahannya saja. Halaman ini dilengkapi dengan *selector* periode dan *filter* berdasarkan *cluster*. Tabel menampilkan kolom nama karyawan, jabatan beserta departemen, label *cluster*, skor akhir, dan tombol lihat detail.

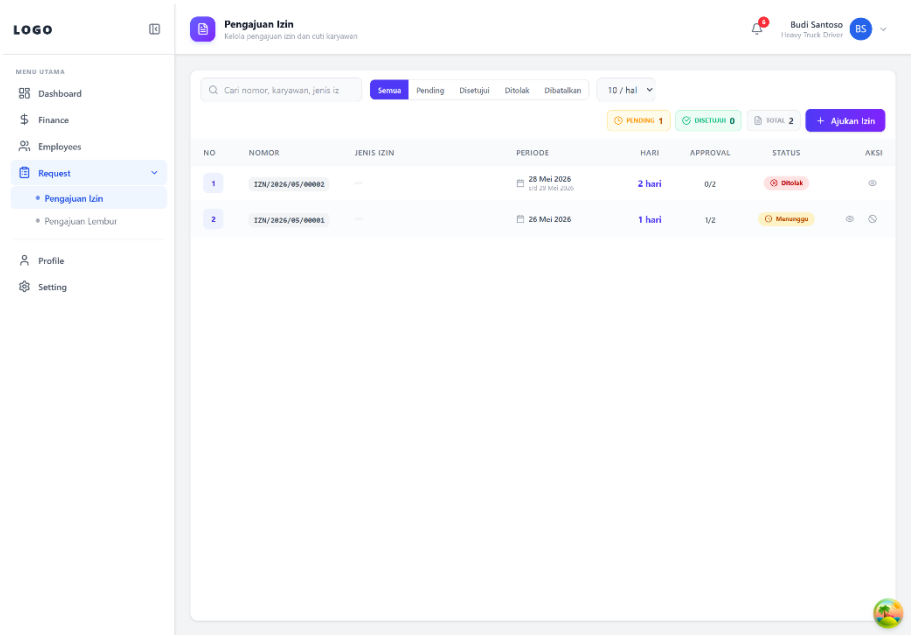
19. *Modal* Detail Nilai KPI (URS-A-08)



Gambar 29. *Modal* Detail Nilai KPI

Modal Detail Nilai KPI dapat diakses oleh Atasan dengan mengklik tombol lihat detail pada salah satu bawahan. *Modal* ini menampilkan nama karyawan, jabatan, departemen, dan periode penilaian di bagian atas. Bagian tengah menampilkan tiga indikator utama yaitu label *cluster*, skor akhir, dan nilai *Silhouette SC*. Di bawahnya terdapat bagian Derajat Keanggotaan yang menampilkan nilai keanggotaan karyawan terhadap setiap *cluster* dalam persentase, di mana hal ini merupakan keunggulan FCM sebagai *soft clustering*. Bagian bawah menampilkan Detail Nilai Per Variabel yang merinci setiap variabel KPI beserta cakupan *assignment* dalam bentuk *badge* berwarna, nilai aktual, persentase normalisasi, dan kontribusi skor beserta nama penilai.

20. Halaman Pengajuan Izin (URS-K-03)

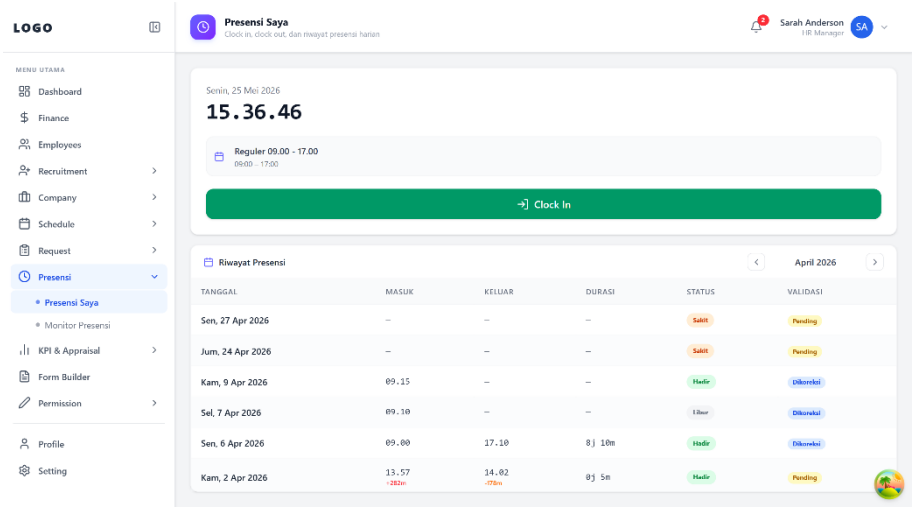


Gambar 30. Halaman Pengajuan Izin

Halaman Pengajuan Izin dapat diakses oleh Karyawan untuk mengajukan dan memantau status pengajuan izin. Halaman ini menampilkan riwayat pengajuan izin karyawan yang sedang *login* dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nomor, karyawan, atau jenis izin, serta *filter* status yaitu Semua, *Pending*, Disetujui, Ditolak, dan Dibatalkan.

Bagian atas tabel menampilkan ringkasan jumlah pengajuan berstatus *Pending* dan Disetujui beserta total keseluruhan pengajuan, serta tombol Ajukan Izin untuk membuat pengajuan baru. Tabel pengajuan menampilkan kolom nomor pengajuan, jenis izin, periode izin beserta tanggal mulai dan selesai, jumlah hari, *progress approval* yang menunjukkan *level* yang sudah menyetujui dari total *level* yang diperlukan, status pengajuan yang ditampilkan dalam bentuk *badge* berwarna seperti Ditolak berwarna merah dan Menunggu berwarna kuning, serta kolom aksi berupa tombol lihat detail dan batalkan pengajuan.

21. Halaman Presensi Saya (URS-K-04)

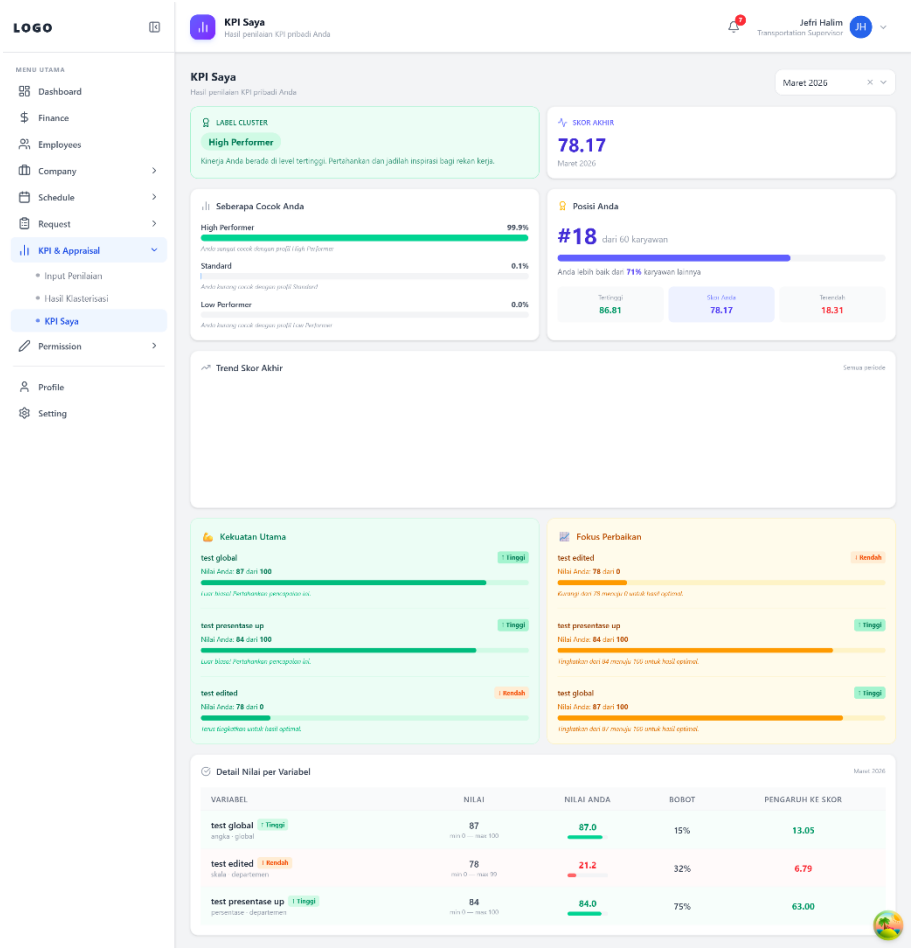


Gambar 31. Halaman Presensi Saya

Halaman Presensi Saya dapat diakses oleh Karyawan untuk melakukan *clock in*, *clock out*, dan memantau riwayat presensi harian. Bagian atas halaman menampilkan tanggal dan jam saat ini secara *real-time* beserta informasi jadwal kerja yang berlaku pada hari tersebut. Karyawan dapat melakukan *clock in* melalui tombol *Clock In* berwarna hijau yang mencolok, di mana tombol ini akan berubah menjadi tombol *Clock Out* setelah karyawan berhasil melakukan *clock in*.

Bagian bawah halaman menampilkan riwayat presensi karyawan yang dikelompokkan per bulan dengan navigasi panah untuk berpindah antar bulan. Tabel riwayat presensi menampilkan kolom tanggal, waktu masuk, waktu keluar, durasi kerja, status kehadiran yang ditampilkan dalam bentuk *badge* berwarna seperti Hadir, Sakit, dan Libur, serta status validasi yang menunjukkan apakah data presensi sudah divalidasi, sedang *pending*, atau telah dikoreksi oleh HR.

22. Halaman KPI Saya (URS-K-05, URS-K-06)



Gambar 32. Halaman KPI Saya

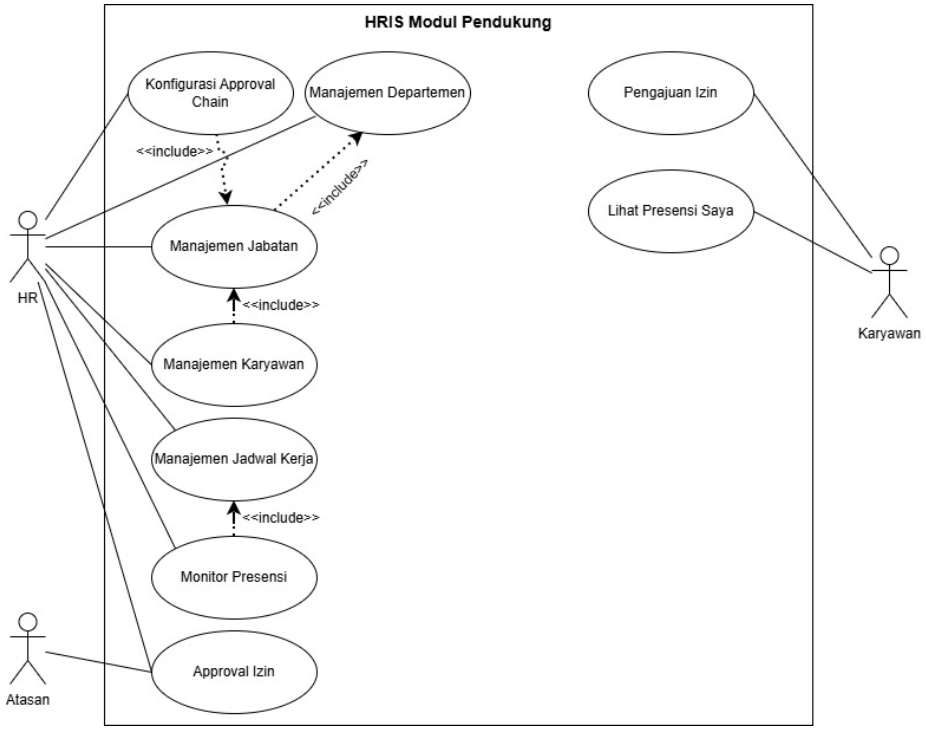
Halaman KPI Saya dapat diakses oleh Karyawan untuk melihat hasil penilaian KPI pribadi per periode, di mana bagian atas halaman terdapat *selector* periode yang memungkinkan karyawan memilih periode yang ingin ditampilkan. Halaman ini menampilkan Label *Cluster* yang menunjukkan hasil klasterisasi karyawan beserta pesan motivasi dan Skor Akhir yang menampilkan nilai skor karyawan pada periode tersebut. Panel Seberapa Cocok Anda menampilkan derajat keanggotaan karyawan terhadap setiap *cluster* dalam bentuk *progress bar* yang merupakan visualisasi konsep *soft clustering* FCM di mana setiap karyawan memiliki derajat keanggotaan terhadap semua *cluster* sekaligus, sedangkan Panel Posisi Anda menampilkan peringkat karyawan di antara seluruh karyawan beserta persentil dan perbandingan skor tertinggi, skor karyawan, dan skor terendah.

Bagian bawah halaman menampilkan *Trend* Skor Akhir yang menggambarkan perkembangan skor karyawan antar periode, panel Kekuatan Utama yang menunjukkan variabel KPI dengan performa terbaik beserta saran mempertahankan pencapaian, dan panel Fokus Perbaikan yang menunjukkan variabel yang perlu ditingkatkan beserta saran perbaikan. Bagian paling bawah menampilkan Detail Nilai per Variabel yang merinci setiap variabel KPI beserta nilai aktual, nilai karyawan yang dinormalisasi, bobot, dan pengaruh ke skor akhir.

4.2.6 Use Case Diagram

Use case diagram digunakan untuk menggambarkan hubungan antara aktor dengan fungsionalitas sistem berdasarkan kebutuhan fungsional yang telah diidentifikasi pada sub-bab 4.1.1. Sistem ini melibatkan tiga aktor yaitu HR, Atasan, dan Karyawan. Perancangan *use case diagram* dibagi menjadi dua diagram berdasarkan kelompok fungsionalitas sistem.

Diagram pertama menggambarkan *use case modul* pendukung sistem HRIS sebagaimana ditampilkan pada Gambar 33 berikut.

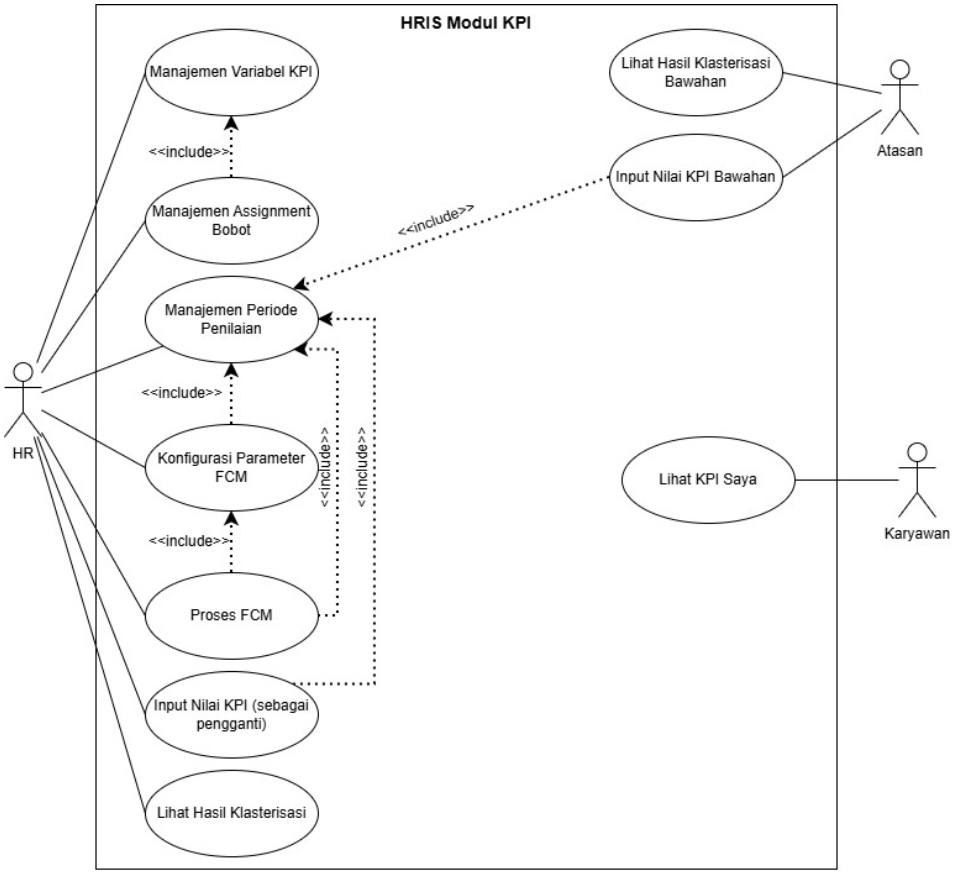


Gambar 33. *Use Case Diagram* HRIS Modul Pendukung

Diagram pertama mencakup fungsionalitas pengelolaan data master dan operasional harian sistem. HR dapat mengelola data karyawan, departemen,

jabatan, jadwal kerja, memantau presensi, mengkonfigurasi *approval chain*, dan memproses *approval* izin. Atasan dapat memproses *approval* izin bawahan. Karyawan dapat mengajukan izin dan memantau presensi pribadinya. Beberapa *use case* memiliki ketergantungan satu sama lain, di mana Manajemen Karyawan membutuhkan Manajemen Jabatan, Manajemen Jabatan membutuhkan Manajemen Departemen, *Monitor* Presensi membutuhkan Manajemen Jadwal Kerja, dan Konfigurasi *Approval Chain* membutuhkan Manajemen Jabatan.

Diagram kedua menggambarkan *use case* modul KPI yang menjadi fokus utama penelitian ini sebagaimana ditampilkan pada Gambar 34 berikut.



Gambar 34. *Use Case Diagram* HRIS Modul KPI

Diagram kedua mencakup fungsionalitas penilaian kinerja dan klasterisasi FCM. HR dapat mengelola variabel KPI, *assignment* bobot, periode penilaian, mengkonfigurasi parameter FCM, menjalankan proses FCM, melihat hasil klasterisasi, dan menginput nilai KPI sebagai pengganti

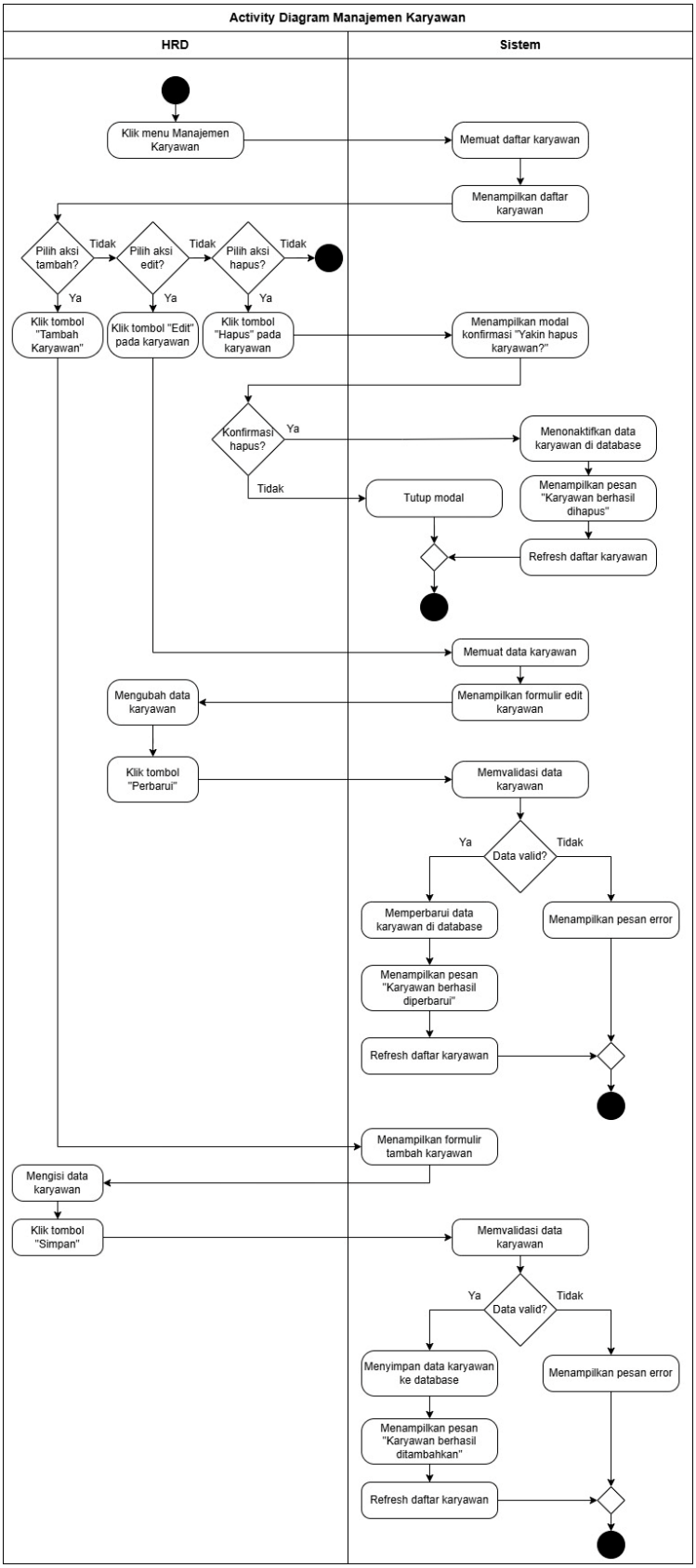
atasan jika diperlukan. Atasan dapat menginput nilai KPI bawahan dan melihat hasil klasterisasi bawahan. Karyawan dapat melihat hasil klasterisasi kinerja dirinya sendiri melalui halaman KPI Saya. Beberapa *use case* memiliki ketergantungan satu sama lain, di mana Manajemen *Assignment* Bobot membutuhkan Manajemen Variabel KPI, Konfigurasi Parameter FCM dan Proses FCM membutuhkan Manajemen Periode Penilaian, serta Proses FCM juga membutuhkan Konfigurasi Parameter FCM. *Input* Nilai KPI sebagai pengganti merupakan perluasan opsional dari *Input* Nilai KPI Bawahan yang hanya dilakukan HR dalam kondisi tertentu.

Di luar ketiga aktor yang tergambar pada use case diagram, terdapat pihak Staff IT yang bertindak sebagai pekerja lepas dengan tanggung jawab penuh atas keseluruhan aplikasi HRIS. Staff IT bertanggung jawab pada aspek teknis sistem, meliputi deployment, konfigurasi server, pemeliharaan aplikasi, penanganan permasalahan teknis, serta pengelolaan akses awal pengguna ke sistem. Peran ini tidak digambarkan dalam use case diagram karena aktivitasnya berada di luar interaksi fungsional bisnis, melainkan pada tingkat operasional teknis yang menopang berjalannya sistem secara keseluruhan.

4.2.7 Activity Diagram

Activity diagram digunakan untuk menggambarkan alur aktivitas dari setiap fungsionalitas sistem yang telah diidentifikasi pada *use case diagram* di sub-bab 4.2.6. Diagram ini memperlihatkan urutan aktivitas yang dilakukan oleh aktor dan respons yang diberikan oleh sistem, sehingga interaksi antara pengguna dan sistem dapat dipahami secara terperinci. Setiap *activity diagram* disajikan dalam bentuk dua kolom yang memisahkan tanggung jawab antara aktor dan sistem, di mana aktor merepresentasikan pengguna yang melakukan aksi dan sistem merepresentasikan respons yang diberikan terhadap aksi tersebut. Perancangan *activity diagram* dikelompokkan berdasarkan pembagian modul pada *use case diagram*, yaitu *activity diagram* untuk modul pendukung sistem HRIS dan *activity diagram* untuk modul KPI.

1. *Activity Diagram* Manajemen Karyawan



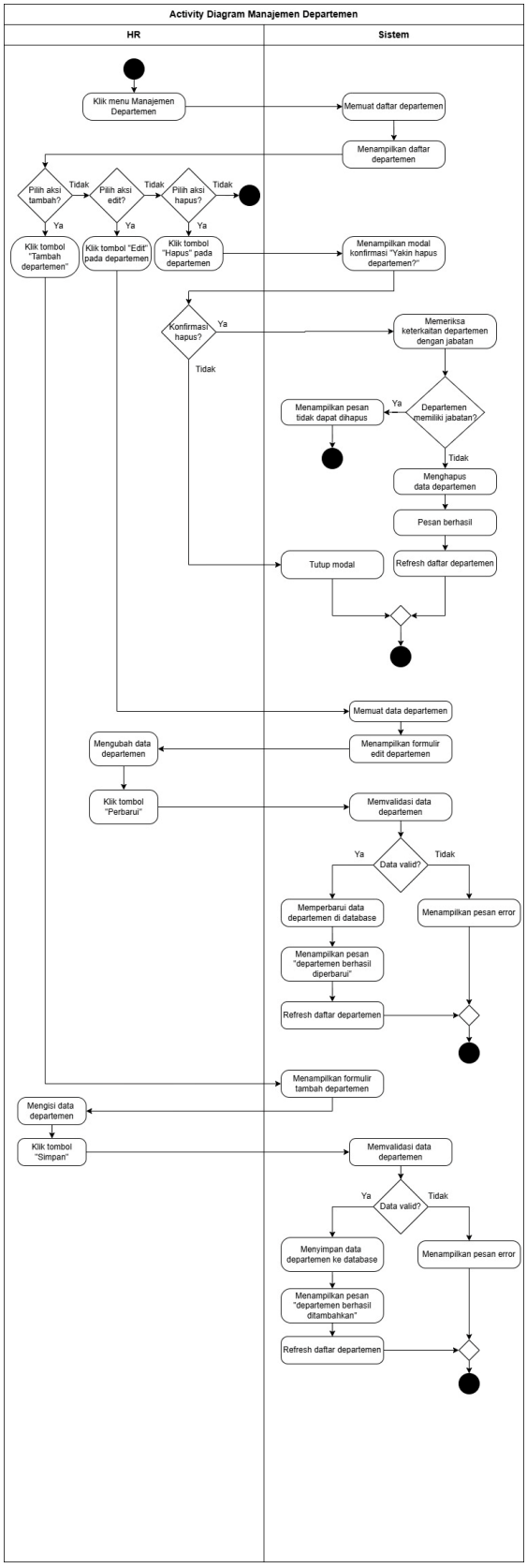
Gambar 35. *Activity Diagram* Manajemen Karyawan

Activity Diagram Manajemen Karyawan menggambarkan alur aktivitas pengelolaan data karyawan oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data karyawan. Alur dimulai ketika HR mengakses menu Manajemen Karyawan dan sistem menampilkan daftar karyawan, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan, di mana apabila tidak ada aksi yang dipilih, alur berakhir tanpa operasi apa pun.

Pada aksi penambahan, HR menekan tombol Tambah Karyawan sehingga sistem menampilkan formulir tambah karyawan, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris karyawan yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar karyawan, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris karyawan yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem menonaktifkan data karyawan di basis data, menampilkan pesan keberhasilan, dan memperbarui daftar karyawan.

2. *Activity Diagram* Manajemen Departemen



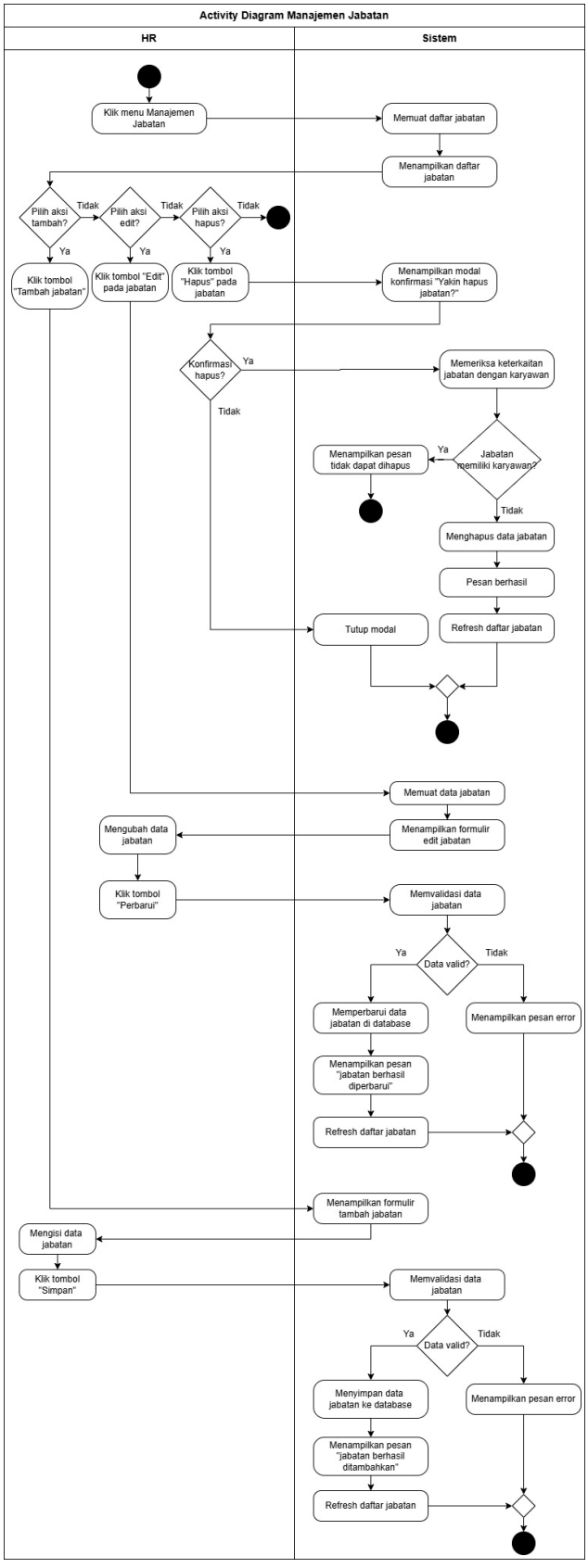
Gambar 36. *Activity Diagram* Manajemen Departemen

Activity Diagram Manajemen Departemen menggambarkan alur aktivitas pengelolaan data departemen oleh aktor HR. Alur dimulai ketika HR mengakses menu Manajemen Departemen dan sistem menampilkan daftar departemen yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Departemen sehingga sistem menampilkan formulir tambah departemen, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris departemen yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data, menampilkan pesan keberhasilan, dan memperbarui daftar departemen, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris departemen yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem memeriksa keterkaitan departemen dengan jabatan untuk menjaga integritas data agar tidak terjadi referensi yang menggantung. Jika departemen masih memiliki jabatan terkait, sistem menampilkan pesan bahwa departemen tidak dapat dihapus dan alur berakhir, sedangkan apabila tidak memiliki jabatan terkait, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar departemen.

3. *Activity Diagram* Manajemen Jabatan



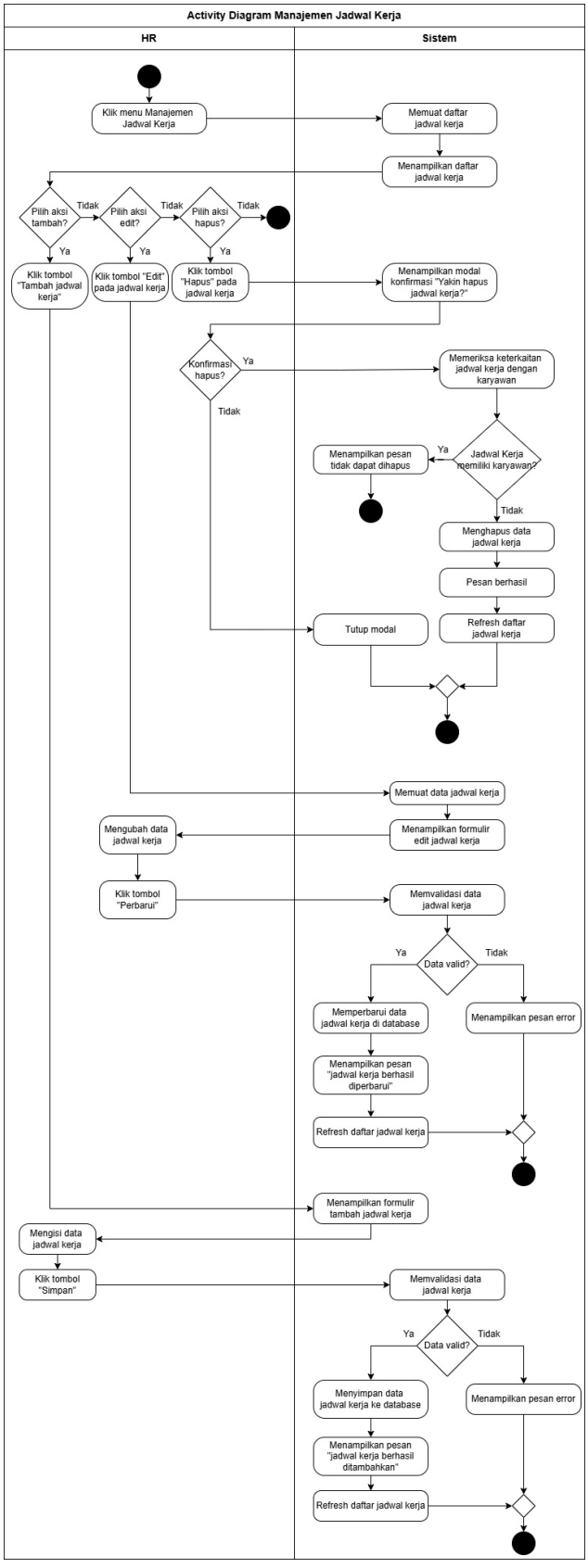
Gambar 37. *Activity Diagram* Manajemen Jabatan

Activity Diagram Manajemen Jabatan menggambarkan alur aktivitas pengelolaan data jabatan oleh aktor HR. Alur dimulai ketika HR mengakses menu Manajemen Jabatan dan sistem menampilkan daftar jabatan yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Jabatan sehingga sistem menampilkan formulir tambah jabatan, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris jabatan yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar jabatan, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris jabatan yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem memeriksa keterkaitan jabatan dengan karyawan untuk menjaga integritas data agar tidak terjadi referensi yang menggantung. Jika jabatan masih memiliki karyawan terkait, sistem menampilkan pesan bahwa jabatan tidak dapat dihapus dan alur berakhir, sedangkan apabila tidak memiliki karyawan terkait, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar jabatan.

4. *Activity Diagram* Manajemen Jadwal Kerja



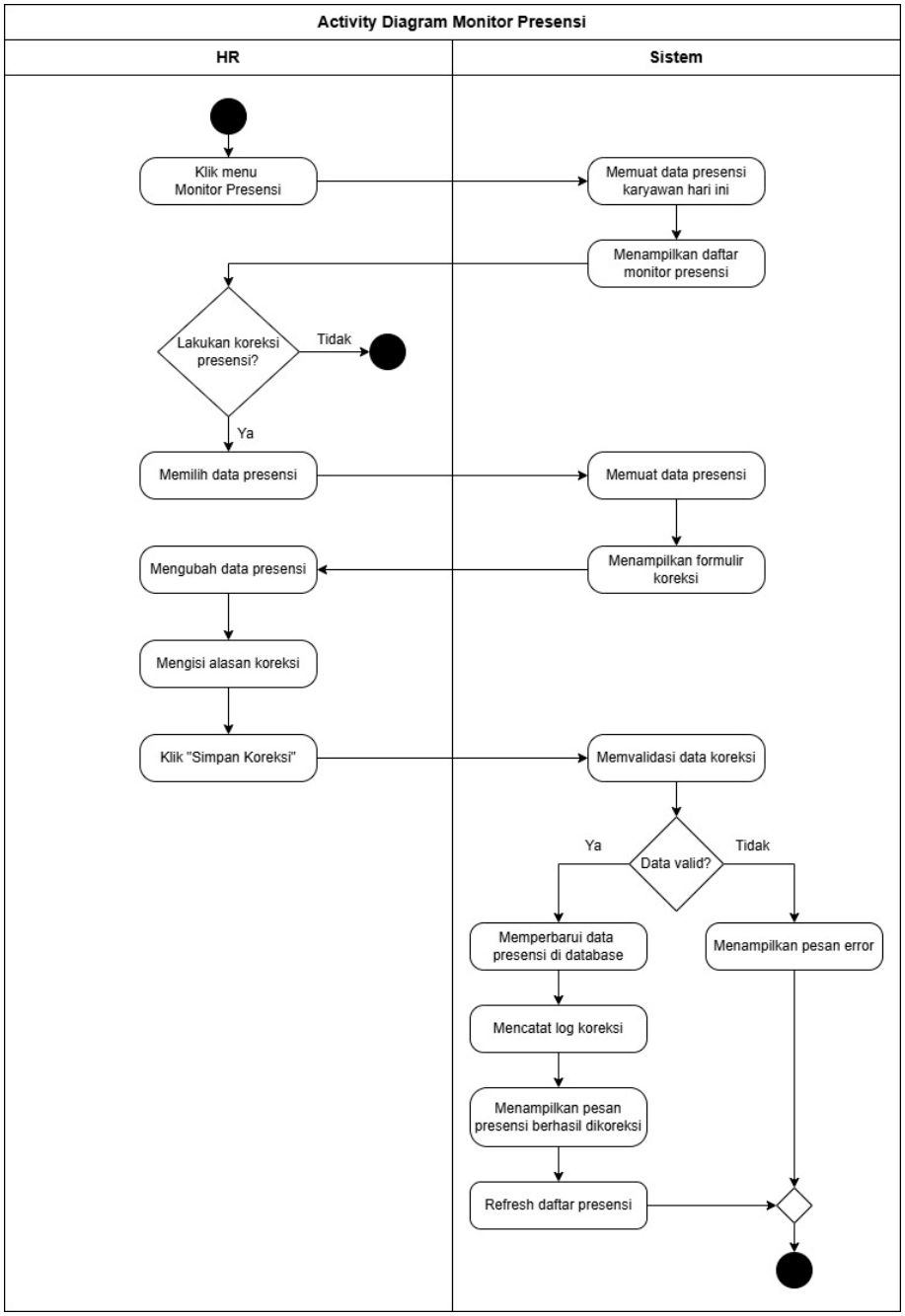
Gambar 38. *Activity Diagram* Manajemen Jadwal Kerja

Activity Diagram Manajemen Jadwal Kerja menggambarkan alur aktivitas pengelolaan data jadwal kerja oleh aktor HR. Alur dimulai ketika HR mengakses menu Manajemen Jadwal Kerja dan sistem menampilkan daftar jadwal kerja yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Jadwal Kerja sehingga sistem menampilkan formulir tambah jadwal kerja, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris jadwal kerja yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar jadwal kerja, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris jadwal kerja yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem memeriksa keterkaitan jadwal kerja dengan karyawan untuk menjaga integritas data agar tidak terjadi referensi yang menggantung. Jika jadwal kerja masih digunakan karyawan, sistem menampilkan pesan bahwa jadwal kerja tidak dapat dihapus dan alur berakhir, sedangkan apabila tidak digunakan karyawan, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar jadwal kerja.

5. *Activity Diagram Monitor* Presensi



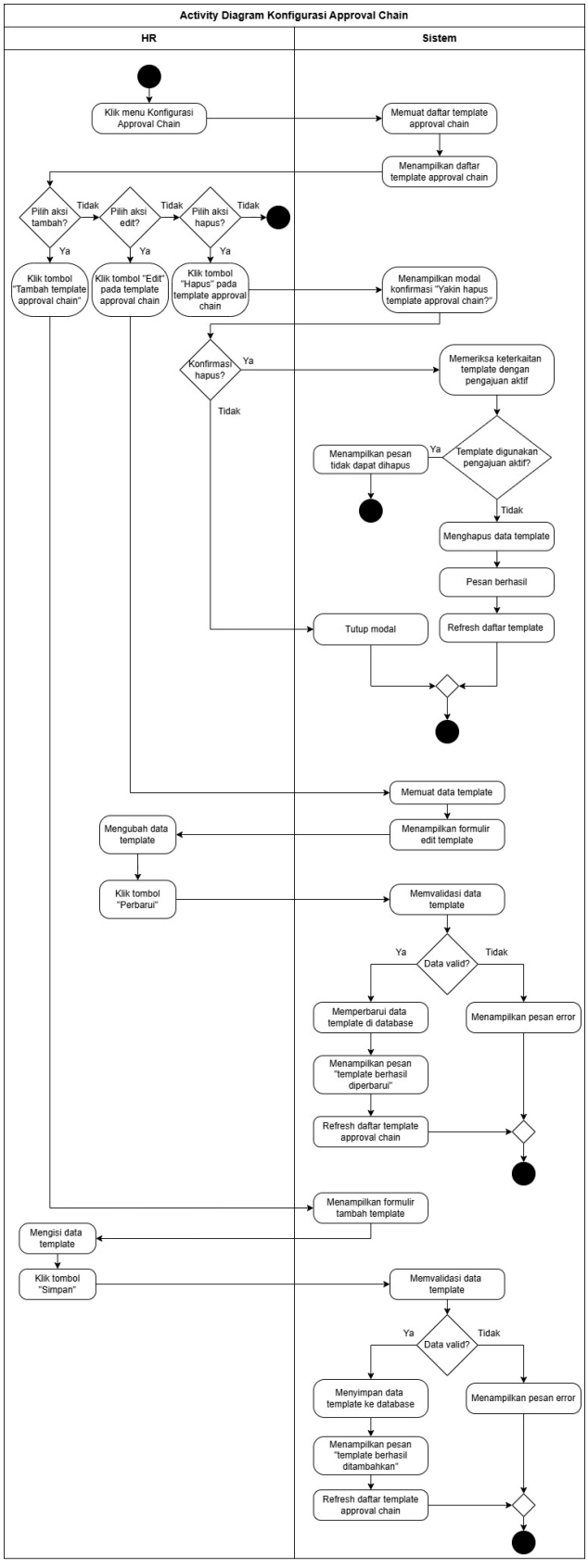
Gambar 39. *Activity Diagram Monitor* Presensi

Activity Diagram Monitor Presensi menggambarkan alur aktivitas pemantauan dan koreksi data presensi karyawan yang dilakukan oleh aktor HR. Alur aktivitas dimulai ketika HR mengakses menu *Monitor* Presensi, kemudian sistem memuat dan menampilkan daftar *monitor* presensi karyawan hari ini secara *real-time* yang mencakup status kehadiran, waktu *clock in*, waktu *clock out*, dan keterlambatan. HR

selanjutnya dapat memilih untuk melakukan koreksi data presensi atau keluar dari halaman tanpa melakukan aksi apa pun.

Pada aksi koreksi presensi, HR memilih data presensi karyawan yang ingin dikoreksi, kemudian sistem memuat data presensi dan menampilkan formulir koreksi. HR mengubah data presensi sesuai kebutuhan dan mengisi alasan koreksi sebagai dasar perubahan, lalu menekan tombol Simpan Koreksi. Sistem memvalidasi data koreksi yang dikirimkan, di mana proses pembaruan hanya dilakukan apabila data dinyatakan valid. Jika data valid, sistem memperbarui data presensi di basis data, mencatat *log* koreksi sebagai jejak audit untuk menjaga akuntabilitas perubahan data, menampilkan pesan keberhasilan, dan memperbarui daftar *monitor* presensi. Sebaliknya, apabila data tidak valid, sistem menampilkan pesan kesalahan dan alur aktivitas berakhir.

6. *Activity Diagram* Konfigurasi *Approval Chain*



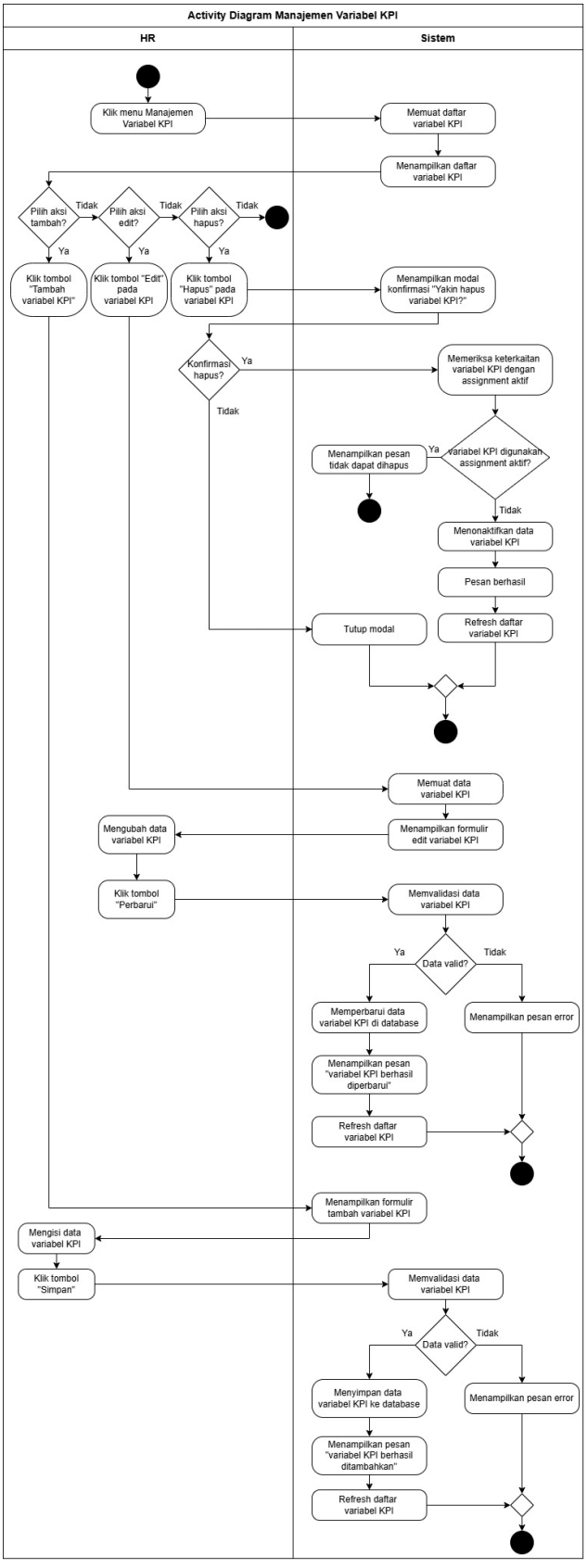
Gambar 40. *Activity Diagram* Konfigurasi *Approval Chain*

Activity Diagram Konfigurasi *Approval Chain* menggambarkan alur aktivitas pengelolaan *template approval chain* oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data *template*. *Template approval chain* merupakan aturan yang menentukan alur persetujuan pengajuan izin atau lembur berdasarkan jenis pengajuan dan departemen tertentu. Alur dimulai ketika HR mengakses menu Konfigurasi *Approval Chain* dan sistem menampilkan daftar *template* yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah *Template* sehingga sistem menampilkan formulir tambah *template*, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris *template* yang ingin diubah sehingga sistem menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar *template*, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris *template* yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan, sistem menutup *modal* tanpa perubahan, sedangkan apabila HR mengonfirmasi, sistem memeriksa keterkaitan *template* dengan pengajuan aktif untuk menjaga integritas proses persetujuan yang sedang berjalan. Jika *template* masih digunakan pengajuan aktif, sistem menampilkan pesan tidak dapat dihapus, sedangkan apabila tidak digunakan, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar *template*.

7. *Activity Diagram* Manajemen Variabel KPI



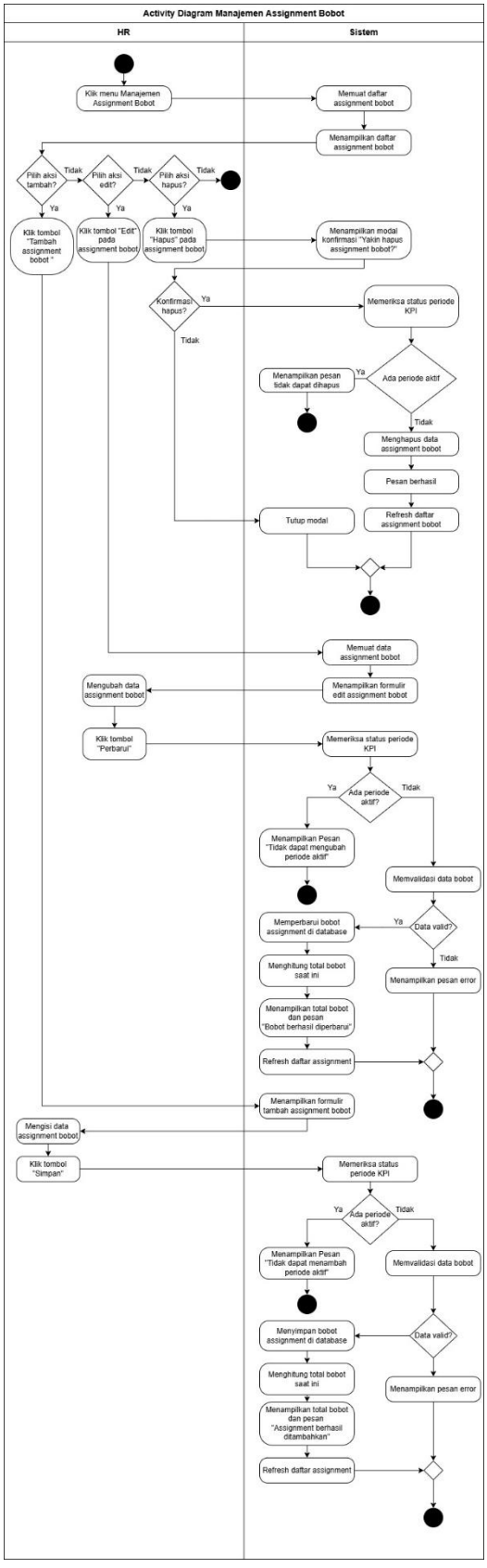
Gambar 41. *Activity Diagram* Manajemen Variabel KPI

Activity Diagram Manajemen Variabel KPI menggambarkan alur aktivitas pengelolaan data variabel KPI oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data variabel KPI. Alur dimulai ketika HR mengakses menu Manajemen Variabel KPI dan sistem menampilkan daftar variabel KPI yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan, di mana apabila tidak ada aksi yang dipilih, alur berakhir tanpa operasi apa pun.

Pada aksi penambahan, HR menekan tombol Tambah Variabel KPI sehingga sistem menampilkan formulir tambah variabel KPI, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris variabel KPI yang ingin diubah sehingga sistem menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar variabel KPI, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris variabel KPI yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan, sistem menutup *modal* tanpa perubahan, sedangkan apabila HR mengonfirmasi, sistem memeriksa keterkaitan variabel KPI dengan *assignment* aktif. Jika variabel KPI masih digunakan *assignment* aktif, sistem menampilkan pesan tidak dapat dihapus, sedangkan apabila tidak digunakan, sistem menonaktifkan data variabel KPI dengan mekanisme *soft delete*, menampilkan pesan keberhasilan, dan memperbarui daftar variabel KPI.

8. *Activity Diagram* Manajemen *Assignment* Bobot



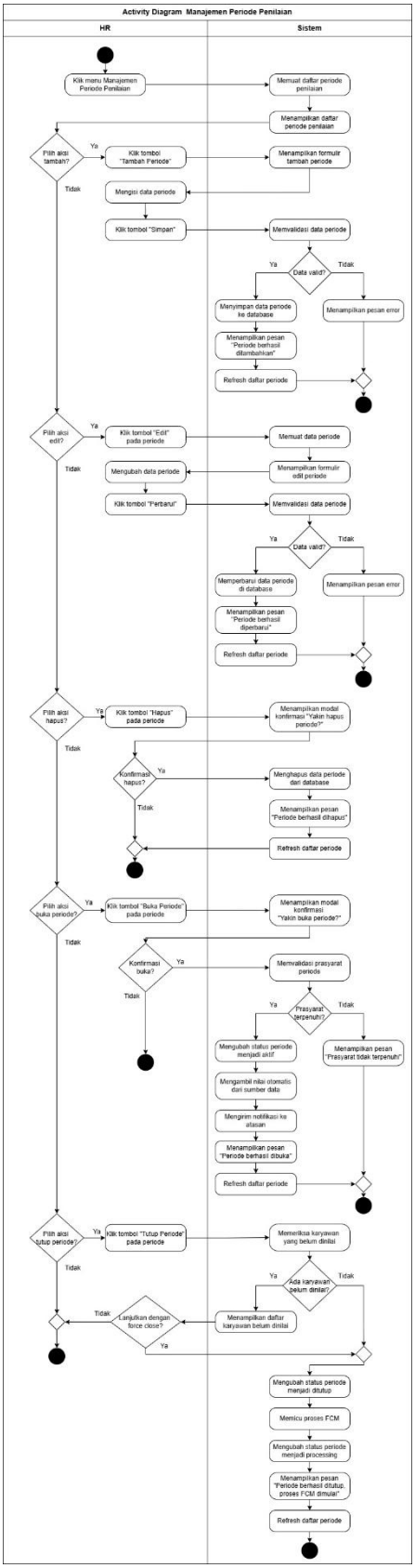
Gambar 42. *Activity Diagram* Manajemen *Assignment* Bobot

Activity Diagram Manajemen *Assignment* Bobot menggambarkan alur aktivitas pengelolaan *assignment* bobot variabel KPI oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data *assignment*. Alur dimulai ketika HR mengakses menu Manajemen *Assignment* Bobot dan sistem menampilkan daftar *assignment* yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan atau keluar dari halaman tanpa operasi.

Pada aksi penambahan, HR menekan tombol Tambah *Assignment* Bobot lalu mengisi data pada formulir dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris *assignment* lalu mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem terlebih dahulu memeriksa status periode KPI sebagai mekanisme penjagaan agar bobot tidak dapat diubah saat periode berjalan, di mana apabila terdapat periode aktif sistem menampilkan pesan tidak dapat ditambahkan atau diubah, sedangkan apabila tidak ada sistem memvalidasi data. Jika data valid, sistem menyimpan atau memperbarui *assignment*, menghitung total bobot saat ini, menampilkan pesan keberhasilan beserta total bobot, dan memperbarui daftar *assignment*.

Pada aksi penghapusan, HR menekan tombol Hapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan, sistem menutup *modal* tanpa perubahan, sedangkan apabila HR mengonfirmasi, sistem memeriksa status periode KPI, di mana jika terdapat periode aktif sistem menampilkan pesan tidak dapat dihapus, sedangkan apabila tidak ada, sistem menghapus data *assignment*, menampilkan pesan keberhasilan, dan memperbarui daftar *assignment*.

9. *Activity Diagram* Manajemen Periode Penilaian



Gambar 43. *Activity Diagram* Manajemen Periode Penilaian

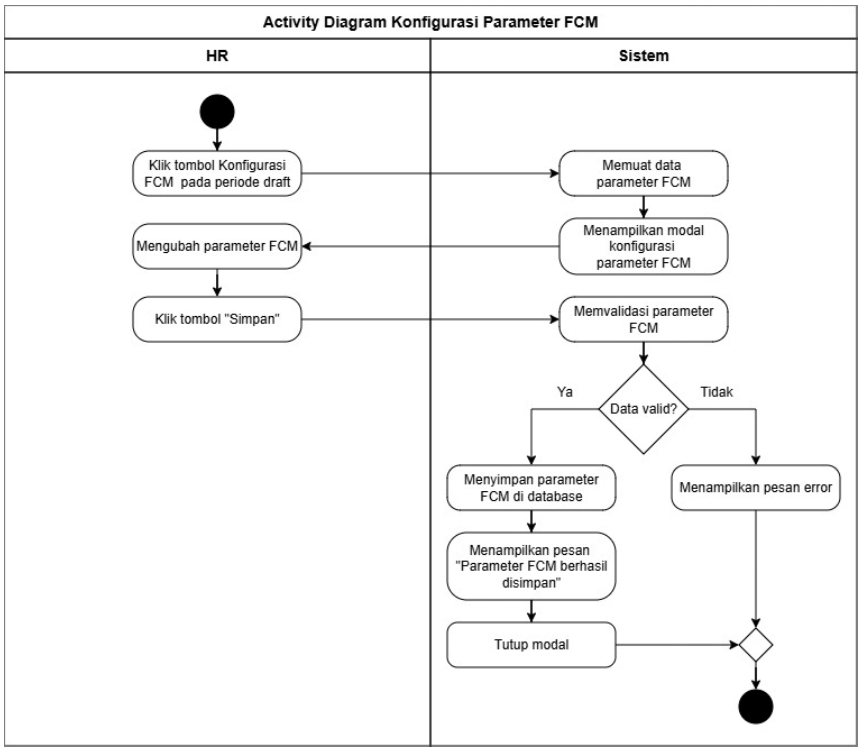
Activity Diagram Manajemen Periode Penilaian menggambarkan alur aktivitas pengelolaan periode penilaian KPI oleh aktor HR yang mencakup lima operasi utama, yaitu penambahan, pembaruan, penghapusan, pembukaan, dan penutupan periode. Periode penilaian merupakan entitas inti yang menjadi kerangka waktu pelaksanaan penilaian kinerja karyawan, di mana setiap periode memiliki rentang tanggal mulai dan tanggal selesai serta status yang berubah secara berurutan dari *draft* menjadi aktif, kemudian ditutup, *processing*, dan akhirnya selesai. Alur dimulai ketika HR mengakses menu Manajemen Periode Penilaian dan sistem menampilkan daftar periode beserta statusnya, kemudian HR dapat memilih salah satu dari lima aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Periode lalu mengisi data periode yang mencakup bulan, tahun, tanggal mulai, dan tanggal selesai, kemudian menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris periode yang ingin diubah sehingga sistem menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pembaruan dan penghapusan hanya dapat dilakukan pada periode yang masih berstatus *draft*. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan dengan memeriksa kelengkapan data, kesesuaian rentang tanggal, dan keunikan kombinasi bulan dan tahun, di mana jika valid sistem akan menyimpan atau memperbarui data, menampilkan pesan keberhasilan, dan memperbarui daftar periode, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan. Pada aksi penghapusan, HR menekan tombol Hapus sehingga sistem menampilkan *modal* konfirmasi, di mana apabila HR mengonfirmasi sistem akan menghapus data periode dari basis data.

Pada aksi pembukaan periode, sistem memvalidasi prasyarat yang mencakup tidak adanya periode lain yang sedang aktif dan total bobot variabel KPI untuk seluruh jabatan aktif harus sebesar 100% setelah resolusi tiga *layer* melalui mekanisme *Decorator Pattern*, di mana jika

terpenuhi sistem akan mengubah status periode menjadi aktif, mengambil nilai otomatis dari sumber data sistem, dan mengirim notifikasi kepada seluruh atasan aktif. Pada aksi penutupan periode, sistem memeriksa karyawan yang belum dinilai dan menyediakan opsi *force close* untuk menutup periode meskipun terdapat karyawan yang belum dinilai (karyawan tersebut akan dilewati dari proses klasterisasi). Apabila penutupan dilanjutkan, sistem mengubah status periode menjadi ditutup, memicu proses FCM secara *asynchronous* melalui antrian pekerjaan, dan mengubah status periode menjadi *processing*.

10. *Activity Diagram* Konfigurasi Parameter FCM

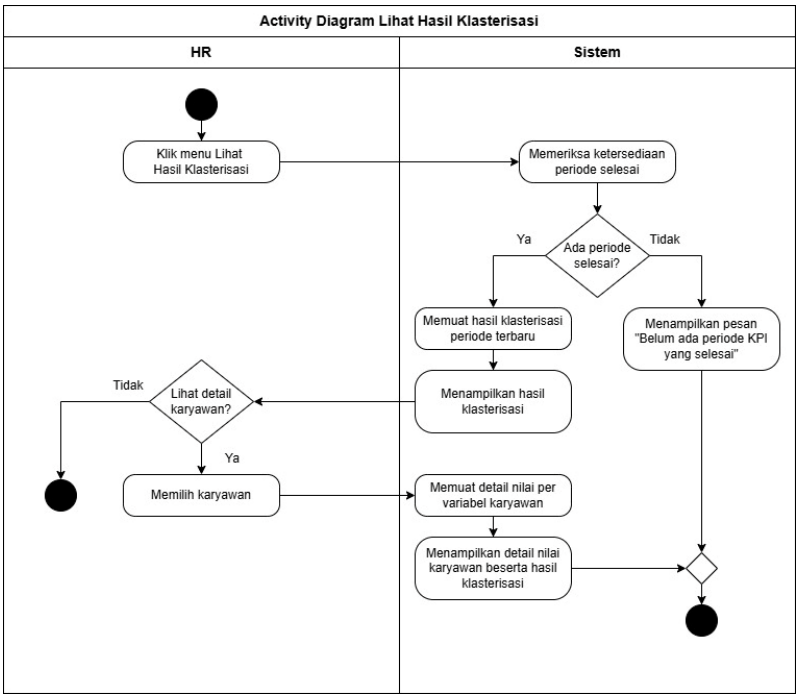


Gambar 44. *Activity Diagram* Konfigurasi Parameter FCM

Activity Diagram Konfigurasi Parameter FCM menggambarkan alur aktivitas pengaturan parameter algoritma *Fuzzy C-Means* oleh aktor HR pada periode penilaian yang berstatus *draft*. Parameter FCM yang dapat diatur mencakup jumlah *cluster*, pangkat pembobot, jumlah iterasi maksimum, dan ambang batas *error* minimum, di mana pengaturan ini hanya dapat dilakukan sebelum periode dibuka karena nilai parameter akan menjadi acuan proses klasterisasi kinerja karyawan ketika periode ditutup.

Alur dimulai ketika HR menekan tombol Konfigurasi FCM pada salah satu periode berstatus *draft* di halaman Manajemen Periode Penilaian, sehingga sistem memuat data parameter FCM yang saat ini berlaku dan menampilkan *modal* konfigurasi. HR mengubah parameter sesuai kebutuhan lalu menekan tombol Simpan, kemudian sistem memvalidasi parameter yang dikirimkan, di mana apabila valid sistem menyimpan parameter FCM ke basis data, menampilkan pesan keberhasilan, dan menutup *modal*, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

11. *Activity Diagram* Lihat Hasil Klasterisasi



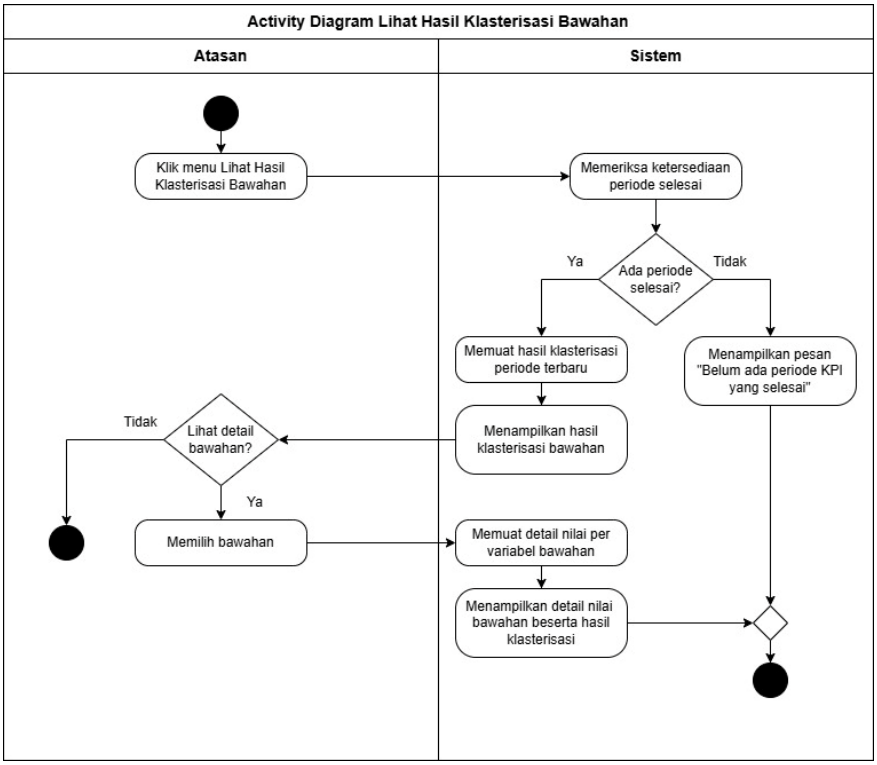
Gambar 45. *Activity Diagram* Lihat Hasil Klasterisasi

Activity Diagram Lihat Hasil Klasterisasi menggambarkan alur aktivitas pemantauan hasil klasterisasi kinerja karyawan oleh aktor HR setelah proses *Fuzzy C-Means* selesai. Alur dimulai ketika HR mengakses menu Lihat Hasil Klasterisasi, kemudian sistem memeriksa ketersediaan periode KPI yang sudah selesai, di mana apabila tidak terdapat periode yang sudah selesai sistem menampilkan pesan bahwa belum ada periode KPI yang selesai dan alur berakhir, sedangkan apabila terdapat periode selesai sistem memuat hasil klasterisasi dari periode terbaru secara otomatis yang mencakup daftar karyawan

beserta *cluster* yang ditempati, skor akhir, dan ringkasan jumlah karyawan per *cluster*.

HR selanjutnya dapat memilih untuk melihat detail nilai karyawan tertentu atau keluar dari halaman tanpa aksi. Pada aksi lihat detail karyawan, HR memilih karyawan yang ingin dilihat detailnya, kemudian sistem memuat dan menampilkan detail nilai per variabel karyawan tersebut beserta hasil klasterisasinya yang mencakup nilai aktual, nilai ternormalisasi, bobot, kontribusi terhadap skor akhir, dan derajat keanggotaan terhadap setiap *cluster* sebagai representasi konsep *soft clustering* pada *Fuzzy C-Means*.

12. *Activity Diagram* Lihat Hasil Klasterisasi Bawahan



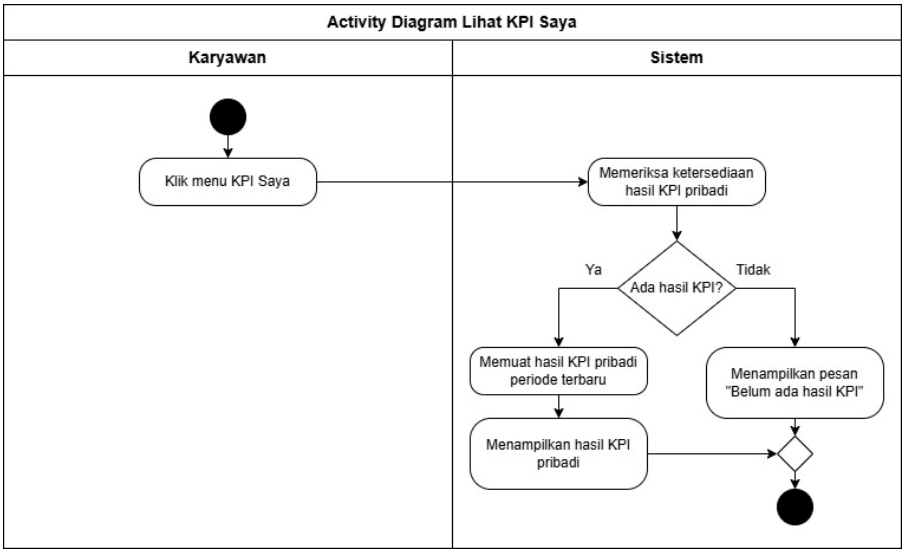
Gambar 46. *Activity Diagram* Lihat Hasil Klasterisasi Bawahan

Activity Diagram Lihat Hasil Klasterisasi Bawahan menggambarkan alur aktivitas pemantauan hasil klasterisasi kinerja karyawan oleh aktor Atasan setelah proses *Fuzzy C-Means* selesai. Atasan hanya dapat melihat hasil klasterisasi karyawan yang berada langsung di bawah jabatannya sebagai bawahan, di mana akses ini dibatasi oleh relasi atasan-bawahan pada struktur jabatan. Alur dimulai ketika Atasan

mengakses menu Lihat Hasil Klasterisasi Bawahan, kemudian sistem memeriksa ketersediaan periode KPI yang sudah selesai, di mana apabila tidak terdapat periode yang sudah selesai sistem menampilkan pesan bahwa belum ada periode KPI yang selesai dan alur berakhir, sedangkan apabila terdapat periode selesai sistem memuat hasil klasterisasi bawahan dari periode terbaru secara otomatis yang mencakup daftar bawahan beserta *cluster* yang ditempati, skor akhir, dan ringkasan jumlah bawahan per *cluster*.

Atasan selanjutnya dapat memilih untuk melihat detail nilai bawahan tertentu atau keluar dari halaman tanpa aksi. Pada aksi lihat detail bawahan, Atasan memilih bawahan yang ingin dilihat detailnya, kemudian sistem memuat dan menampilkan detail nilai per variabel bawahan tersebut beserta hasil klasterisasinya yang mencakup nilai aktual, nilai ternormalisasi, bobot, kontribusi terhadap skor akhir, dan derajat keanggotaan terhadap setiap *cluster* sebagai representasi konsep *soft clustering* pada *Fuzzy C-Means*.

13. *Activity Diagram* Lihat KPI Saya



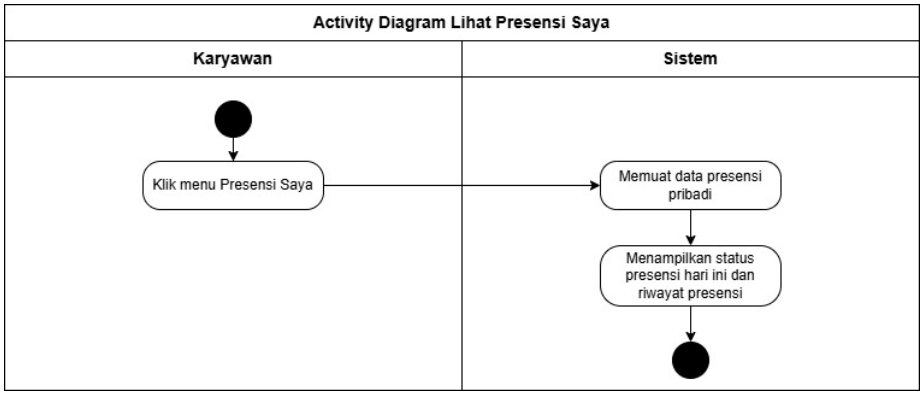
Gambar 47. *Activity Diagram* Lihat KPI Saya

Activity Diagram Lihat KPI Saya menggambarkan alur aktivitas pemantauan hasil penilaian KPI pribadi yang dilakukan oleh aktor Karyawan. Alur dimulai ketika Karyawan mengakses menu KPI Saya, kemudian sistem memeriksa ketersediaan hasil KPI pribadi karyawan

tersebut, di mana apabila Karyawan belum memiliki hasil KPI sistem menampilkan pesan bahwa belum ada hasil KPI dan alur berakhir.

Apabila Karyawan sudah memiliki hasil KPI, sistem memuat hasil KPI pribadi dari periode terbaru secara otomatis dan menampilkannya kepada Karyawan. Hasil KPI yang ditampilkan mencakup label *cluster* beserta deskripsi pencapaian kinerja, skor akhir, derajat keanggotaan terhadap setiap *cluster* sebagai representasi konsep *soft clustering* pada *Fuzzy C-Means*, posisi peringkat di antara seluruh karyawan beserta persentil, *trend* skor akhir antar periode, variabel KPI yang menjadi kekuatan utama dan fokus perbaikan, serta detail nilai per variabel KPI lengkap dengan nilai aktual, nilai ternormalisasi, bobot, dan kontribusi terhadap skor akhir.

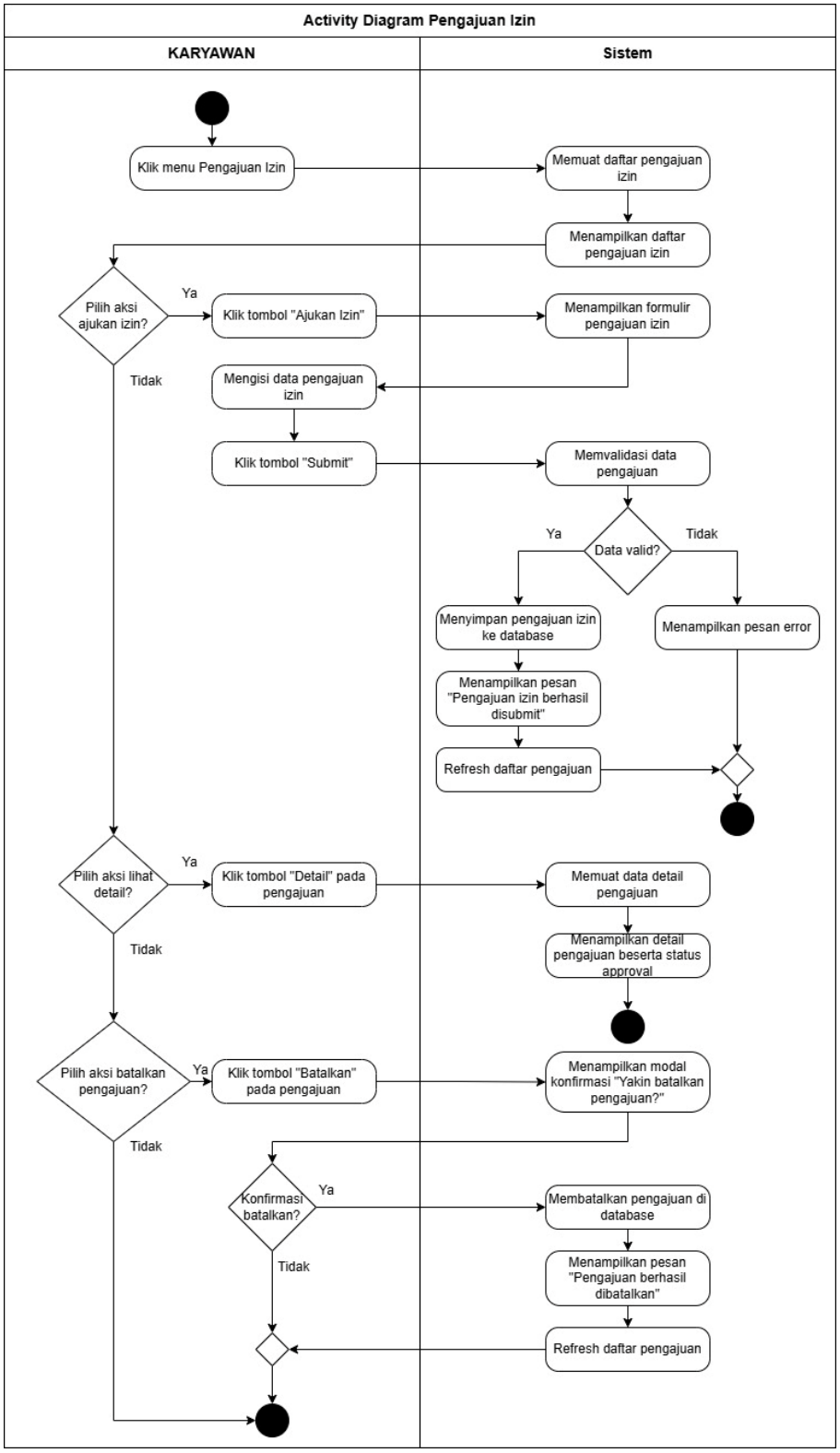
14. *Activity Diagram* Lihat Presensi Saya



Gambar 48. *Activity Diagram* Lihat Presensi Saya

Activity Diagram Lihat Presensi Saya menggambarkan alur aktivitas pemantauan data presensi pribadi yang dilakukan oleh aktor Karyawan. Alur aktivitas dimulai ketika Karyawan mengakses menu Presensi Saya, kemudian sistem memuat data presensi pribadi karyawan tersebut dan menampilkannya kepada Karyawan. Data presensi yang ditampilkan mencakup status presensi hari ini yang terdiri atas jadwal kerja, waktu *clock in*, waktu *clock out*, total jam kerja, status kehadiran, dan status validasi, serta riwayat presensi bulanan yang menampilkan rekapitulasi presensi pada bulan berjalan.

15. *Activity Diagram* Pengajuan Izin



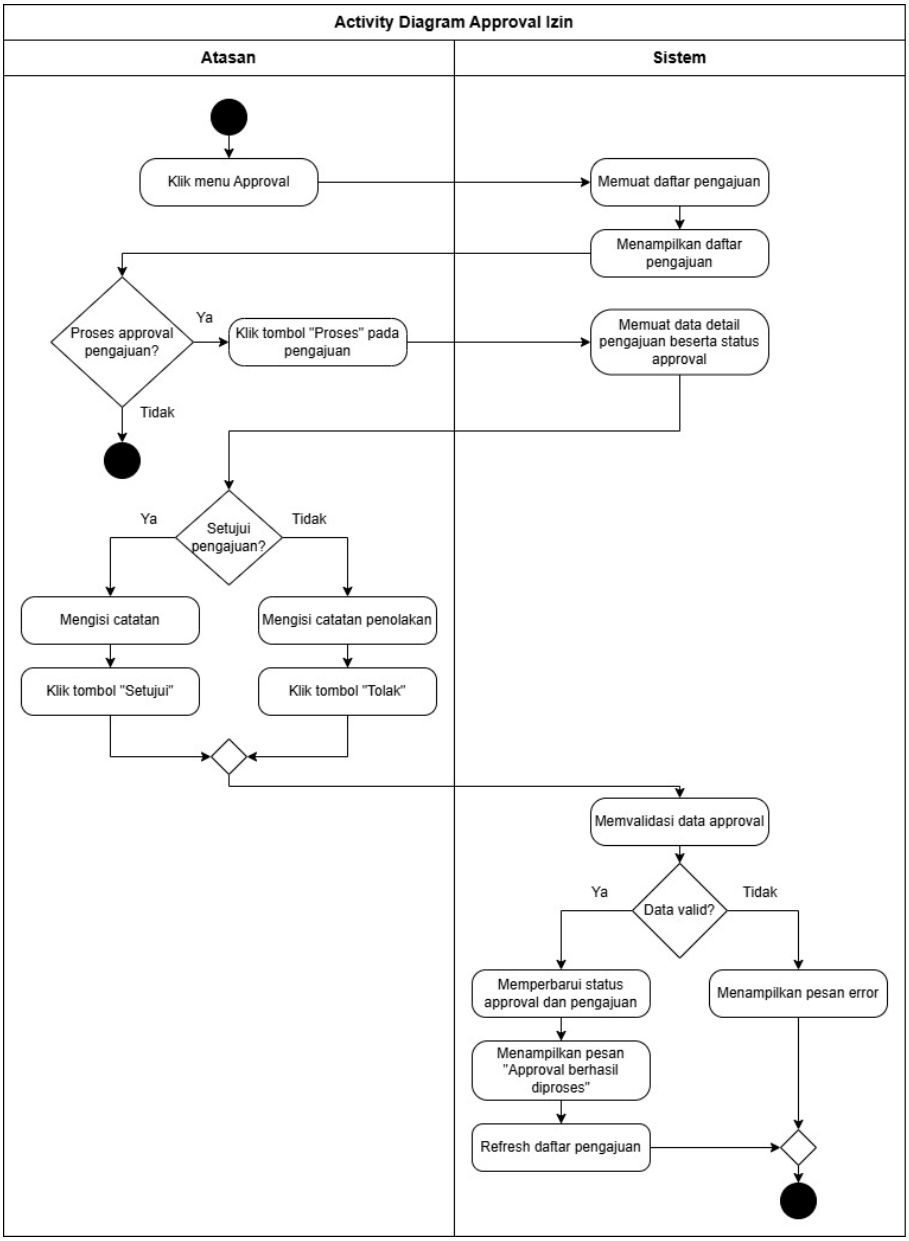
Gambar 49. *Activity Diagram* Pengajuan Izin

Activity Diagram Pengajuan Izin menggambarkan alur aktivitas pengelolaan pengajuan izin oleh aktor Karyawan yang mencakup tiga

operasi utama, yaitu pengajuan izin baru, pelihatan detail pengajuan, dan pembatalan pengajuan. Alur dimulai ketika Karyawan mengakses menu Pengajuan Izin dan sistem memuat serta menampilkan daftar pengajuan izin miliknya, kemudian Karyawan dapat memilih salah satu dari tiga aksi pengelolaan atau keluar dari halaman tanpa operasi. Pada aksi pengajuan izin, Karyawan menekan tombol Ajukan Izin sehingga sistem menampilkan formulir pengajuan izin, kemudian Karyawan mengisi data pengajuan yang mencakup jenis izin, tanggal mulai, tanggal selesai, keterangan, dan bukti pendukung apabila jenis izin yang dipilih mewajibkan bukti, lalu menekan tombol *Submit*. Sistem memvalidasi data dengan memeriksa kelengkapan data, kesesuaian rentang tanggal, dan kesesuaian jumlah hari dengan ketentuan jenis izin, di mana jika valid sistem akan menyimpan pengajuan ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar pengajuan, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi pelihatan detail, Karyawan menekan tombol Detail pada baris pengajuan yang ingin dilihat sehingga sistem memuat dan menampilkan data detail pengajuan beserta status persetujuan dari masing-masing *approver* dalam *approval chain*, sehingga Karyawan dapat memantau perkembangan pengajuannya. Pada aksi pembatalan pengajuan, Karyawan menekan tombol Batalkan pada baris pengajuan yang ingin dibatalkan sehingga sistem menampilkan *modal* konfirmasi, di mana pembatalan hanya dapat dilakukan apabila pengajuan masih berstatus *pending* dan tanggal mulai pengajuan belum tiba. Apabila Karyawan mengonfirmasi pembatalan, sistem mengubah status pengajuan menjadi dibatalkan di basis data, menampilkan pesan keberhasilan, dan memperbarui daftar pengajuan, sedangkan apabila Karyawan membatalkan tindakan, alur berakhir.

16. *Activity Diagram Approval* Izin



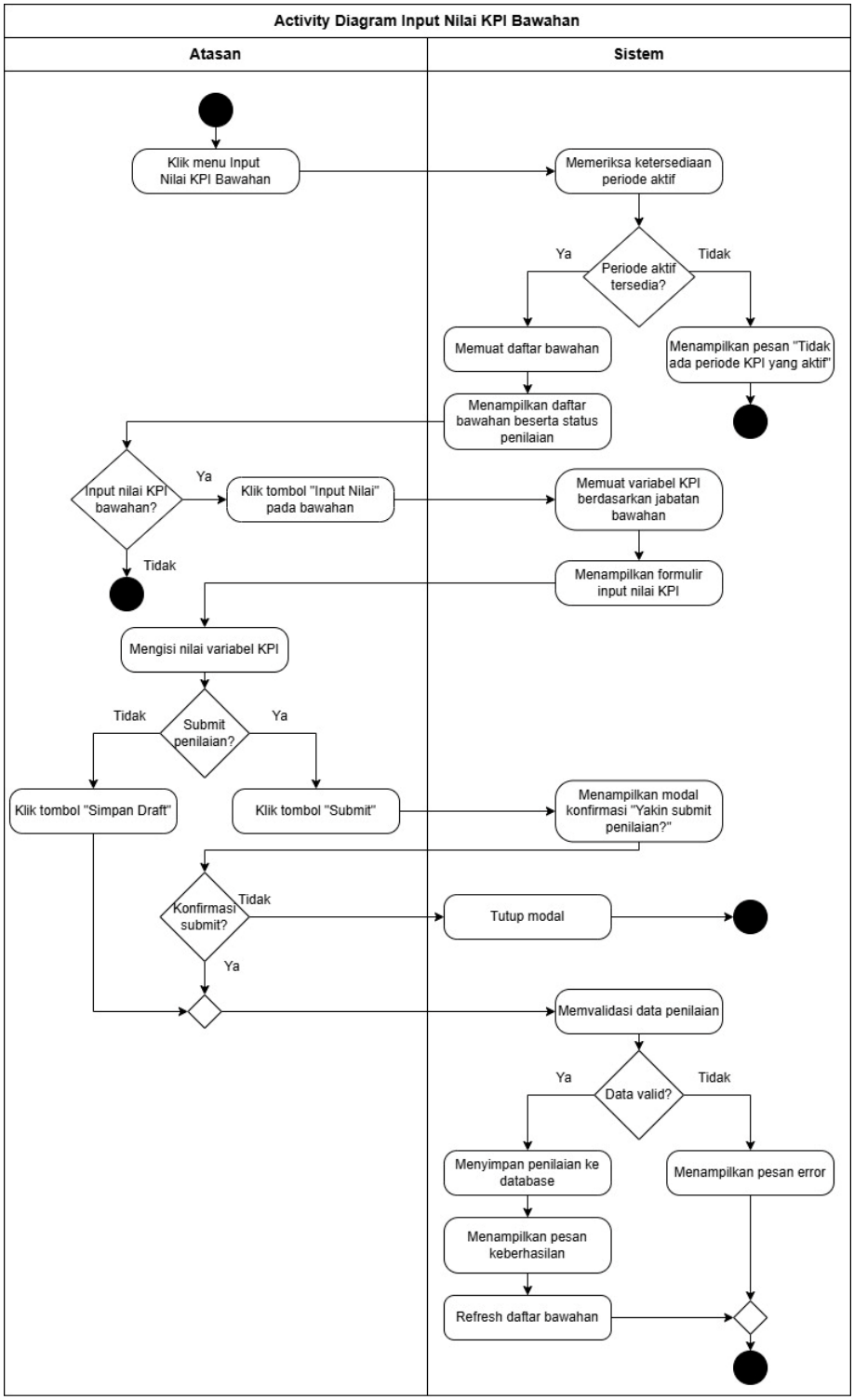
Gambar 50. *Activity Diagram Approval* Izin

Activity Diagram Approval Izin menggambarkan alur aktivitas pemrosesan persetujuan pengajuan izin yang dilakukan oleh aktor Atasan sebagai *approver* dalam *approval chain*. Alur aktivitas dimulai ketika Atasan mengakses menu *Approval*, kemudian sistem memuat dan menampilkan daftar pengajuan yang ditujukan kepadanya untuk diproses. Apabila Atasan tidak melakukan proses *approval*, alur aktivitas berakhir tanpa perubahan.

Apabila Atasan memilih untuk memproses *approval*, Atasan menekan tombol Proses pada baris pengajuan yang ingin diproses sehingga sistem memuat data detail pengajuan beserta status *approval* dari masing-masing *level* dalam *approval chain*. Atasan selanjutnya memutuskan untuk menyetujui atau menolak pengajuan tersebut berdasarkan informasi yang ditampilkan.

Apabila Atasan menyetujui pengajuan, Atasan mengisi catatan persetujuan kemudian menekan tombol Setujui. Sebaliknya, apabila Atasan menolak pengajuan, Atasan mengisi catatan penolakan sebagai alasan penolakan kemudian menekan tombol Tolak. Sistem selanjutnya memvalidasi data *approval* yang dikirimkan, di mana apabila data tidak valid, sistem menampilkan pesan kesalahan dan alur aktivitas berakhir. Apabila data valid, sistem memperbarui status *approval* pada *level* yang bersangkutan dan memperbarui status pengajuan secara keseluruhan, menampilkan pesan keberhasilan, dan memperbarui daftar pengajuan.

17. *Activity Diagram Input* Nilai KPI Bawahan



Gambar 51. *Activity Diagram Input* Nilai KPI Bawahan

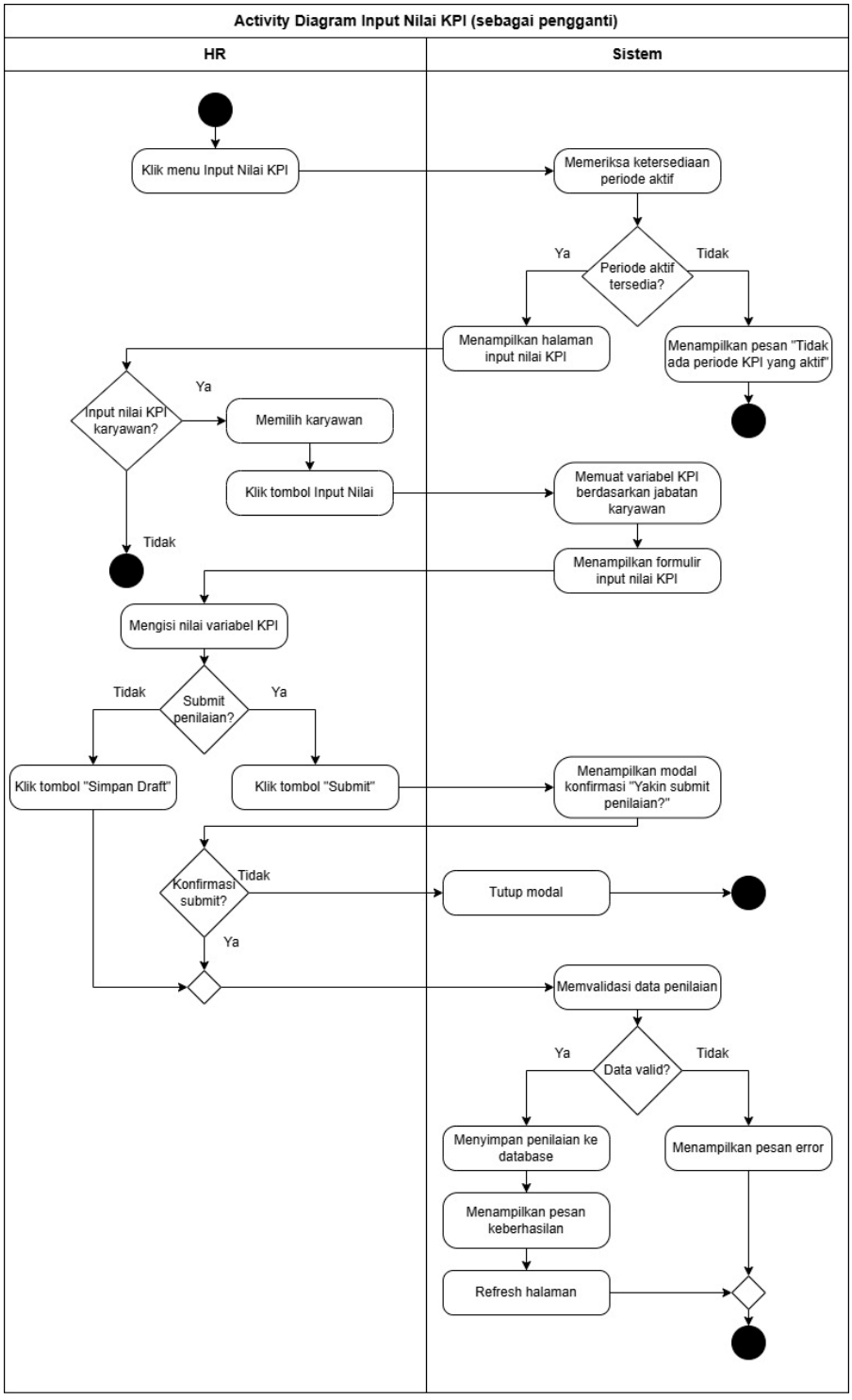
Activity Diagram Input Nilai KPI Bawahan menggambarkan alur aktivitas pemberian nilai KPI yang dilakukan oleh aktor Atasan untuk karyawan yang berada langsung di bawah jabatannya sebagai bawahan.

Alur dimulai ketika Atasan mengakses menu *Input* Nilai KPI Bawahan, kemudian sistem memeriksa ketersediaan periode KPI yang sedang aktif, di mana apabila tidak terdapat periode aktif atau hari ini berada di luar rentang tanggal periode aktif sistem menampilkan pesan bahwa tidak ada periode KPI yang aktif dan alur berakhir. Apabila terdapat periode aktif, sistem memuat daftar bawahan yang harus dinilai beserta status penilaiannya yang mencakup tiga status yaitu belum diisi, *draft*, dan *submitted*.

Pada aksi *input* nilai, Atasan menekan tombol *Input* Nilai pada baris bawahan yang ingin dinilai, kemudian sistem memuat variabel KPI berdasarkan jabatan bawahan tersebut dengan menerapkan mekanisme *Decorator Pattern* yang menggabungkan tiga *layer* variabel yaitu *layer* global, *layer* departemen, dan *layer* jabatan, di mana variabel yang sama pada *layer* yang lebih spesifik akan menggantikan variabel pada *layer* yang lebih umum. Sistem kemudian menampilkan formulir *input* nilai KPI yang berisi daftar variabel yang harus diisi beserta informasi pendukung seperti tipe nilai, rentang nilai, dan bobot variabel.

Atasan mengisi nilai untuk setiap variabel KPI, kemudian memilih untuk menyimpan sebagai *draft* melalui tombol Simpan *Draft*, atau melakukan *submit* penilaian melalui tombol *Submit* yang akan menampilkan *modal* konfirmasi karena penilaian yang sudah di-*submit* akan terkunci dan tidak dapat diubah lagi tanpa *unlock* dari HR. Sistem kemudian memvalidasi data penilaian dengan memeriksa kesesuaian nilai terhadap rentang yang ditentukan untuk setiap variabel, di mana apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir, sedangkan apabila valid sistem menyimpan penilaian ke basis data dengan status sesuai pilihan, menampilkan pesan keberhasilan, dan memperbarui daftar bawahan.

18. *Activity Diagram Input* Nilai KPI (sebagai pengganti)



Gambar 52. *Activity Diagram Input* Nilai KPI (sebagai pengganti)

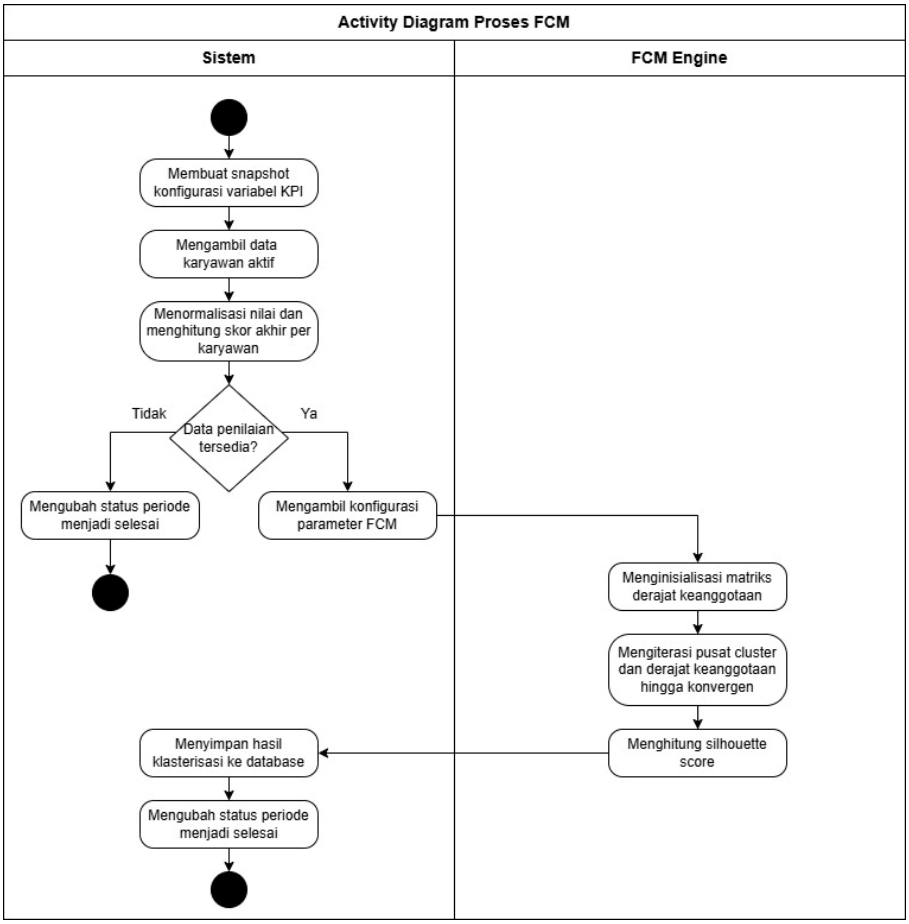
Activity Diagram Input Nilai KPI (sebagai pengganti) menggambarkan alur aktivitas pemberian nilai KPI yang dilakukan oleh aktor HR sebagai pengganti dalam kondisi tertentu, di mana HR dapat

memberikan nilai KPI untuk karyawan apabila atasan yang seharusnya menilai tidak dapat melakukan penilaian, seperti karena cuti panjang, *resign*, atau tidak menyelesaikan penilaian sampai batas waktu yang ditentukan. Alur dimulai ketika HR mengakses menu *Input* Nilai KPI, kemudian sistem memeriksa ketersediaan periode KPI yang sedang aktif, di mana apabila tidak terdapat periode aktif atau hari ini berada di luar rentang tanggal periode aktif sistem menampilkan pesan bahwa tidak ada periode KPI yang aktif dan alur berakhir. Apabila terdapat periode aktif, sistem menampilkan halaman *input* nilai KPI yang berisi panel pemilihan karyawan, kemudian HR dapat memilih untuk melakukan *input* nilai KPI atau keluar dari halaman tanpa aksi.

Pada aksi *input* nilai, HR memilih karyawan yang ingin dinilai melalui kolom pencarian karyawan kemudian menekan tombol *Input* Nilai, sehingga sistem memuat variabel KPI berdasarkan jabatan karyawan tersebut dengan menerapkan mekanisme *Decorator Pattern* yang menggabungkan tiga *layer* variabel yaitu *layer* global, *layer* departemen, dan *layer* jabatan, di mana variabel yang sama pada *layer* yang lebih spesifik akan menggantikan variabel pada *layer* yang lebih umum. Sistem kemudian menampilkan formulir *input* nilai KPI yang berisi daftar variabel yang harus diisi beserta informasi pendukung seperti tipe nilai, rentang nilai, dan bobot variabel.

HR mengisi nilai untuk setiap variabel KPI, kemudian memilih untuk menyimpan sebagai *draft* melalui tombol Simpan *Draft*, atau melakukan *submit* penilaian melalui tombol *Submit* yang akan menampilkan *modal* konfirmasi karena penilaian yang sudah di-*submit* akan terkunci dan tidak dapat diubah lagi. Sistem kemudian memvalidasi data penilaian dengan memeriksa kesesuaian nilai terhadap rentang yang ditentukan untuk setiap variabel, di mana apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir, sedangkan apabila valid sistem menyimpan penilaian ke basis data dengan status sesuai pilihan, menampilkan pesan keberhasilan, dan memperbarui halaman.

19. *Activity Diagram* Proses FCM



Gambar 53. *Activity Diagram* Proses FCM

Activity Diagram Proses FCM menggambarkan alur aktivitas pelaksanaan algoritma *Fuzzy C-Means* untuk klasterisasi kinerja karyawan yang berjalan secara otomatis oleh sistem setelah HR menutup periode penilaian KPI. Proses ini berjalan secara *asynchronous* sehingga tidak melibatkan aktor manusia secara langsung, melainkan dikoordinasikan oleh dua komponen utama yaitu Sistem yang menangani orkestrasi data dan *FCM Engine* yang menangani algoritma klasterisasi inti.

Alur dimulai ketika sistem membuat *snapshot* konfigurasi variabel KPI yang berlaku pada saat itu untuk membekukan konfigurasi variabel sehingga proses klasterisasi tetap konsisten meskipun konfigurasi mengalami perubahan di masa depan. Sistem kemudian mengambil data karyawan aktif, melakukan normalisasi nilai dengan metode *min-*

max berdasarkan arah nilai variabel, dan menghitung skor akhir per karyawan sebagai akumulasi dari hasil perkalian nilai ternormalisasi dengan bobot masing-masing variabel KPI yang berlaku setelah resolusi tiga *layer* melalui mekanisme *Decorator Pattern*. Sistem kemudian memeriksa ketersediaan data penilaian, di mana apabila tidak terdapat data penilaian sistem mengubah status periode menjadi selesai dan alur berakhir tanpa hasil klasterisasi, sedangkan apabila terdapat data penilaian sistem mengambil konfigurasi parameter FCM yang mencakup jumlah *cluster*, pangkat pembobot, jumlah iterasi maksimum, dan ambang batas *error* minimum, lalu mengirimkan data skor akhir karyawan beserta konfigurasi parameter tersebut kepada *FCM Engine*.

FCM Engine memulai algoritma klasterisasi dengan menginisialisasi matriks derajat keanggotaan secara acak menggunakan generator bilangan acak deterministik, kemudian mengiterasi pusat *cluster* dan derajat keanggotaan hingga konvergen, di mana iterasi berhenti ketika perubahan derajat keanggotaan antar iterasi telah berada di bawah ambang batas *error* minimum atau ketika jumlah iterasi telah mencapai batas maksimum. *FCM Engine* kemudian menghitung *silhouette score* untuk mengevaluasi kualitas pengelompokan yang dihasilkan, di mana nilai *silhouette score* yang lebih tinggi mengindikasikan struktur klasterisasi yang lebih baik. Setelah *FCM Engine* menyelesaikan algoritma, sistem menerima hasil klasterisasi yang mencakup *cluster* karyawan, derajat keanggotaan setiap karyawan terhadap setiap *cluster*, dan *silhouette score*, kemudian menyimpan hasil ke basis data dan mengubah status periode menjadi selesai sebagai indikasi bahwa proses klasterisasi telah selesai dan hasil siap diakses oleh HR melalui menu Lihat Hasil Klasterisasi.

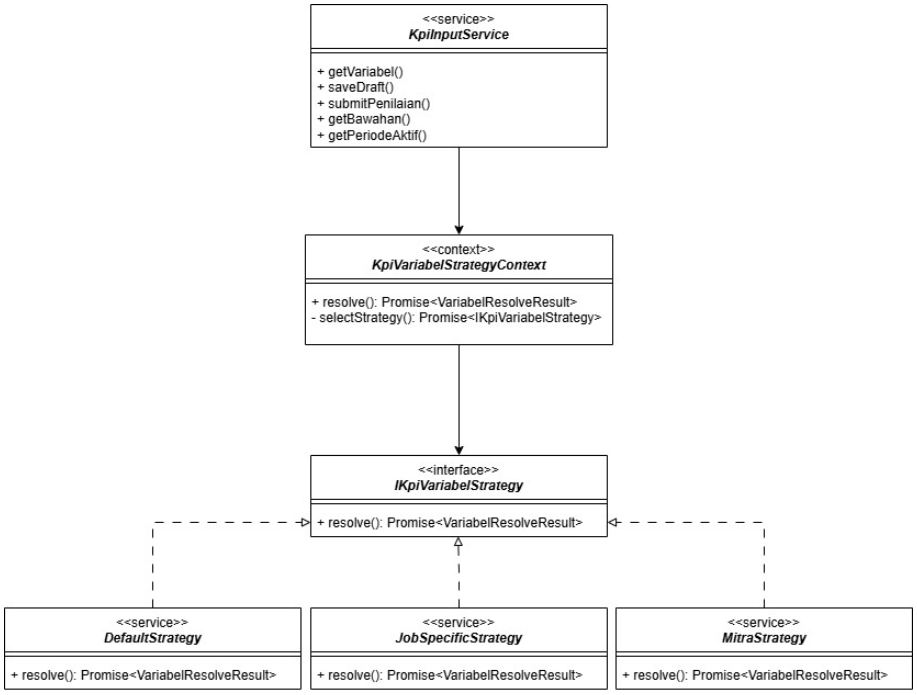
4.2.8 Class Diagram

Sub-bab ini menyajikan rancangan *class diagram* sistem yang dikembangkan, yang digunakan untuk menggambarkan struktur statis sistem meliputi *class-class* yang menyusun arsitektur, atribut dan *method* yang dimiliki, serta relasi antar *class*. Mengingat sistem yang dibangun mengimplementasikan dua pola desain utama yaitu *Strategy Pattern* dan *Decorator Pattern*, serta sebuah algoritma klasterisasi yang mengintegrasikan komponen *reproducibility*, maka *class diagram* pada penelitian ini disajikan dalam empat diagram terpisah agar setiap aspek arsitektur dapat dipahami secara fokus.

Pembagian keempat diagram dilakukan berdasarkan area tanggung jawab yang berbeda, yaitu diagram pertama berfokus pada implementasi *Strategy Pattern* untuk pemilihan strategi resolusi variabel KPI, diagram kedua menampilkan implementasi *Decorator Pattern* yang menyediakan resolusi variabel secara berlapis dan dapat dikomposisi, diagram ketiga menggambarkan arsitektur algoritma *Fuzzy C-Means* beserta komponen *Linear Congruential Generator* yang menjamin sifat deterministik hasil klasterisasi, dan diagram keempat menyajikan gambaran arsitektur *service layer* secara keseluruhan sebagai konteks integrasi antar komponen.

Beberapa keputusan desain perlu disampaikan agar pembacaan diagram menjadi lebih akurat. Pertama, beberapa komponen seperti *KpiVariabelDecorator*, *FcmEngine*, dan *LcgRandomGenerator* direalisasikan dalam bentuk fungsi modular pada implementasi nyata, namun dimodelkan sebagai *class* konseptual pada *class diagram* untuk menampilkan tanggung jawab arsitekturalnya secara visual. Kedua, atribut hasil *dependency injection* dari *framework* NestJS seperti *PrismaService* tidak ditampilkan pada kompartemen atribut karena merupakan komponen infrastruktur yang dikelola oleh *framework*. Ketiga, struktur *entity* dan relasi data yang lebih detail disajikan secara khusus pada sub-bab 4.2.9 (*Entity Relationship Diagram*).

1. *Class Diagram Strategy Pattern*



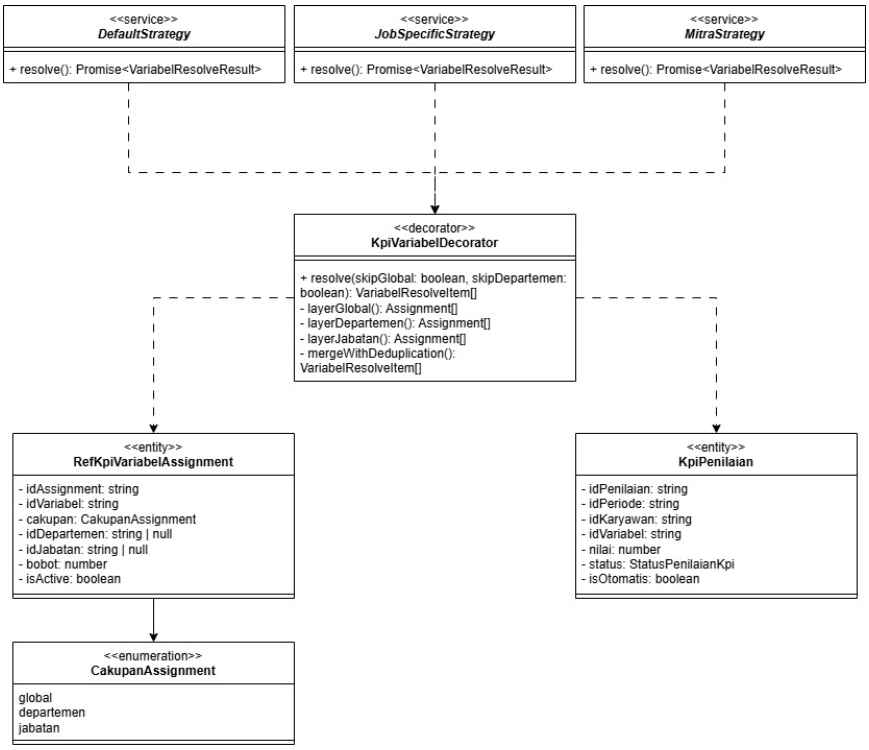
Gambar 54. *Class Diagram Strategy Pattern*

Class diagram pertama ini menggambarkan implementasi *Strategy Pattern* yang digunakan untuk memilih strategi resolusi variabel KPI per karyawan secara dinamis. Pendekatan ini diterapkan karena sistem perlu mengakomodasi tiga jenis karyawan yang memiliki kebutuhan resolusi variabel berbeda, yaitu karyawan dengan jabatan reguler, karyawan tanpa departemen, dan karyawan mitra, sehingga sistem dapat memilih algoritma resolusi yang sesuai pada saat *runtime* tanpa mengubah struktur *class* yang memanggilnya. Diagram ini terdiri dari enam *class*, di mana *IKpiVariabelStrategy* adalah *interface* yang mendefinisikan kontrak resolusi variabel dengan *method* utama *resolve()*, dan tiga *class concrete strategy* mengimplementasikan *interface* tersebut dengan algoritma resolusi yang berbeda, yaitu *DefaultStrategy* untuk karyawan tanpa departemen dengan resolusi dua *layer* (global dan jabatan), *JobSpecificStrategy* untuk karyawan reguler dengan resolusi tiga *layer* penuh (global, departemen, dan jabatan), serta *MitraStrategy* untuk karyawan mitra yang hanya melakukan resolusi pada *layer* jabatan. Selanjutnya, *KpiVariabelStrategyContext* berperan sebagai *context* yang memilih *strategy* yang tepat berdasarkan

karakteristik karyawan kemudian mendelegasikan eksekusi ke *strategy* terpilih, sedangkan *KpiInputService* bertindak sebagai pemanggil *context* tersebut yang merepresentasikan *layer service* yang menerima permintaan dari *controller*.

Relasi antar *class* pada diagram ini menggunakan dua jenis notasi, yaitu tiga *concrete strategy* terhubung ke *interface* melalui relasi *realization* dengan notasi *tree style* yang menunjukkan bahwa ketiganya mengimplementasikan kontrak yang sama, sedangkan hubungan antara *KpiInputService* dengan *KpiVariabelStrategyContext* dan antara *context* dengan *interface* menggunakan notasi *association* yang menggambarkan bahwa *context* menggunakan *strategy* melalui *interface* tanpa terikat pada implementasi tertentu. Desain ini memberikan tiga manfaat utama, yaitu kemudahan untuk menambah *strategy* baru tanpa mengubah *class* yang sudah ada, fleksibilitas dalam memilih algoritma resolusi pada saat *runtime*, dan kejelasan tanggung jawab masing-masing *class* yang sesuai dengan prinsip *Single Responsibility*.

2. *Class Diagram Decorator Pattern*



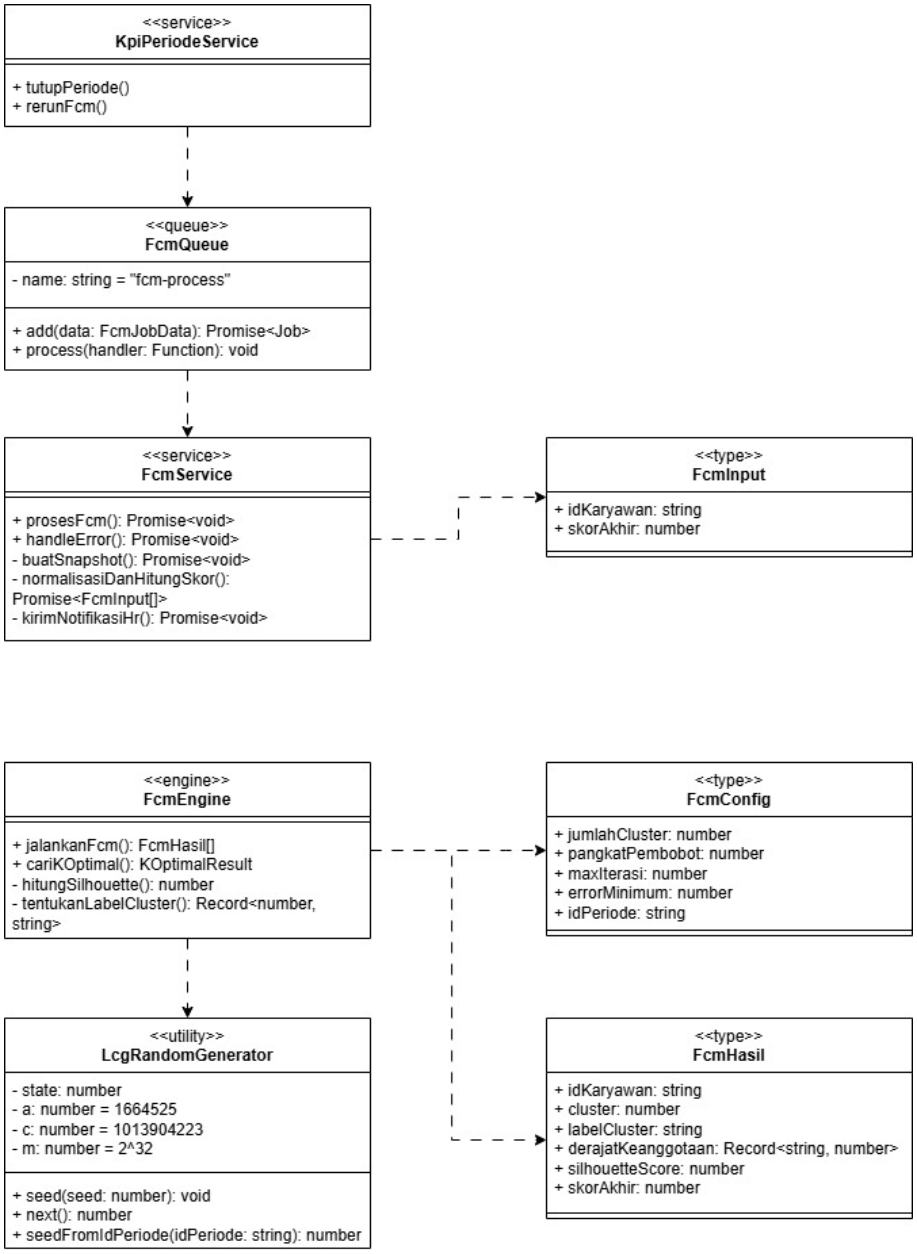
Gambar 55. *Class Diagram Decorator Pattern*

Class diagram kedua ini menggambarkan implementasi *Decorator Pattern* yang digunakan untuk melakukan resolusi variabel KPI secara berlapis. Pola ini diterapkan karena setiap karyawan dapat memiliki variabel KPI yang berasal dari tiga cakupan berbeda yaitu global, departemen, dan jabatan, di mana ketiga cakupan tersebut perlu diakses secara fleksibel dengan dukungan terhadap kemampuan *skip* per *layer* dan deduplikasi prioritas. Diagram ini terdiri dari enam *class*, di mana *KpiVariabelDecorator* adalah *class* utama yang berperan sebagai *decorator* dengan *method* utama *resolve()* yang menerima parameter *skipGlobal* dan *skipDepartemen* untuk mengontrol *layer* mana yang aktif, serta memiliki tiga *method* privat yaitu *layerGlobal()*, *layerDepartemen()*, dan *layerJabatan()*, dan *method mergeWithDeduplication()* yang menggabungkan hasil ketiga *layer* dengan prioritas jabatan terhadap departemen dan departemen terhadap global. Tiga *concrete strategy* yaitu *DefaultStrategy*, *JobSpecificStrategy*, dan *MitraStrategy* ditampilkan kembali sebagai pemanggil *decorator* dengan kombinasi parameter yang berbeda sesuai jenis karyawan, serta dua *entity* yang diakses oleh *decorator* yaitu *RefKpiVariabelAssignment* dan *KpiPenilaian*, dan *class CakupanAssignment* sebagai *enumeration* yang mendefinisikan tiga nilai cakupan yang valid.

Pada implementasi sistem, *KpiVariabelDecorator* direalisasikan dalam bentuk fungsi modular bernama *resolveVariabelDecorator* dan bukan dalam bentuk *class wrapper* sebagaimana pola *Decorator* klasik, untuk menyederhanakan komposisi dan menghindari *overhead* instansiasi karena fungsi ini bersifat *stateless*. Meskipun struktur implementasinya berbeda, peran dan tanggung jawab arsitektural tetap identik yaitu menambahkan kemampuan resolusi *multi-layer* yang dapat dikontrol secara dinamis. Relasi antar *class* pada diagram ini seluruhnya menggunakan notasi *dependency* yang menggambarkan bahwa setiap *concrete strategy* menggunakan *decorator*, *decorator* membaca data dari kedua *entity*, dan *entity RefKpiVariabelAssignment*

memiliki keterkaitan tipe dengan *enumeration CakupanAssignment* melalui atribut cakupan.

3. *Class Diagram* FCM dan LCG



Gambar 56. *Class Diagram* FCM dan LCG

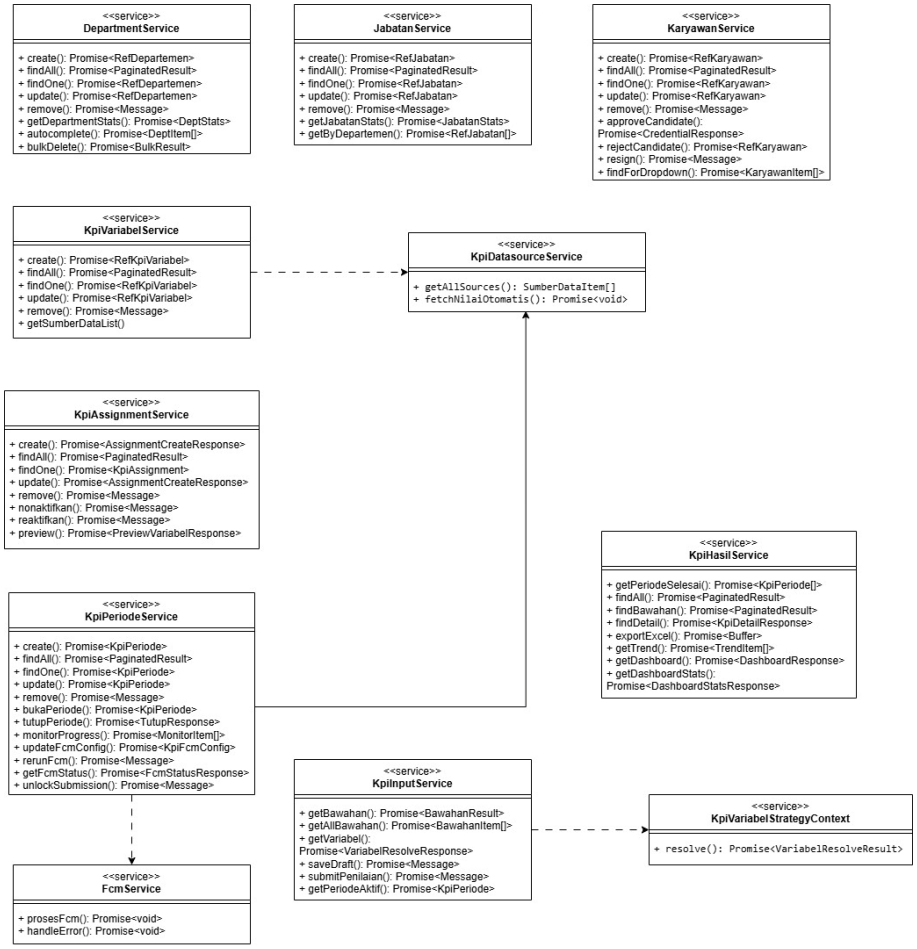
Class diagram ketiga ini menggambarkan arsitektur algoritma *Fuzzy C-Means* yang digunakan untuk klasterisasi kinerja karyawan, beserta komponen *Linear Congruential Generator* yang menjamin sifat deterministik hasil klasterisasi. Algoritma *Fuzzy C-Means* dipilih karena karakteristiknya yang memberikan nilai derajat keanggotaan

terhadap setiap *cluster* yang lebih sesuai dengan realita kinerja karyawan yang umumnya bersifat gradual, sementara integrasi *Linear Congruential Generator* dilakukan untuk mengatasi sifat acak dari inisialisasi matriks partisi awal sehingga hasil klasterisasi dapat direproduksi secara konsisten untuk keperluan audit dan validasi penelitian. Diagram ini terdiri dari delapan *class* yang dikelompokkan menjadi dua bagian, di mana bagian pertama adalah lima *class* utama yang menggambarkan alur eksekusi yaitu *KpiPeriodeService* yang bertindak sebagai *trigger* saat periode KPI ditutup, *FcmQueue* sebagai antrian pemrosesan *asynchronous* yang menerima *job* dari *service* dan mendistribusikannya ke *worker* agar proses klasterisasi tidak menghalangi alur HTTP utama, *FcmService* sebagai *orchestrator* yang melakukan persiapan data berupa *snapshot* konfigurasi, normalisasi nilai, dan penghitungan skor akhir, *FcmEngine* yang berisi logika murni algoritma dengan empat *method* inti yaitu *jalankanFcm()*, *cariKOptimal()*, *hitungSilhouette()*, dan *tentukanLabelCluster()*, serta *LcgRandomGenerator* sebagai *utility* yang menghasilkan bilangan acak deterministik dengan parameter klasik dari *Numerical Recipes*. Bagian kedua adalah tiga tipe data yang berfungsi sebagai kontrak antar komponen yaitu *FcmInput* yang dihasilkan oleh *service* dan dikonsumsi oleh *engine*, *FcmConfig* yang membawa parameter konfigurasi algoritma, serta *FcmHasil* yang merupakan keluaran akhir berisi *cluster*, label, derajat keanggotaan, dan *Silhouette Score* per karyawan.

Pada implementasi sistem, *FcmEngine* dan *LcgRandomGenerator* direalisasikan dalam bentuk fungsi modular pada *file* terpisah dan bukan sebagai *class instantiable* untuk menjaga sifat *stateless* dan menghindari *overhead* instansiasi, namun dimodelkan sebagai *class* konseptual pada *class diagram* untuk menampilkan tanggung jawab arsitektural dan komposisi antar komponen secara visual. Pemisahan tegas antara *FcmService* sebagai *orchestrator* yang berinteraksi dengan *database* dan *FcmEngine* sebagai algoritma murni tanpa *side effect* memberikan dua manfaat utama, yaitu kemudahan dalam pengujian

unit karena *engine* dapat dijalankan tanpa *dependency database* serta kejelasan tanggung jawab yang sesuai dengan prinsip *Separation of Concerns*. Sifat deterministik yang diberikan oleh *LcgRandomGenerator* dijamin melalui pembangkitan *seed* dari *idPeriode* menggunakan 8 karakter heksadesimal pertama, sehingga setiap periode menghasilkan urutan bilangan acak yang sama setiap kali algoritma dijalankan ulang.

4. *Class Diagram Service Architecture Overview*



Gambar 57. *Class Diagram Service Architecture Overview*

Class diagram keempat ini menggambarkan gambaran arsitektur *service layer* secara keseluruhan, di mana ketiga pola desain yang telah dijelaskan pada sub-bab sebelumnya berintegrasi menjadi satu sistem yang utuh. Diagram ini berfokus pada bagaimana setiap *service* berinteraksi melalui mekanisme *dependency injection* yang dikelola oleh *framework* NestJS, yang menjadi salah satu bukti penerapan

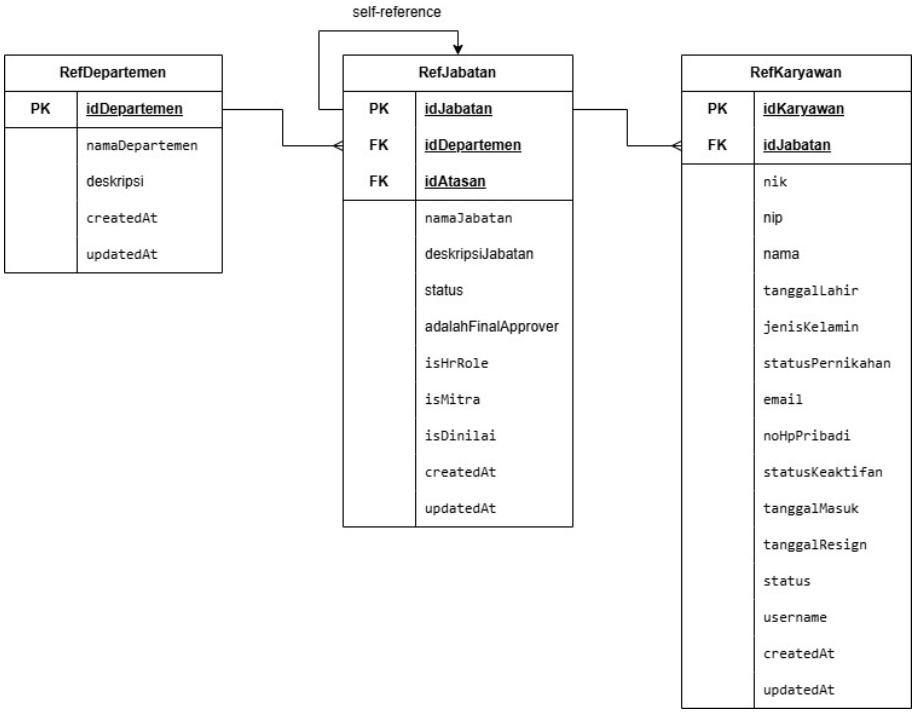
composable architecture. Diagram ini terdiri dari sebelas *class* dengan tanggung jawab spesifik, yaitu *DepartmentService*, *JabatanService*, dan *KaryawanService* untuk mengelola *master data* struktur organisasi, *KpiVariabelService* untuk pengelolaan *master* variabel KPI, *KpiAssignmentService* untuk *assignment* variabel ke *scope* tertentu, *KpiPeriodeService* untuk *lifecycle* periode KPI beserta konfigurasi FCM, *KpiInputService* untuk *input* penilaian, *FcmService* sebagai *orchestrator* proses klasterisasi *asynchronous*, dan *KpiHasilService* untuk akses hasil klasterisasi, serta dua *class* sebagai *dependency* yaitu *KpiVariabelStrategyContext* dan *KpiDatasourceService* yang menyediakan integrasi dengan sumber data otomatis.

Relasi antar *service* pada diagram ini ditampilkan berdasarkan *dependency injection* yang terjadi melalui *constructor* sesuai *pattern* NestJS, di mana terdapat empat relasi yaitu *KpiPeriodeService* menggunakan *FcmService* melalui *Bull Queue* untuk pemrosesan *asynchronous*, *KpiPeriodeService* menggunakan *KpiDatasourceService* untuk pengambilan nilai otomatis, *KpiInputService* menggunakan *KpiVariabelStrategyContext* untuk resolusi variabel per karyawan, serta *KpiVariabelService* menggunakan *KpiDatasourceService* untuk validasi sumber data. Beberapa keputusan desain perlu disampaikan, yaitu parameter *method* dengan tipe DTO tidak ditampilkan pada *signature* untuk menjaga keterbacaan, kompartemen atribut sengaja dikosongkan karena atribut implementasi hanya berupa *dependency injection* seperti *PrismaService* yang bukan *state* domain, dan *entity* serta struktur data dipisahkan ke sub-bab 4.2.9 dalam bentuk *Entity Relationship Diagram*.

4.2.9 Entity Relationship Diagram

Sub-bab ini menyajikan rancangan *Entity Relationship Diagram* (ERD) sistem yang dikembangkan, yang digunakan untuk menggambarkan struktur *entity* dalam basis data meliputi atribut yang dimiliki setiap *entity*, *primary key*, *foreign key*, beserta relasi antar *entity*. Mengingat sistem yang dibangun terdiri dari beberapa modul fungsional dengan karakteristik domain yang berbeda, ERD pada penelitian ini dipecah menjadi lima sub-ERD berdasarkan area tanggung jawab masing-masing modul, yaitu sub-ERD pertama menyajikan struktur *entity* untuk organisasi dan *user management* yang menjadi fondasi referensi bagi modul-modul lainnya, sub-ERD kedua menampilkan struktur *entity* untuk modul KPI yang merupakan inti dari penelitian ini, sub-ERD ketiga menggambarkan struktur *entity* untuk modul autentikasi dan *permission*, sub-ERD keempat menyajikan struktur *entity* untuk modul presensi dan jadwal kerja karyawan, serta sub-ERD kelima menampilkan struktur *entity* untuk modul *approval*, izin, dan notifikasi yang menangani alur pengajuan dan pemberitahuan dalam sistem.

1. ERD Organisasi & *User Management*



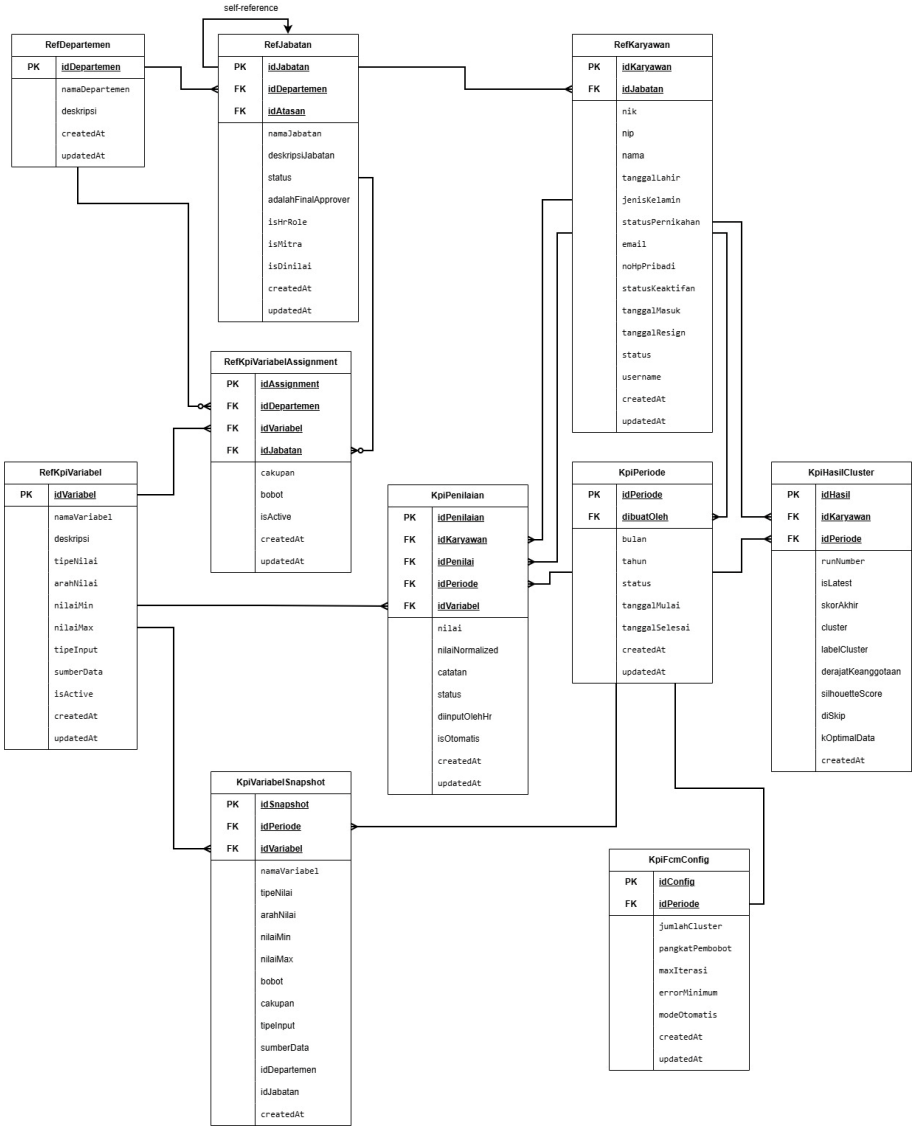
Gambar 58. ERD Organisasi & *User Management*

Sub-ERD pertama ini menggambarkan struktur *entity* yang menjadi fondasi referensi bagi seluruh modul sistem, yaitu data organisasi perusahaan dan data karyawan sebagai pengguna sistem. Sub-ERD ini disusun pertama karena ketiga *entity* di dalamnya menjadi acuan bagi hampir seluruh *entity* di sub-ERD berikutnya.

Sub-ERD ini terdiri dari tiga *entity* utama, yaitu *RefDepartemen* yang menyimpan data *master* departemen sebagai dasar pengelompokan jabatan dan *assignment* variabel KPI pada cakupan departemen, *RefJabatan* yang menyimpan data *master* jabatan dan merupakan *entity* paling krusial karena tidak hanya merepresentasikan posisi karyawan tetapi juga menjadi dasar pemilihan strategi resolusi variabel KPI melalui *flag isMitra* dan *isDinilai* serta menentukan peran HR melalui *flag isHrRole*, dan *RefKaryawan* yang menyimpan data *master* karyawan mencakup informasi identitas, jabatan yang diduduki, status keaktifan, serta atribut autentikasi untuk *login* ke sistem. Selain itu, *RefJabatan* juga memiliki relasi *self-reference* melalui atribut *idAtasan* untuk merepresentasikan hierarki atasan-bawahan dalam struktur organisasi.

Pola relasi pada sub-ERD ini mengikuti hierarki organisasi dari yang paling abstrak ke yang paling konkret, di mana satu departemen membawahi banyak jabatan, satu jabatan dapat diduduki oleh banyak karyawan, dan satu jabatan juga dapat membawahi banyak jabatan lain melalui relasi *self-reference* yang memungkinkan sistem membangun struktur organisasi berjenjang yang fleksibel tanpa batasan kedalaman hierarki.

2. ERD KPI



Gambar 59. ERD KPI

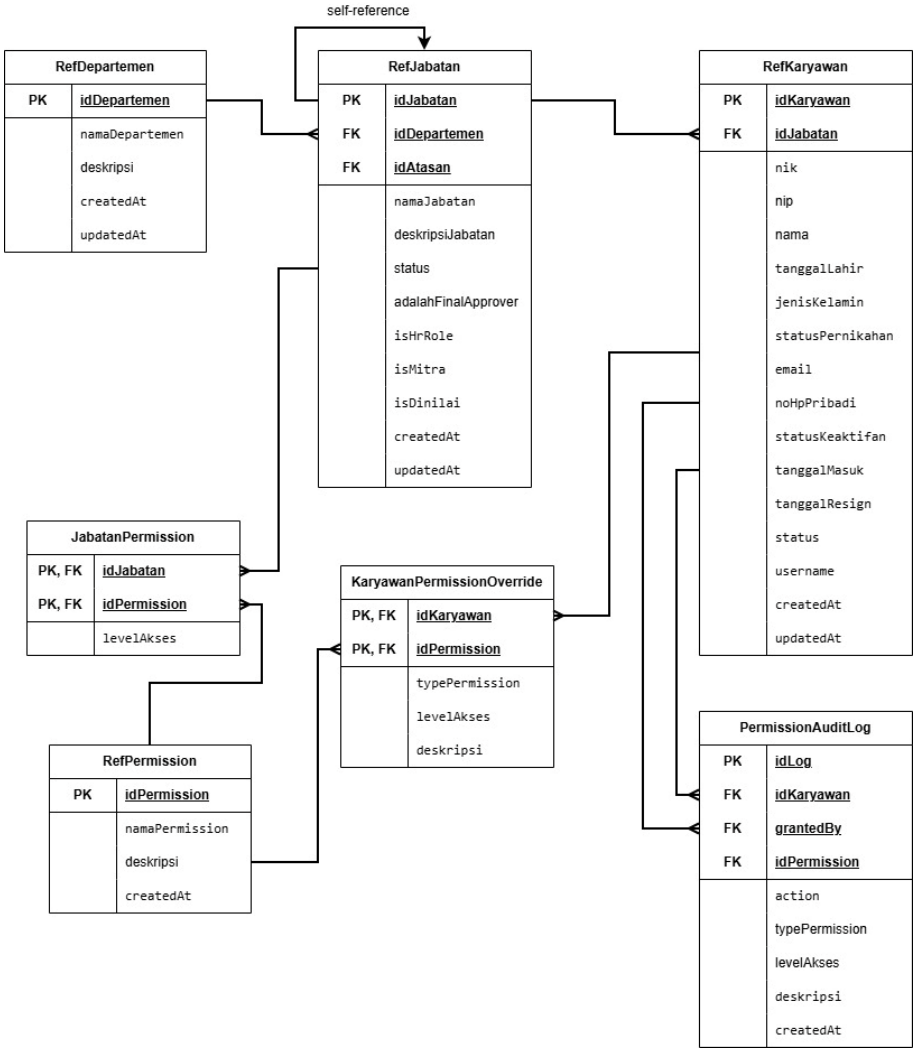
Sub-ERD kedua ini menggambarkan struktur *entity* untuk modul KPI yang merupakan inti dari penelitian skripsi ini, yang mencakup tujuh *entity* utama untuk mendukung seluruh alur penilaian KPI mulai dari definisi variabel, *assignment* variabel ke *scope* tertentu, periode penilaian, hingga hasil klasterisasi karyawan menggunakan algoritma *Fuzzy C-Means*. Pada bagian *master* variabel, *RefKpiVariabel* menyimpan *master* variabel KPI yang dapat ditambahkan dan dikonfigurasi secara dinamis oleh HR tanpa perlu mengubah kode program, di mana setiap variabel memiliki atribut tipe nilai, arah nilai, batas minimum dan maksimum, serta tipe *input* yang menentukan

apakah nilai diinput manual atau diambil secara otomatis, sedangkan *RefKpiVariabelAssignment* menjadi *entity* penghubung yang merepresentasikan implementasi tiga cakupan *assignment* (global, departemen, dan jabatan) melalui atribut cakupan beserta *idDepartemen* dan *idJabatan* yang bersifat opsional. Pada bagian periode dan konfigurasi, *KpiPeriode* menyimpan periode penilaian KPI beserta status *lifecycle* periode mulai dari *draft*, aktif, ditutup, *processing*, hingga selesai atau *error*, sedangkan *KpiFcmConfig* menyimpan parameter konfigurasi algoritma *Fuzzy C-Means* per periode yang mencakup jumlah *cluster*, pangkat pembobot, maksimum iterasi, *error* minimum, dan *flag* mode otomatis untuk pencarian *k* optimal. Pada bagian data *transactional*, *KpiPenilaian* mencatat hasil *input* penilaian per karyawan per variabel per periode beserta nilai aktual, nilai normalisasi, dan penilai yang melakukan *input*, *KpiVariabelSnapshot* menyimpan *snapshot* konfigurasi variabel pada saat periode ditutup sehingga hasil klasterisasi tetap konsisten meskipun konfigurasi *master* variabel diubah di kemudian hari (krusial untuk *reproducibility*), dan *KpiHasilCluster* menyimpan hasil klasterisasi karyawan per periode beserta *cluster*, label *cluster*, derajat keanggotaan *fuzzy*, dan nilai *Silhouette Coefficient*.

Pola relasi pada sub-ERD ini menunjukkan alur hidup data KPI yang mengalir dari konfigurasi menuju eksekusi, di mana *KpiPeriode* berperan sebagai *entity* pusat yang menjadi acuan bagi sebagian besar *entity transactional*, sehingga satu periode dapat memiliki banyak penilaian, banyak *snapshot* variabel, dan banyak hasil *cluster* sesuai jumlah karyawan yang dinilai dan jumlah *run* klasterisasi yang dilakukan. *Entity KpiPenilaian* memiliki dua relasi terpisah ke *RefKaryawan* yaitu sebagai karyawan yang dinilai dan sebagai karyawan yang menilai, di mana kedua peran tersebut dapat berbeda karena penilaian dilakukan oleh atasan atau oleh HR sebagai pengganti atasan jika diperlukan, sedangkan *KpiVariabelSnapshot* menyimpan atribut *idDepartemen* dan *idJabatan* sebagai data historis tanpa relasi

formal ke *entity* referensi agar *snapshot* tetap valid sebagai catatan historis meskipun data *master* organisasi mengalami perubahan di kemudian hari.

3. ERD *Auth* & Permission



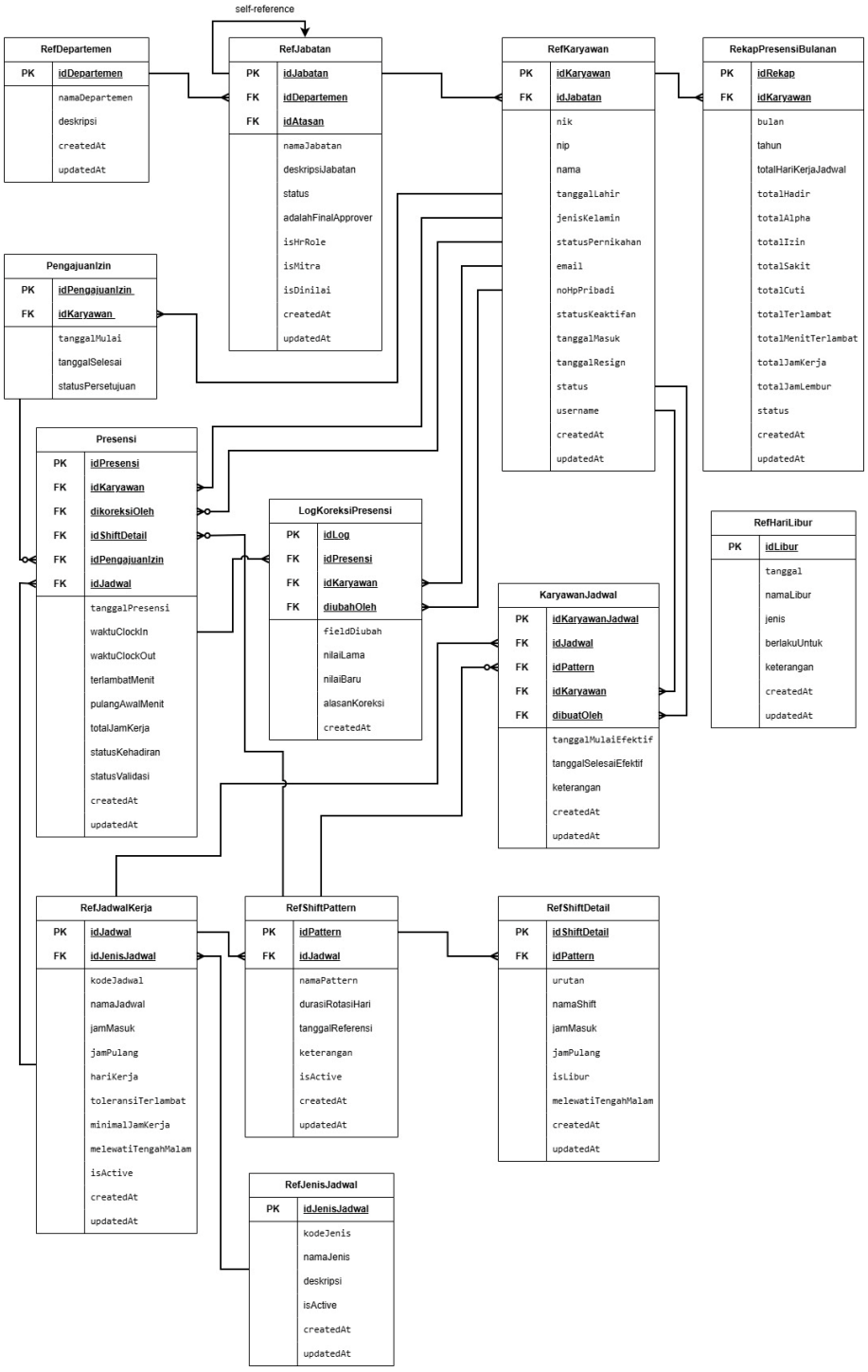
Gambar 60. ERD *Auth* & Permission

Sub-ERD ketiga ini menggambarkan struktur *entity* untuk sistem autentikasi dan otorisasi sistem, yang menerapkan pola *permission* berlapis untuk memungkinkan pengelolaan hak akses secara fleksibel baik melalui jabatan secara *default* maupun melalui *override* per individu karyawan jika diperlukan. *RefPermission* menyimpan *master permission* yang mendefinisikan jenis-jenis hak akses dalam sistem seperti hak untuk mengelola KPI, menginput penilaian, atau mengakses modul tertentu. *JabatanPermission* adalah *entity junction* yang menjadi

lapis pertama pengaturan *permission* yang mengaitkan setiap jabatan dengan *permission* yang dimilikinya beserta *level* akses berupa *bitmask* untuk operasi *read*, *write*, dan *delete*, sehingga semua karyawan yang menduduki jabatan tertentu akan secara otomatis mewarisi seluruh *permission* yang ditetapkan pada jabatan tersebut. *KaryawanPermissionOverride* adalah *entity* lapis kedua yang memungkinkan HR memberikan *permission* khusus per individu karyawan baik berupa penambahan *permission* yang tidak dimiliki oleh jabatannya maupun pencabutan *permission default* jabatan, di mana atribut *typePermission* membedakan kedua peran ini (*true* berarti *grant* dan *false* berarti *revoke*). *PermissionAuditLog* mencatat seluruh aktivitas perubahan *permission* untuk keperluan akuntabilitas dan *traceability* mencakup informasi tentang karyawan yang *permission*-nya diubah, jenis aksi yang dilakukan, *level* akses yang berubah, HR yang melakukan aksi, serta alasan perubahan.

Pola relasi pada sub-ERD ini mencerminkan arsitektur *permission* berlapis yang dianut sistem, di mana *RefPermission* berperan sebagai *master* yang menjadi acuan bagi tiga *entity transactional* yaitu *JabatanPermission*, *KaryawanPermissionOverride*, dan *PermissionAuditLog*. *Entity JabatanPermission* dan *KaryawanPermissionOverride* masing-masing menggunakan *composite primary key* sebagai mekanisme untuk menjamin tidak ada duplikasi *permission* per jabatan atau per karyawan, sedangkan *entity PermissionAuditLog* memiliki dua relasi terpisah ke *RefKaryawan* yaitu sebagai karyawan yang *permission*-nya berubah dan sebagai karyawan HR yang melakukan aksi perubahan, agar *audit log* dapat menjawab pertanyaan "siapa mengubah *permission* siapa" secara akurat.

4. ERD Presensi & Jadwal Kerja



Gambar 61. ERD Presensi & Jadwal Kerja

Sub-ERD keempat ini menggambarkan struktur *entity* untuk modul presensi dan jadwal kerja karyawan dan merupakan sub-ERD paling kompleks dalam sistem karena mencakup hierarki konfigurasi jadwal yang berjenjang, data presensi harian dengan banyak konteks, serta

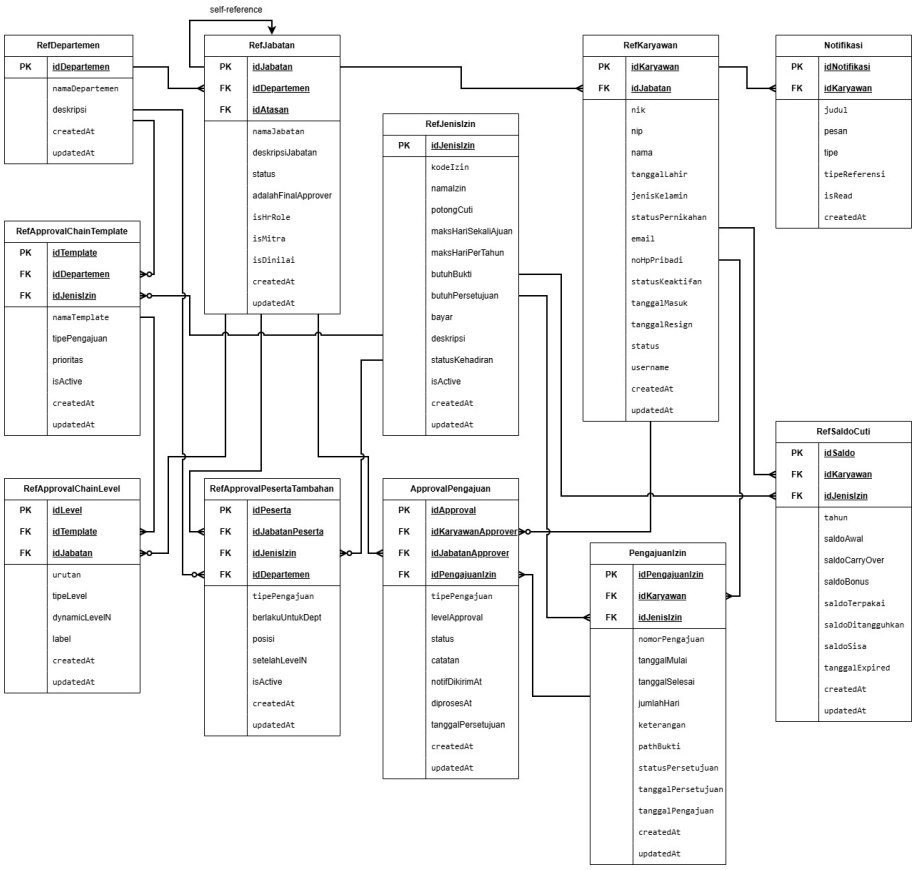
mekanisme koreksi dan rekapitulasi presensi bulanan. Pada bagian *master* jadwal, *RefJenisJadwal* menyimpan klasifikasi jenis jadwal yang didukung sistem yaitu reguler, *shift*, *flexible*, *project*, dan *hybrid*, *RefJadwalKerja* menyimpan detail konfigurasi jadwal seperti jam masuk, jam pulang, hari kerja, toleransi keterlambatan, dan minimal jam kerja, sedangkan *RefShiftPattern* dan *RefShiftDetail* membentuk hierarki tambahan untuk jadwal tipe *shift* di mana *pattern* mendefinisikan durasi rotasi *shift* dan *detail* mendefinisikan urutan *shift* dalam satu siklus rotasi. *RefHariLibur* adalah *entity standalone* yang menyimpan kalender hari libur tanpa relasi formal dengan *entity* lain karena dipakai oleh *business logic* saat menghitung presensi.

Pada bagian *assignment* dan presensi, *KaryawanJadwal* adalah *entity junction* yang mengaitkan karyawan dengan jadwal kerja yang berlaku untuknya beserta tanggal mulai dan tanggal selesai efektif, sedangkan *Presensi* adalah *entity* inti yang mencatat presensi harian karyawan beserta informasi waktu *clock in* dan *clock out*, total jam kerja, keterlambatan, status kehadiran, sumber presensi, lokasi GPS, foto, serta status validasi. Pada bagian audit dan rekapitulasi, *LogKoreksiPresensi* mencatat seluruh perubahan data presensi untuk keperluan jejak audit mencakup *field* yang diubah, nilai lama, nilai baru, alasan koreksi, dan HR yang melakukan perubahan, sedangkan *RekapPresensiBulanan* menyimpan ringkasan presensi per karyawan per bulan untuk keperluan pelaporan dan integrasi ke modul lain di masa mendatang.

Pola relasi pada sub-ERD ini menunjukkan dua aliran utama, yaitu aliran konfigurasi jadwal yang berjenjang dari *RefJenisJadwal* menuju *RefJadwalKerja*, kemudian menuju *RefShiftPattern* dan *RefShiftDetail* untuk jadwal tipe *shift*, dan akhirnya teraplikasikan ke karyawan melalui *KaryawanJadwal*, serta aliran pencatatan presensi yang berpusat pada *entity Presensi* dengan banyak relasi opsional ke jadwal yang berlaku, *shift detail* jika *applicable*, dan pengajuan izin jika presensi tersebut terkait dengan izin yang disetujui. Beberapa *entity*

dalam sub-ERD ini memiliki dua relasi terpisah ke *RefKaryawan* yang merepresentasikan dua peran berbeda, yaitu *KaryawanJadwal* memiliki relasi sebagai karyawan yang di-*assign* jadwal dan sebagai HR yang membuat penjadwalan, *Presensi* memiliki relasi sebagai karyawan pemilik presensi dan sebagai HR yang melakukan koreksi presensi, sedangkan *LogKoreksiPresensi* memiliki relasi sebagai karyawan yang presensi-nya dikoreksi dan sebagai HR yang melakukan koreksi.

5. ERD *Approval*, Izin & Notifikasi



Gambar 62. ERD *Approval*, Izin & Notifikasi

Sub-ERD kelima ini menggambarkan struktur *entity* untuk modul *approval*, izin, dan notifikasi sistem yang menerapkan pola *approval chain* yang konfiguratif, di mana HR dapat mengatur alur persetujuan secara dinamis sesuai dengan kebutuhan organisasi tanpa perlu mengubah kode program. Pada bagian *master* izin dan saldo, *RefJenisIzin* menyimpan klasifikasi jenis izin yang didukung sistem beserta aturan-aturannya seperti maksimum hari per ajuan, kewajiban

upload bukti, dan tipe pembayaran, sedangkan *RefSaldoCuti* menyimpan saldo cuti per karyawan per jenis izin per tahun mencakup saldo awal, *carry over* dari tahun sebelumnya, bonus, saldo terpakai, dan saldo sisa. Pada bagian pengajuan, *PengajuanIzin* menyimpan data pengajuan izin oleh karyawan beserta tanggal mulai, tanggal selesai, jumlah hari, keterangan, dan status persetujuan, di mana setiap pengajuan memiliki nomor pengajuan unik yang dapat dilacak melalui sistem.

Pada bagian konfigurasi *approval chain*, *RefApprovalChainTemplate* menyimpan *template chain approval* per tipe pengajuan yang dapat dispesifikasikan untuk jenis izin tertentu atau departemen tertentu, *RefApprovalChainLevel* menyimpan *level-level approval* dalam suatu *template* beserta tipe *level* yang menentukan apakah *approver* adalah jabatan spesifik, *role* HR, atau jabatan dinamis berdasarkan hierarki bawahan-atasan, sedangkan *RefApprovalPesertaTambahan* menyimpan peserta *approval* tambahan di luar *chain template default* yang dapat ditempatkan sebelum *chain*, sesudah *chain*, atau setelah *level* tertentu. Pada bagian eksekusi *approval*, *ApprovalPengajuan* mencatat status *approval* per *level* untuk setiap pengajuan mencakup *level approval*, jabatan *approver*, karyawan *approver* yang melakukan aksi, status, catatan, serta waktu pemrosesan, sehingga satu pengajuan akan menghasilkan beberapa *record approval* sesuai jumlah *level* dalam *chain* yang berlaku. *Notifikasi* menyimpan pemberitahuan yang ditujukan kepada karyawan mencakup judul, pesan, tipe notifikasi, serta status sudah dibaca atau belum, dan berperan sebagai penghubung antar modul karena notifikasi dapat dipicu oleh berbagai aktivitas sistem seperti pengajuan izin yang membutuhkan *approval*, hasil klasterisasi KPI yang sudah selesai diproses, atau perubahan *permission* yang melibatkan karyawan.

Pola relasi pada sub-ERD ini menunjukkan arsitektur *approval chain* yang fleksibel, di mana *RefApprovalChainTemplate* menjadi *entity* konfigurasi utama yang memiliki banyak *level* dan dapat

dilengkapi peserta tambahan. Saat pengajuan diajukan oleh karyawan, sistem akan membaca *template* yang sesuai dan menghasilkan beberapa *record ApprovalPengajuan* sesuai dengan *level* dalam *template*, di mana setiap *record approval* merujuk pada jabatan *approver* yang menentukan siapa yang berhak menyetujui pada *level* tersebut dan opsional menyimpan karyawan *approver* spesifik yang melakukan aksi persetujuan atau penolakan. Selain itu, *RefSaldoCuti* memiliki relasi ganda ke *RefKaryawan* dan *RefJenisIzin* dengan kombinasi *unique constraint* yang mencakup tahun, sehingga setiap karyawan hanya memiliki satu *record* saldo per jenis izin per tahun untuk memungkinkan sistem melacak saldo cuti secara akurat sekaligus memudahkan perhitungan saldo lintas tahun.

4.3 Implementasi

Sub-bab ini menjelaskan implementasi sistem yang telah dirancang pada sub-bab sebelumnya, mencakup penerapan pola desain *Strategy* dan *Decorator* yang menjadi inti arsitektur sistem, pra-pemrosesan data sebelum klasterisasi, implementasi algoritma *Fuzzy C-Means* beserta komponen pendukungnya, serta perhitungan *Silhouette Coefficient* sebagai metrik validasi kualitas klasterisasi. Implementasi sistem dibangun menggunakan *framework* NestJS pada sisi *backend* dengan bahasa pemrograman *TypeScript*, di mana arsitektur *dependency injection* yang dianut oleh *framework* digunakan secara konsisten untuk mengelola komposisi antar komponen, sedangkan basis data yang digunakan adalah PostgreSQL dengan ORM Prisma sebagai abstraksi akses data, dan pemrosesan klasterisasi yang berpotensi memakan waktu dijalankan secara *asynchronous* menggunakan *Bull Queue* agar tidak menghalangi alur permintaan utama dari pengguna.

4.3.1 Implementasi *Strategy Pattern*

Sub-bab ini menjelaskan implementasi *Strategy Pattern* pada sistem yang telah dirancang pada sub-bab 4.2.8. Pola ini diterapkan untuk memilih strategi resolusi variabel KPI per karyawan secara dinamis pada saat *runtime*, di mana implementasi dilakukan menggunakan *framework* NestJS dengan bahasa *TypeScript*.